

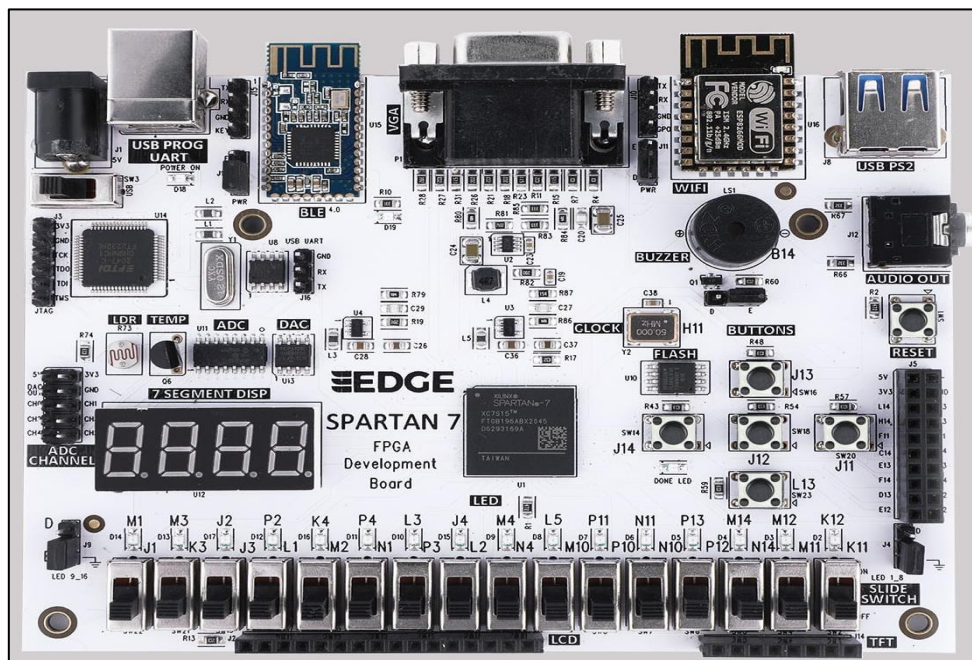
DAYANANDA SAGAR UNIVERSITY

Devarahaggalahalli, Harohalli, Kanakapura Main Road,
Ramnagara District – 562 112



SCHOOL OF ENGINEERING

Department of Electronics & Communication Engineering



Digital Logic Design Lab Manual (Course Code: 22EC2303)

Prepared By

Dr. Gayathri K M
Associate Professor

Mrs. Manasa K R
Assistant Professor

Vision

To create innovative Engineers and Entrepreneurs with technological excellence, professional commitment and social responsibility for serving national and global needs.

Mission

Inculcate Academic Excellence through innovative teaching and learning processes and espousing appropriate pedagogical parameters.

Reinforce the Students with desired technical aptitude, entrepreneurial and leadership skill sets enabling them to face the challenges of globalization and technological sophistication.

Initiation with understanding the psychology of students, socio-cultural aspects of the bidirectional learners, vitality of interdisciplinary approach, value addition through interactive and collaborative learning. This is followed by systematic and sequential implementation of syllabus upgradations on par with industrial revolution.

Program Educational Objectives (PEO) - UG

- Our Graduates will have in-depth knowledge of Electronics and Communication Engineering with promising professional careers in private and public sector or higher education.
- Our Graduates will be successful in solving Engineering Problems with innovative ideas and acquire managerial skills for desired outcomes.
- Our Graduates will have the motivation for perennial learning and progress their careers by inculcating interpersonal, leadership and social skills.

Program Specific Outcome (PSOs) - UG

- Apply the knowledge of Electronics and Communication to solve Engineering Problems in various domains of Engineering Sciences.
- Adopting analytical skills and complementing the cross-cutting technology to arrive at optimum solutions for Engineering Problems.
- Adaptability to dynamic work environment to address the societal needs with ethical approach.

Program Outcomes (PO's)

- A graduate of the Electronics and Communication Engineering Program will demonstrate:
- **PO1 - Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2 - Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences
- **PO3 - Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4 - Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5 - Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6 - The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7 - Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8 - Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9 - Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10 - Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11 - Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.
- **PO12 - Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

COURSE OBJECTIVES:

- To translate the elements of digital logic functions to digital system abstractions using Verilog.
- To illustrate simplification of Boolean expressions using Karnaugh.
- To model combinational logic circuits for arithmetic operations and logical operations.
- To characterise, analyse and model bi-stable elements such as latches and flip-flops.
- To outline the concept of Mealy Model, Moore Model and apply FSM for digital design.

COURSE OUTCOMES:

Course Outcome	Description	Bloom's Taxonomy Level
At the end of the course the student will be able to:		
1	Interpretation of Boolean Expressions of digital design in simplified form	L2
2	Build the various elements of digital logic system with Verilog	L3
3	Construct Combinational and Sequential logic circuits	L3
4	Analyse the hardware model of a digital system at different levels of abstraction in Verilog	L4
5	Evaluate the functionality of digital design by implementing on FPGA kits	L5
6	Modeling of digital systems using FSM	L3

List of Laboratory/Practical Experiments activities to be conducted:

Experiments can be conducted using Verilog tool /Kits

1. Introduction to Xilinx tool, FPGA flow
2. Adder – HA, FA using data flow and behaviour modelling styles
3. Adder – HA, FA using structural modelling style
4. Combinational designs – I (blocking and non-blocking/looping examples)
 - a. Multiplexer: 4:1, 8:1 MUX.
 - b. De Multiplexer: 1:4, 1:8 DEMUX.
5. Combinational designs – II (different types of case statements)
 - c. Encoder with and without Priority: 8:3 and 4:2.
 - d. Decoder: 3:8 and 2:4.
6. Design of 4-bit ALU
7. Flip Flop: D FF, T FF, JK FF
8. Design of Mod – n Up/Down Counter with Synchronous reset
9. Design of Mod – n Up/Down Counter with Asynchronous reset.
10. Design of Universal shift Register using FSM
11. Open Ended Experiments

Course Description

This comprehensive course is a thorough introduction to the VHDL language. The emphasis is on writing solid synthesizable code and enough simulation code to write a viable test bench. Structural, Register Transfer Level (RTL), and behavioral coding styles are covered. This class addresses targeting Xilinx devices specifically and FPGA devices in general. The information gained can be applied to any digital design by using a top-down synthesis design approach. This course combines insightful lectures with practical lab exercises to reinforce key concepts. You will also learn best coding practices that will increase your overall VHDL proficiency and prepare you for the *Advanced VLSI* course.

Digital Design Using Verilog:

This course prepares students to implement Verilog modeling of digital logic. Students learn Verilog constructs and hardware modeling techniques. The course covers Verilog language elements and data types. Students tackle key challenges and learn structural, dataflow and behavioral modeling in Verilog, including common constructs and coding considerations. Instruction in the coding and testing of digital logic includes examples of combinational circuits (gates, mux/demux, encoders/decoders, and Boolean expression), sequential circuits (latches, flip-flops, shift registers, counters, RAMs and ROMs), and complex logic (flavors of ALU and FSM)

Designing Xilinx FPGAs:

This course is a practical introduction to programmable logic designs with Xilinx FPGAs. Using several examples and design techniques, students will be taken through a complete PLD design. Upon completion of the course, students should be able to complete a design with Xilinx FPGAs, and understand the design and timing reports. Topics include a logic design process review, design software, Xilinx CPLDs and FPGAs architecture, design techniques and optimizing.

Lab Descriptions

The labs for this course provide a practical foundation for creating synthesizable RTL code. All aspects of the design flow are covered in the labs. You will write, synthesize, simulate, and implement all the labs. The focus of the labs is to write code that will optimally infer reliable and high-performance circuits.

Introduction to Field Programmable Gate Arrays (FPGAs)

A FPGA is a Field Programmable Gate Array; basically an array of generic gates to perform any logic function. Many FPGAs simply use small blocks of memory, called CLBs (Combinational Logic Blocks), to look up the answer to equations of 4 or 5 variables. In the past, AND and OR gates would be interconnected to solve equations; but this has been replaced with CLB's as they are more flexible and can

be used as memory blocks. As not all equations have as little as 4 variables; typical designs will be spread over several CLBs; requiring signals to be routed between the CLBs. Just how much circuitry there is, and how fast it will run, in a particular FPGA, depends upon the speed of CLBs, the amount of resources for routing signals between CLBs, and how well a design can be "laid out" or optimized. Newer FPGAs are tailored for specific circuits. The Cyclone II FPGA, which is in our labs, has hardware multipliers and adders, which run at 250MHz, allowing ultra-signal processing circuits to be built. Configurable interconnects are provided between the chip resources (CLBs, hardware multipliers and memory blocks for the Cyclone II FPGA). The logic, circuitry, and interconnects in the architecture are configured by uploading a programming file to the FPGA chip. This property makes the FPGA chip very flexible since it is able to realize different digital circuits by uploading a different programming file. FPGAs are different than microprocessors or microcontrollers because the designer is able to change the hardware realized by the chip by programming it. Since hardware is always faster than software FPGAs allow hardware to be built with nearly the speed of software development.

FPGA design steps

1. *Writing an HDL description ("design entry").* HDL is a class of high-level languages which is used to define how the device should work. It can be thought about as a programming language, though significantly different from the conventional programming languages. The most frequently used hardware description languages are *VHDL* and *Verilog*.
2. *Writing a test environment.* It is almost impossible to create a fully correct HDL design at once. Therefore, it should be tested for possible errors. Whereas in the area of software development a program can be tested by simply running it, testing FPGA design involves writing a dedicated *test environment*. Test environment can be written in HDL (VHDL/Verilog), or in SystemC (SystemC is a special class library for C++ with the support for hardware signal simulation). A test environment usually includes a *behavioral model*, which is a higher-level, non-synthesizable device description used to verify HDL design correctness.
3. Behavioral simulation is used to verify the HDL description against the corresponding behavioral model (using test environment). Most design errors are fixed at this stage.
4. Synthesis is an automated process of converting a high-level HDL description to a machine-readable circuit description (a so-called netlist). Although synthesis of a correctly written HDL code shouldn't be a problem, some errors uncaught by behavioral simulation can appear at this stage.
5. Implementation is a process of converting netlist to an FPGA configuration bit stream (tailored for specific FPGA device).

6. Post-implementation simulation is used to verify the implemented design (taking switching and propagation delays into account) against the behavioral model. This step can be omitted for simple designs.
7. Testing a produced bit stream in hardware.

FPGA design is impossible without specialized vendor-specific EDA (electronic design automation) tools:

- Xilinx ISE (Integrated Software Environment) for Xilinx FPGAs,
- Quartus II for Altera devices,
- Libero for Actel devices.

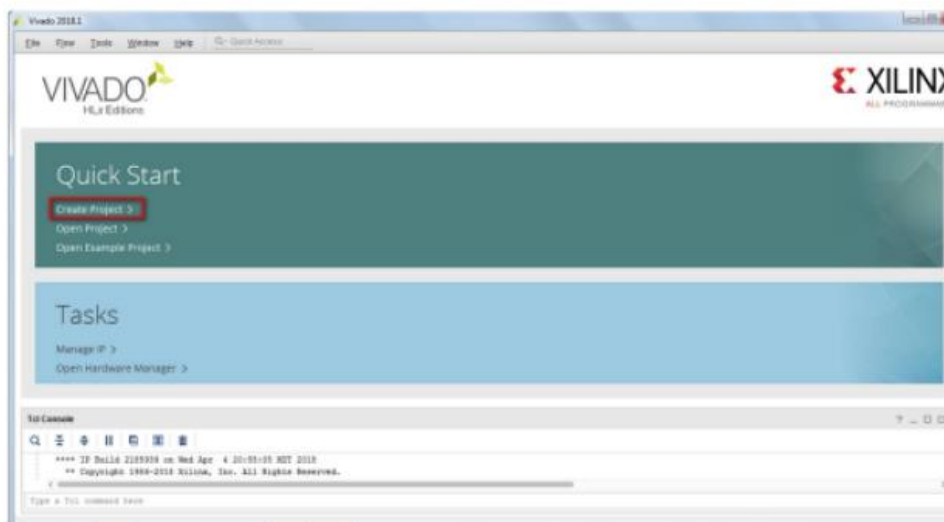
Tool Overview

Getting Started with Vivado Design Suite for Spartan 7 FPGA kit

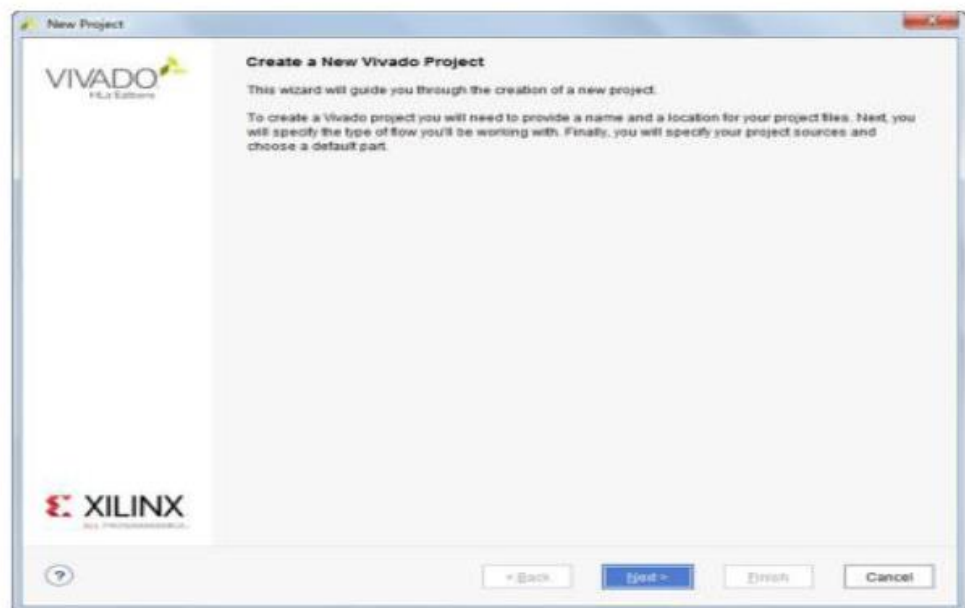
This tutorial explains the step-by-step procedure to create a Vivado project, create source files, synthesize the design, Implement the design and finally verify the functionality in FPGA using the Spartan 7 board.

Step 1: Open Vivado design Suite by selecting

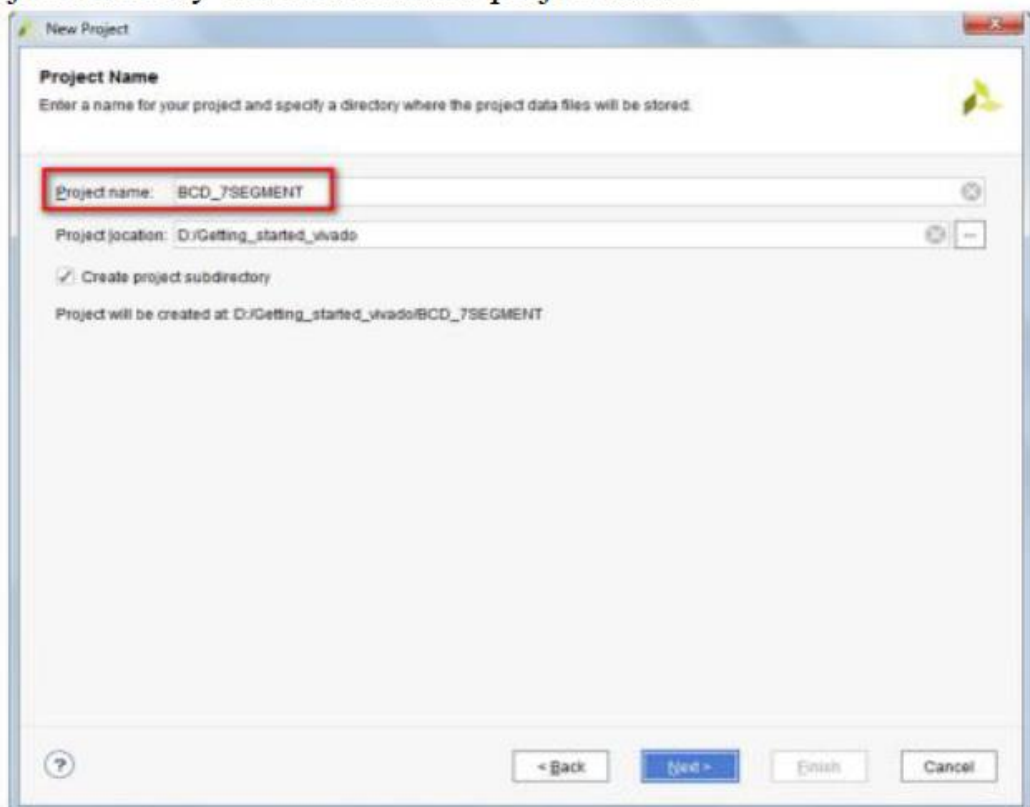
Start > All Programs > Xilinx Design Tools > Vivado 2019.1 > Vivado 2019.1



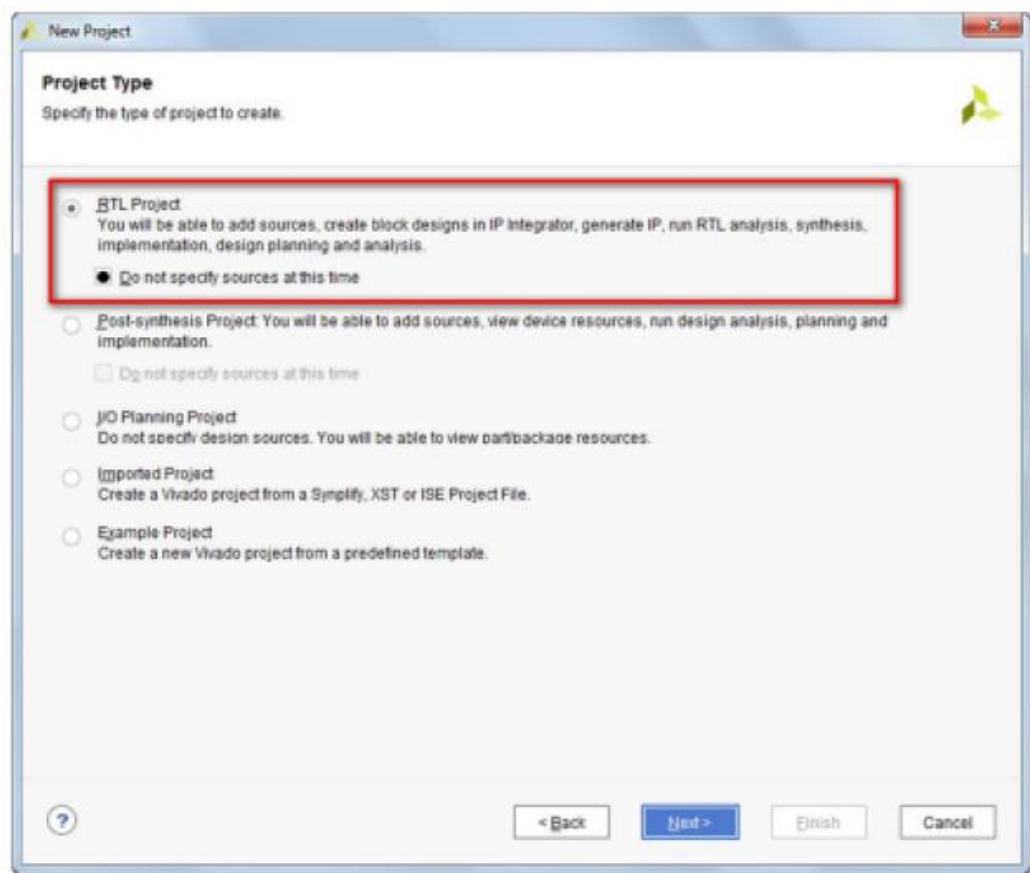
Step 2: Click **Create New Project** and click **next** to open new project wizard.



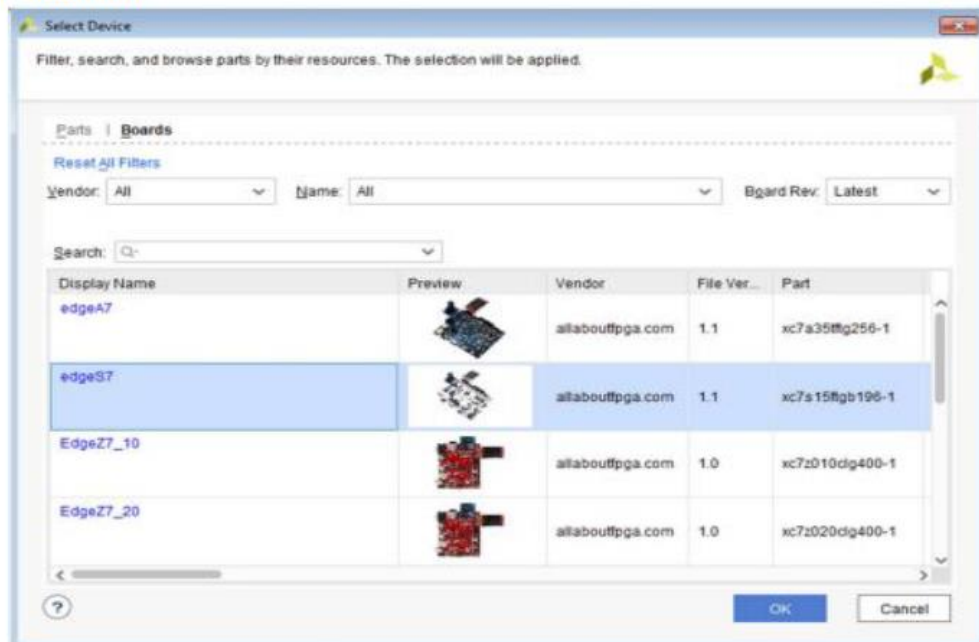
Step 3: Select Project directory and mention new project name.



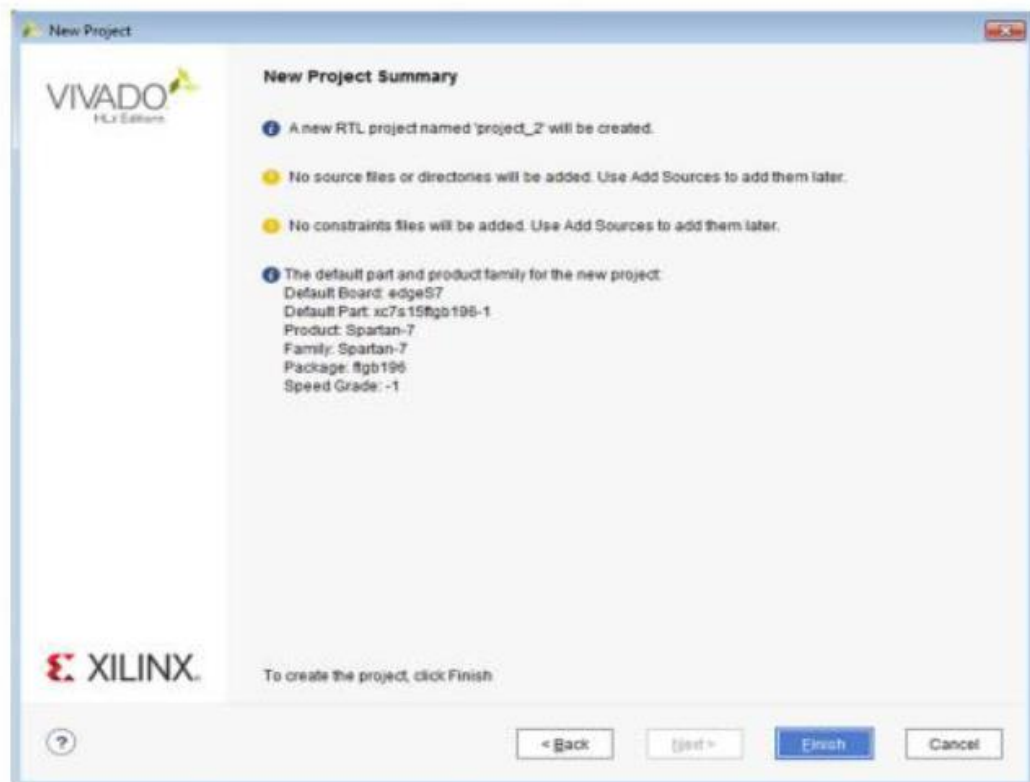
Step 4: Select RTL Project and check Do not specify sources at this time.



Step 5: Click Boards and select S7 in Part Selection window or select Part XC7S15 FTGB196-1 from the list of parts available.

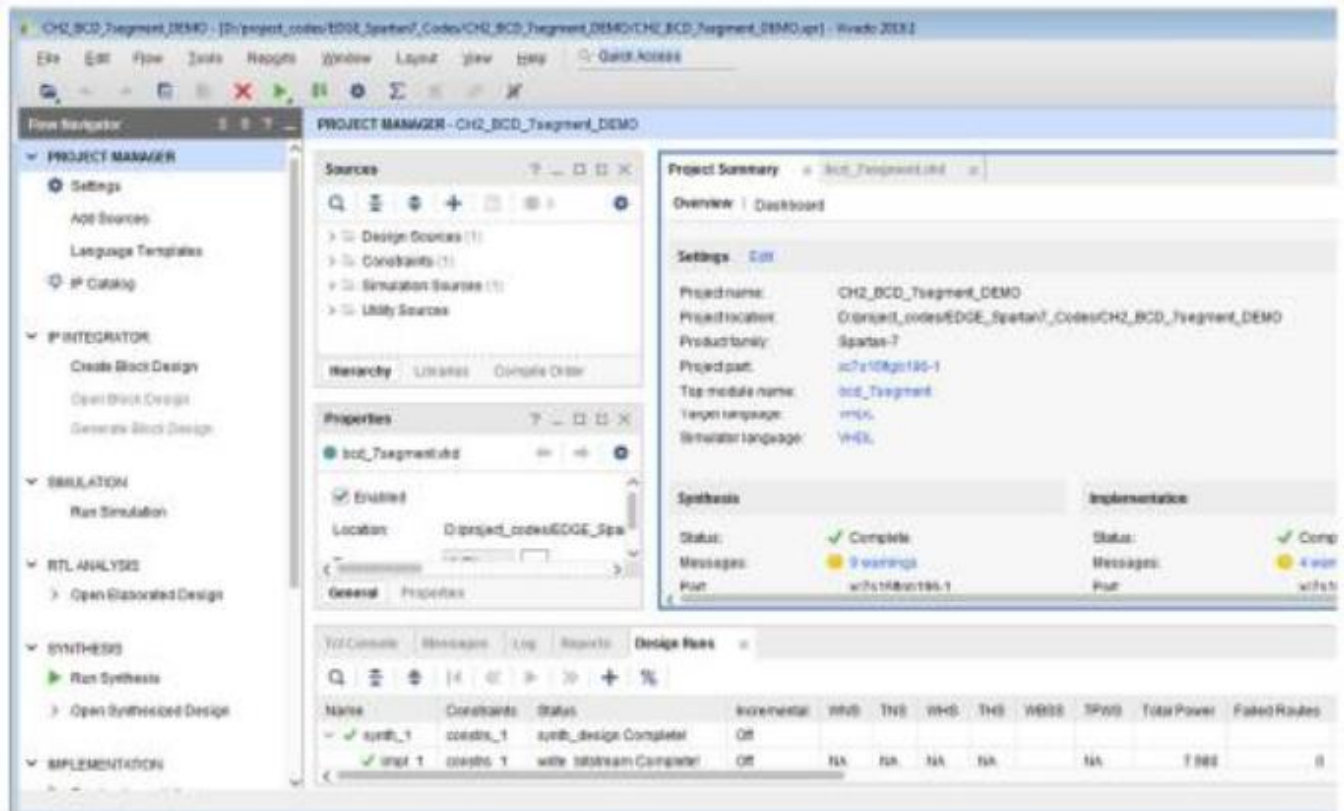


Step 6: Now the New Project summary display with project details selected.

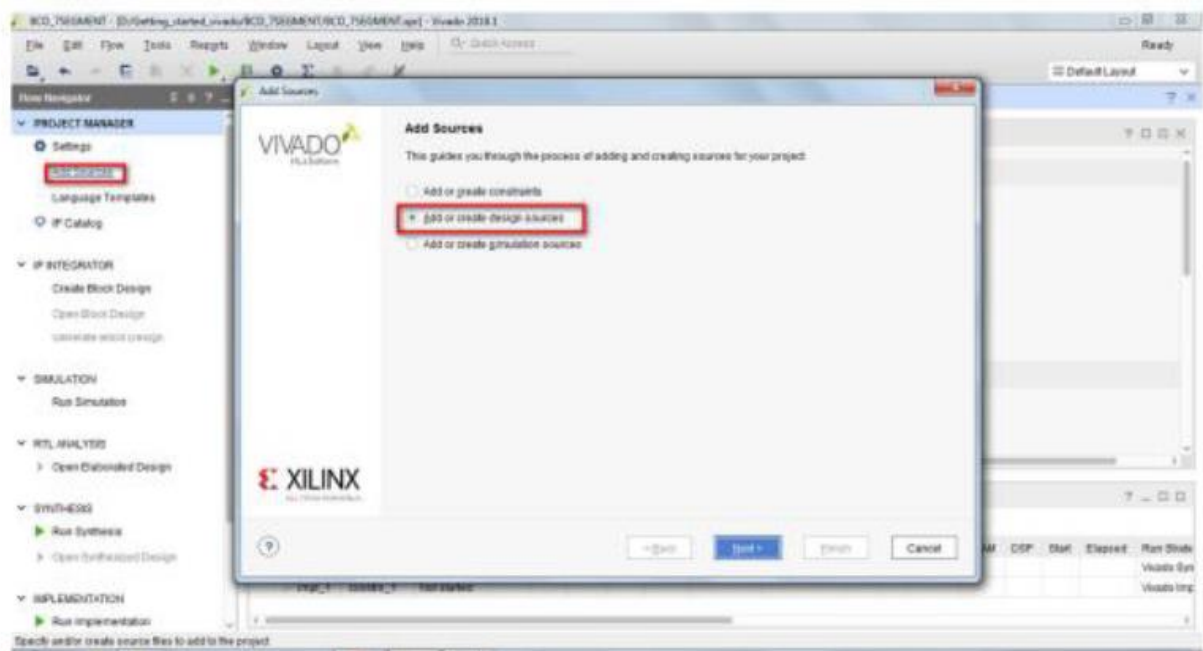


Step 7: Now the Vivado tool opens with Flow Navigator on left and Project Manager on right. Flow Navigator display the list of process involved from HDL input creation to bit file output generation. Project manager consist of Source, Workspace and Report Window.

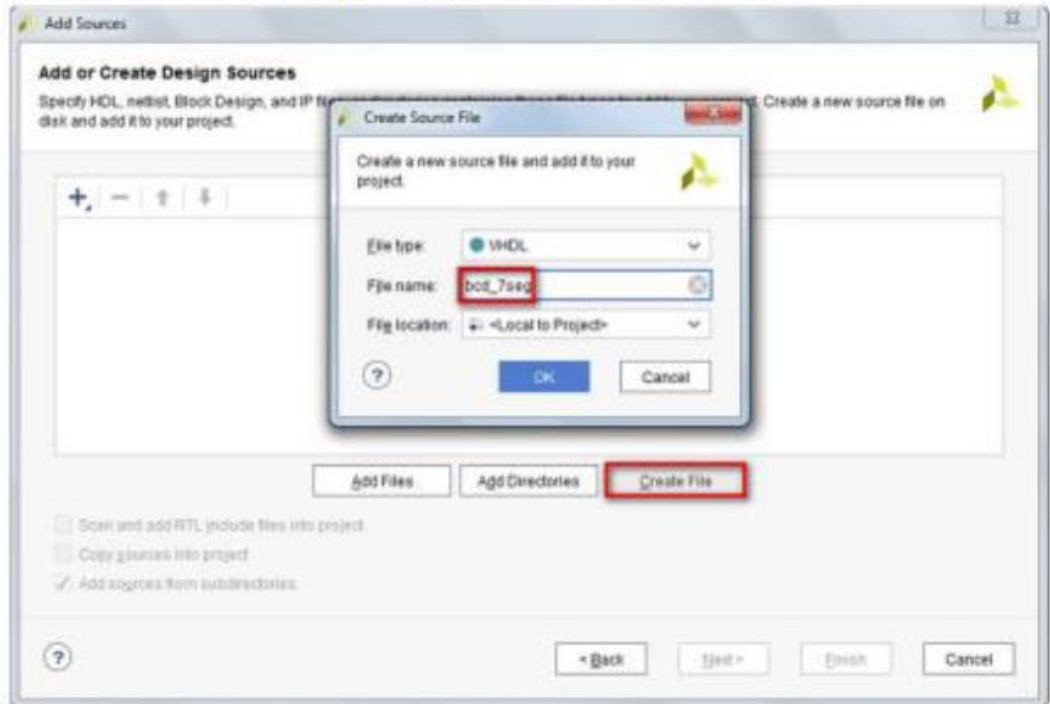
Source Window displays Design Sources (VHDL, Verilog), Constraints (XDC) and Simulation sources. Workspace Window is used to create and view HDL/XDC files. Report Window consist of TCL console, Messages, Logs, Reports and Design Rules



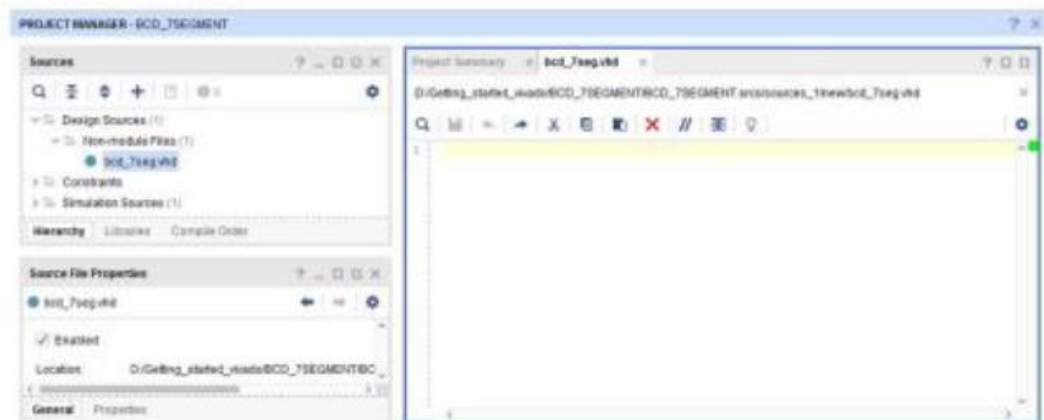
Step 8: Now click Add source and select Add create design sources



Step 9: Click create file and select file type and file name as bcd_7seg



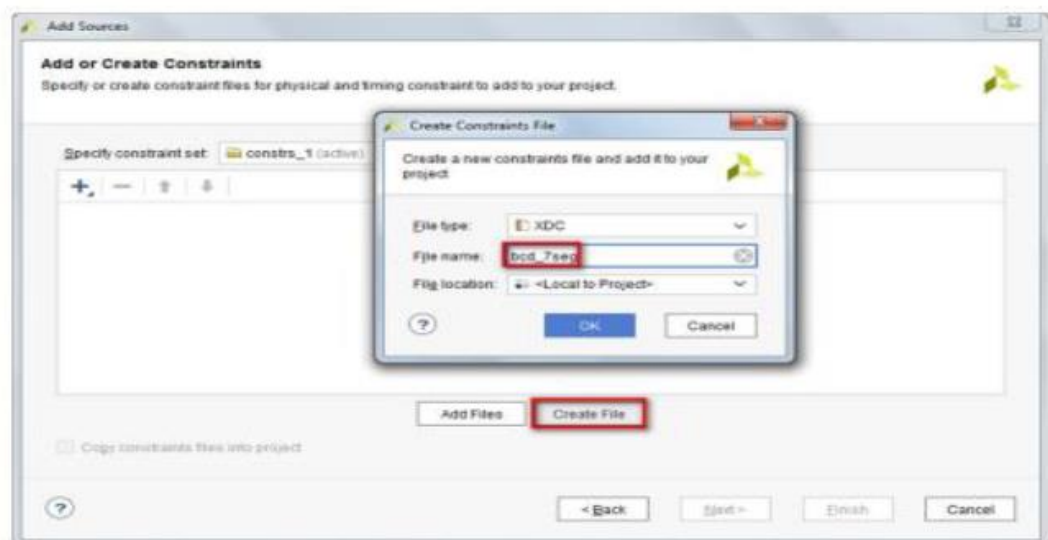
Step 10: Double click on the bcd_7seg.vhd at source window to open on the right side.

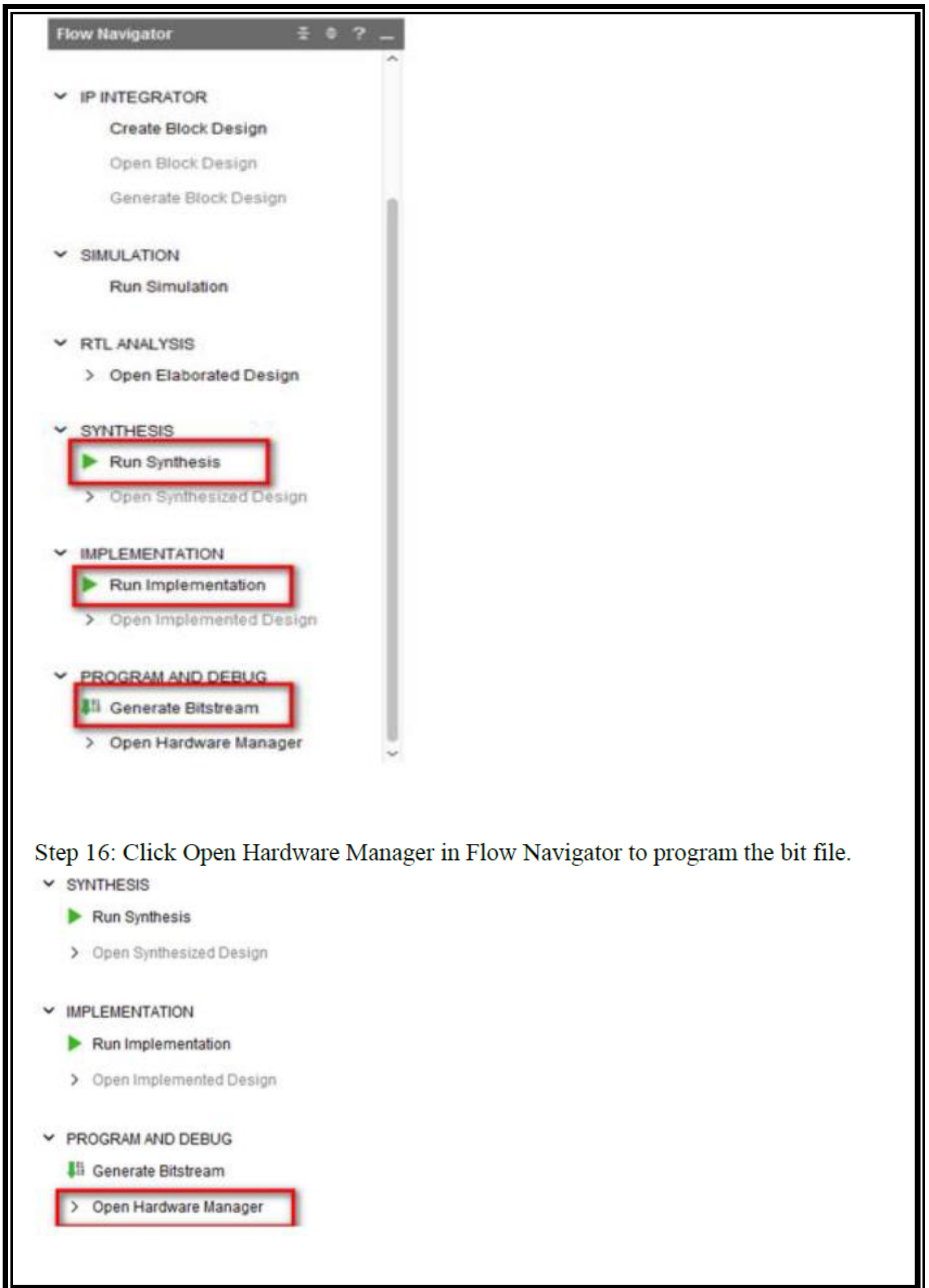


Step 12: Now again select the Add Source and select Add or create constraints

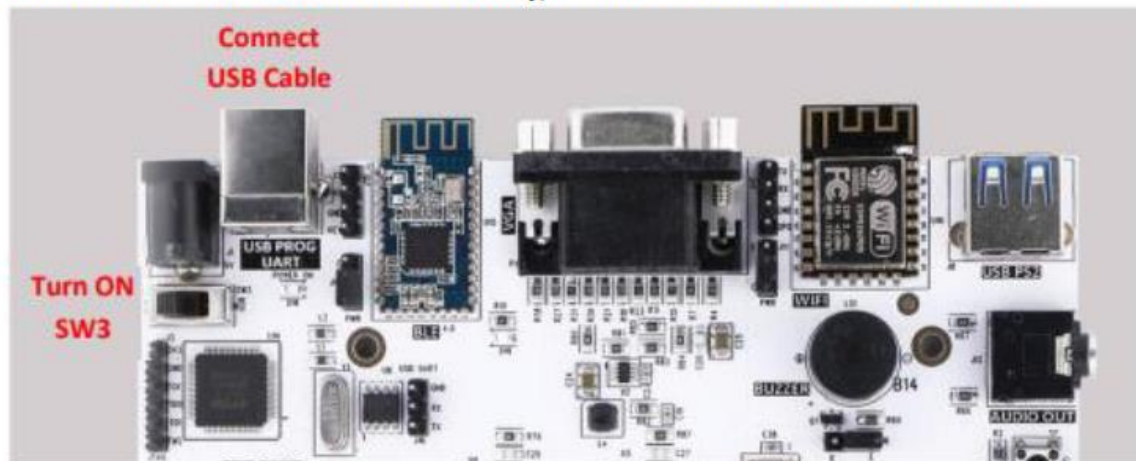


Step 13: Click create file and mention file type and file name

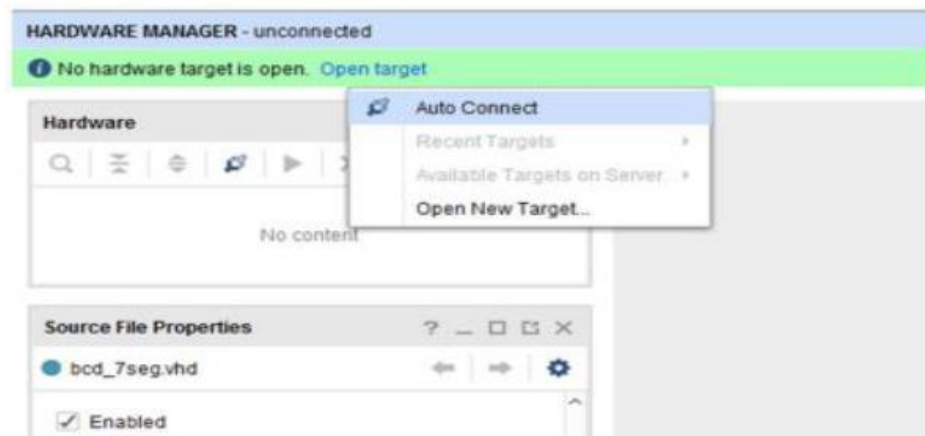




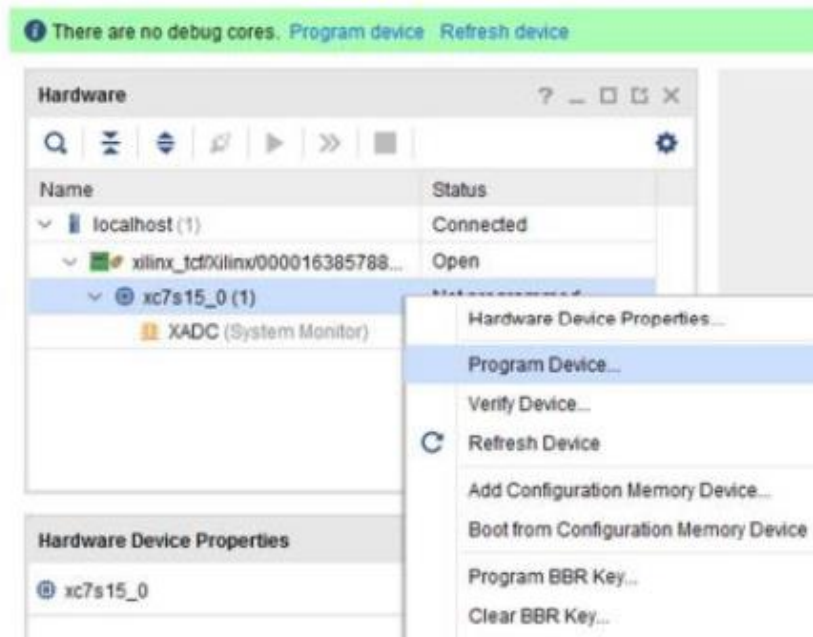
Step 17: Connect Artix 7 FPGA Kit to PC through USB cable and Turn On the kit.



Step 18: To detect FPGA, Do Hardware Manager -> Open Target -> Auto Connect



Step 19: If the device is detected successfully, then select Program Device by right click on the target device xc7s15_0 as shown below Turn ON SW3 Connect USB Cable



Step 20: Browse the Bit file need to be downloaded to the Spartan 7 FPGA and click Program.



Step 21: Once the Program Succeeds, Done LED light up on Spartan 7 FPGA kit. Observe the output by varying the different combination of inputs.

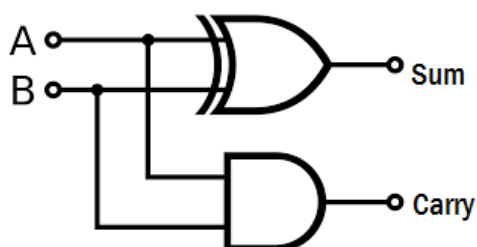
Experiment -1

Adder – Half Adder using data flow, Structural and behaviour modelling styles

AIM: To design half adder using dataflow modeling, Gate level Modelling and Behavioral Modelling style and verify the functionalities along with their synthesis and simulation results and also implement of FPGA kits.

Design and Circuit diagram:

HALF ADDER



Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

VERILOG CODE USING DATA FLOW MODELLING

```
module half_adder (  
    input a,  
    input b,  
    output sum,  
    output carry);  
assign sum = a ^ b;  
assign carry = a & b;  
endmodule
```

VERILOG CODE USING GATE LEVEL MODELLING

```
module HA_gate(a, b, sum, carry);  
    input a, b;  
    output sum, carry;  
    xor (sum, a, b);  
    and (carry, a, b);  
endmodule
```

VERILOG CODE USING BEHAVIORAL MODELLING

```
module half_adder_behave (input a,  
    input b,  
    output sum,  
    output carry);
```

```

reg sum, carry;
always @ (a or b)
begin
sum = a ^ b;
carry = a & b;
end

```

```

endmodule

```

HALF ADDER TESTBENCH FOR ALL 3 MODELLING STYLES

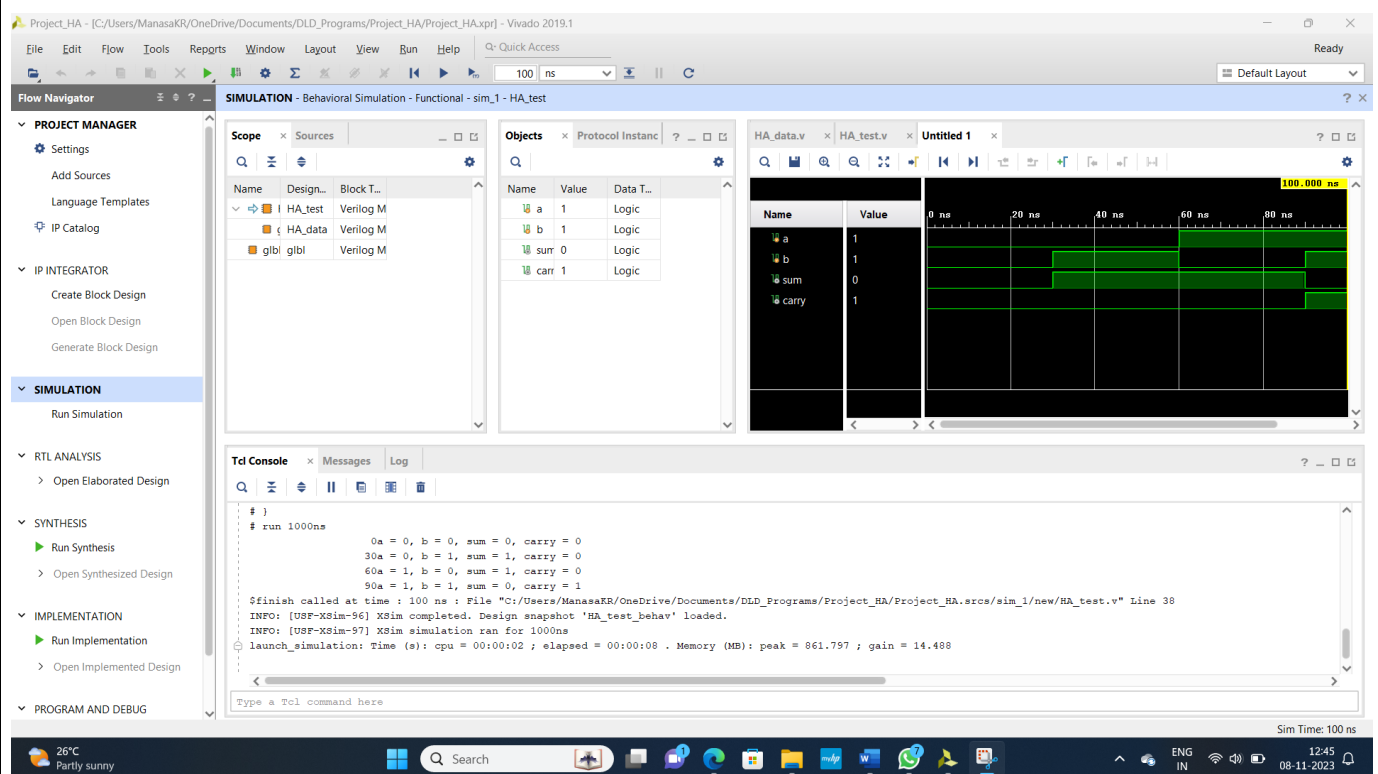
```

module HA_Test( );
reg a, b;
wire sum, carry;
half_adder g1(a,b, sum,carry);
initial
begin
#0 a=1'b0; b=1'b0;
#30 a=1'b0; b=1'b1;
#30 a=1'b1; b=1'b0;
#30 a=1'b1; b=1'b1;
end
initial
$monitor ( $time, "a = %b, b = %b, sum = %b, carry = %b", a, b, sum, carry);

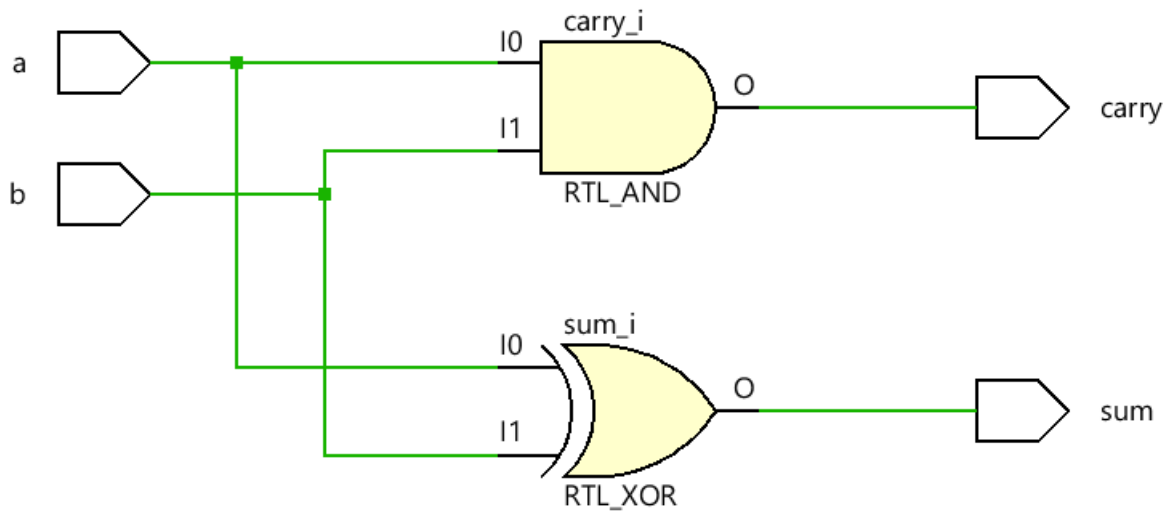
#100 $finish;
endmodule

```

SIMULATION RESULTS:



SYNTHESIS RESULTS:



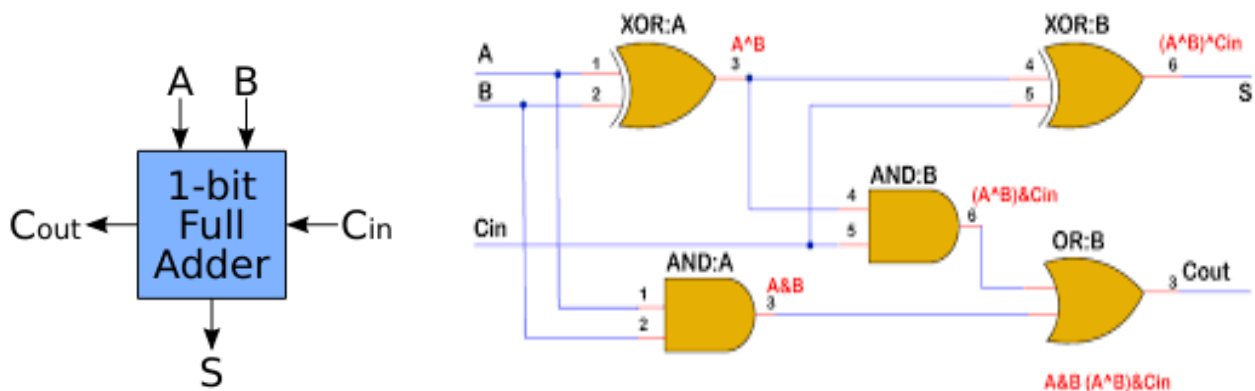
Conclusion: The half adder has been implemented in different modelling styles and verified on FPGA kit

Experiment -1

Adder – Full Adder using data flow, Structural and behaviour modelling styles

AIM: To design full adder using dataflow modeling, Gate level Modelling and Behavioral Modelling style and verify the functionalities along with their synthesis and simulation results and also implement of FPGA kits.

Design and Circuit diagram:



Inputs			Outputs	
A	B	C _{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A	BC			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$\text{SUM} = A \oplus B \oplus C$$

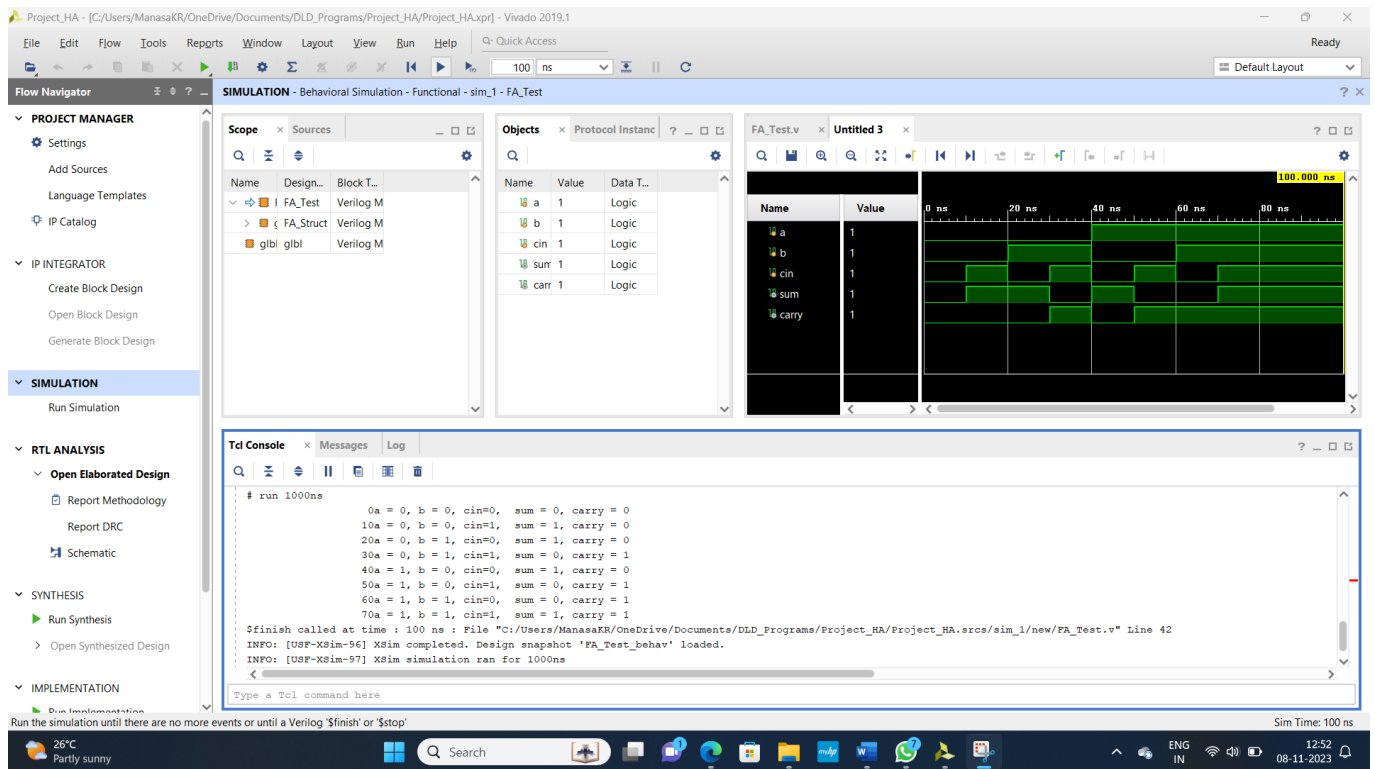
A	BC			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$\text{CARRY} = AB + BC + AC$$

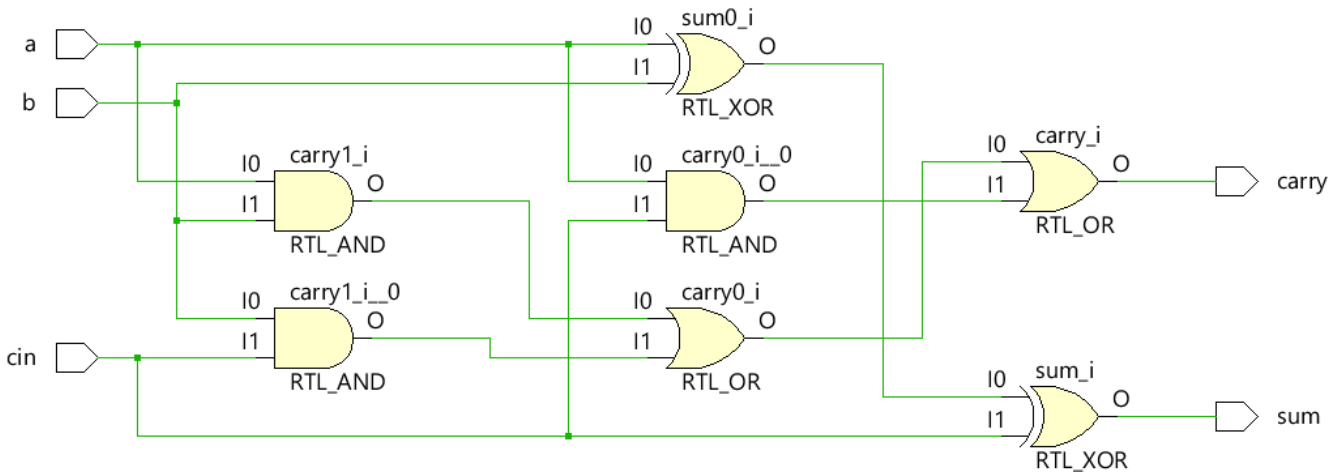
VERILOG CODE USING DATA FLOW MODELLING

```
module FA_Data(a, b, cin, sum, carry);  
input a, b, cin;  
output sum, carry;  
assign sum = a^b^cin;  
assign carry = (a&b)|(b&cin)|(a&cin);  
endmodule
```

SIMULATION RESULTS:

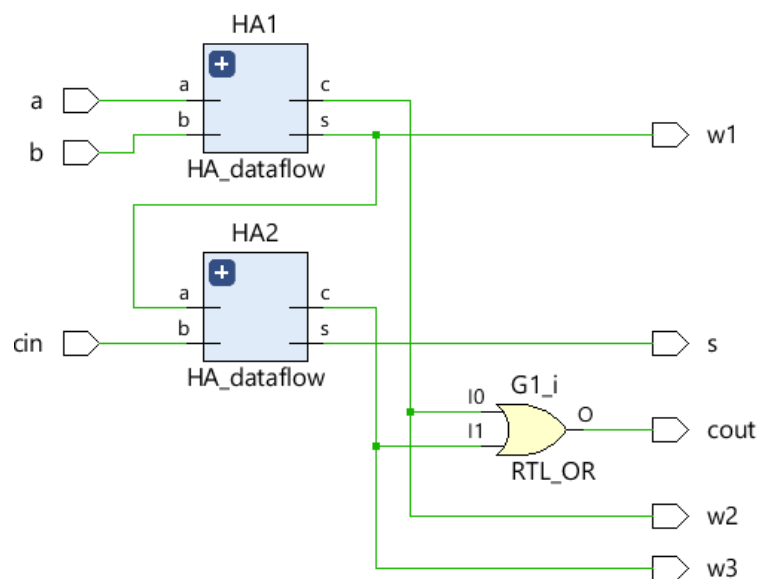


SYNTHESIS RESULTS:



VERILOG CODE USING GATE LEVEL MODELLING FOR FULL ADDER USING 2 HALF ADDERS

```
module FA_gate(a, b, cin, sum, carry);
input a,b,cin;
output sum, carry;
wire s,t,u;
xor (s, a, b);
and (t,a,b);
xor (sum, s, cin);
and (u,s,cin);
or (carry,u,t);
endmodule
```



VERILOG CODE USING BEHAVIORAL

```
module FA_behave(a, b, cin, sum, carry);
input a, b, cin;
output sum, carry;
wire sum, carry;
always @ (a or b or cin)
begin
sum = a^b^cin;
carry = (a&b)|(b&cin)|(a&cin);
end
endmodule
```

VERILOG CODE FOR FULL ADDER TESTBENCH FOR ALL 3 MODELLING STYLES

```
module FA_Test( );
reg a, b, cin;
wire sum, carry;
FA_behave g1(a,b,cin, sum,carry);
initial
begin
#0 a=1'b0; b=1'b0; cin=1'b0;
#10 a=1'b0; b=1'b0; cin=1'b1;
#10 a=1'b0; b=1'b1; cin=1'b0;
#10 a=1'b0; b=1'b1; cin=1'b1;
#10 a=1'b1; b=1'b0; cin=1'b0;
#10 a=1'b1; b=1'b0; cin=1'b1;
#10 a=1'b1; b=1'b1; cin=1'b0;
#10 a=1'b1; b=1'b1; cin=1'b1;
end
initial
$monitor ( $time, "a = %b, b = %b, cin=%b, sum = %b, carry = %b", a, b, cin, sum, carry);
initial
#100 $finish;
endmodule
```

Conclusion: The Verilog code for different modelling styles' has been simulated and implemented on FPGA Kits

Experiment -3

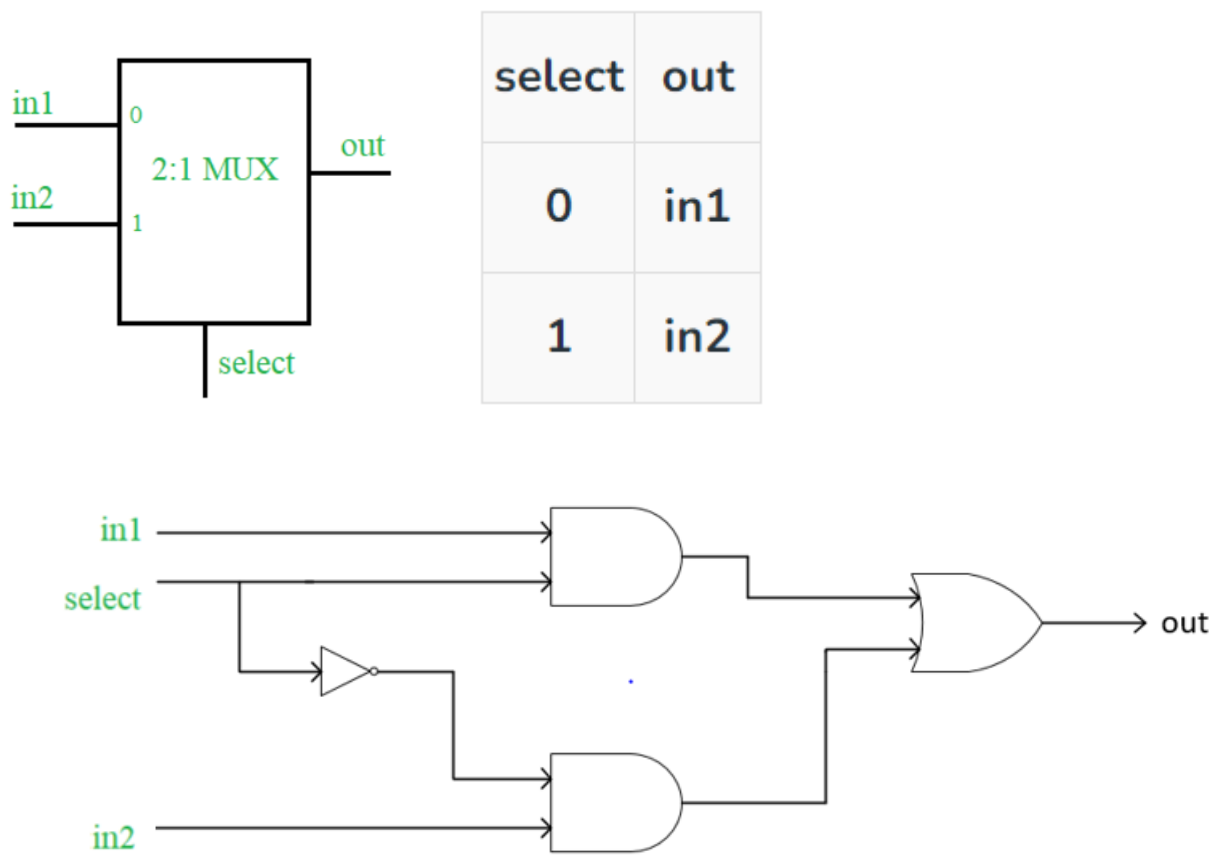
Combinational designs – I

a. Multiplexer: 4:1, 8:1 MUX.

b. De Multiplexer: 1:4, 1:8 DEMUX.

AIM: To design Mux and Demux using if else and case statements and verify the functionalities along with their synthesis and simulation results and also implement of FPGA kits.

2:1 Mux (NOT IN SYLLABUS but can spend 20 mins on this if time is available for better understanding of Conditional operator)



$$\text{Out} = (\text{in1} * \text{select}) + (\text{in2} * \sim \text{select})$$

VERILOG CODE USING CONDITIONAL OPERATOR

```
module mux2to1(in1, in2, sel, out);
input in1, in2, sel;
output reg out;
always @ (in1, in2, sel)
out=sel? in2: in1;
endmodule
```

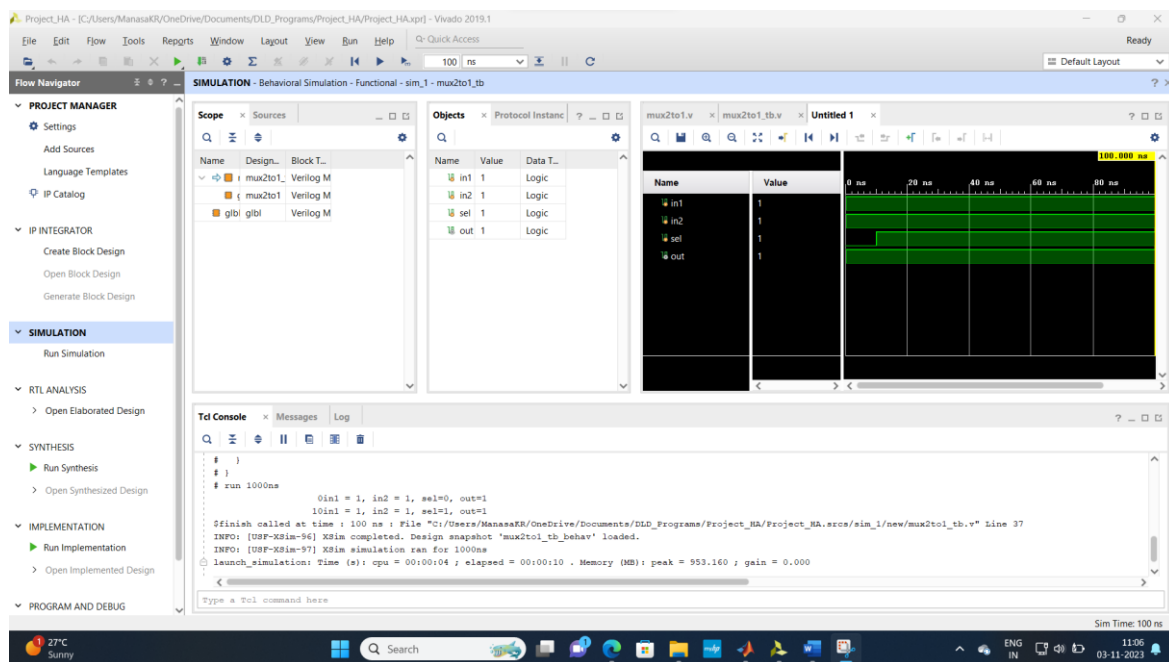
TESTBENCH FOR 2:1 MUX

```
module mux2to1_tb( );
reg in1,in2,sel;
wire out;
mux2to1 g1(in1, in2, sel, out);
initial
begin
#0 sel=1'b0; in1=1'b1; in2=1'b1;
#10 sel=1'b1; in1=1'b1; in2=1'b1;
end
```

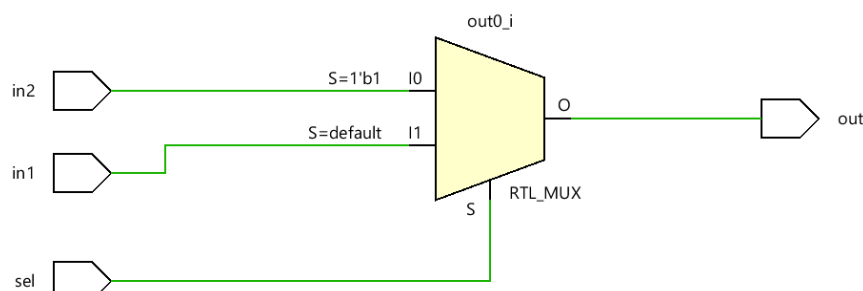
```
initial
$monitor ( $time, "in1 = %b, in2 = %b, sel=%b, out=%b", in1, in2, sel, out);
```

```
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:



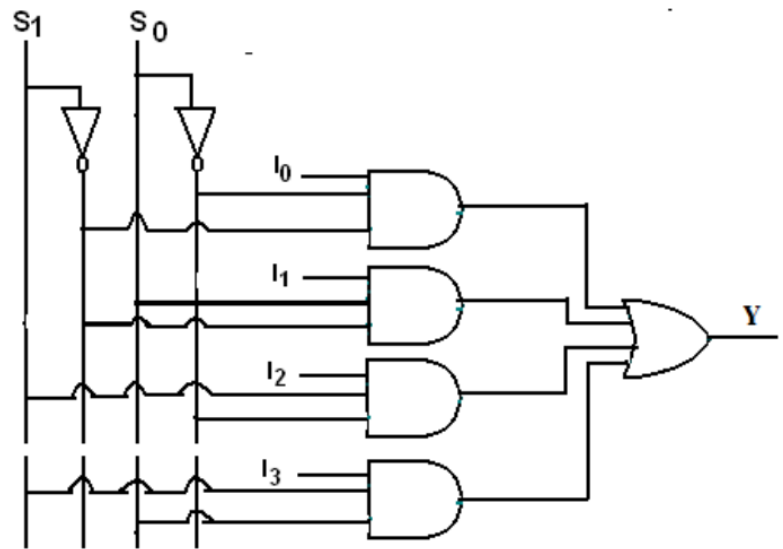
SYNTHESIS RESULTS:



4:1 MUX

Input	S1	S0	Y
I ₀	0	0	I ₀
I ₁	0	1	I ₁
I ₂	1	0	I ₂
I ₃	1	1	I ₃

$$Y = S_1 S_0 I_3 + S_1 \bar{S}_0 I_2 + \bar{S}_1 S_0 I_1 + \bar{S}_1 \bar{S}_0 I_0$$



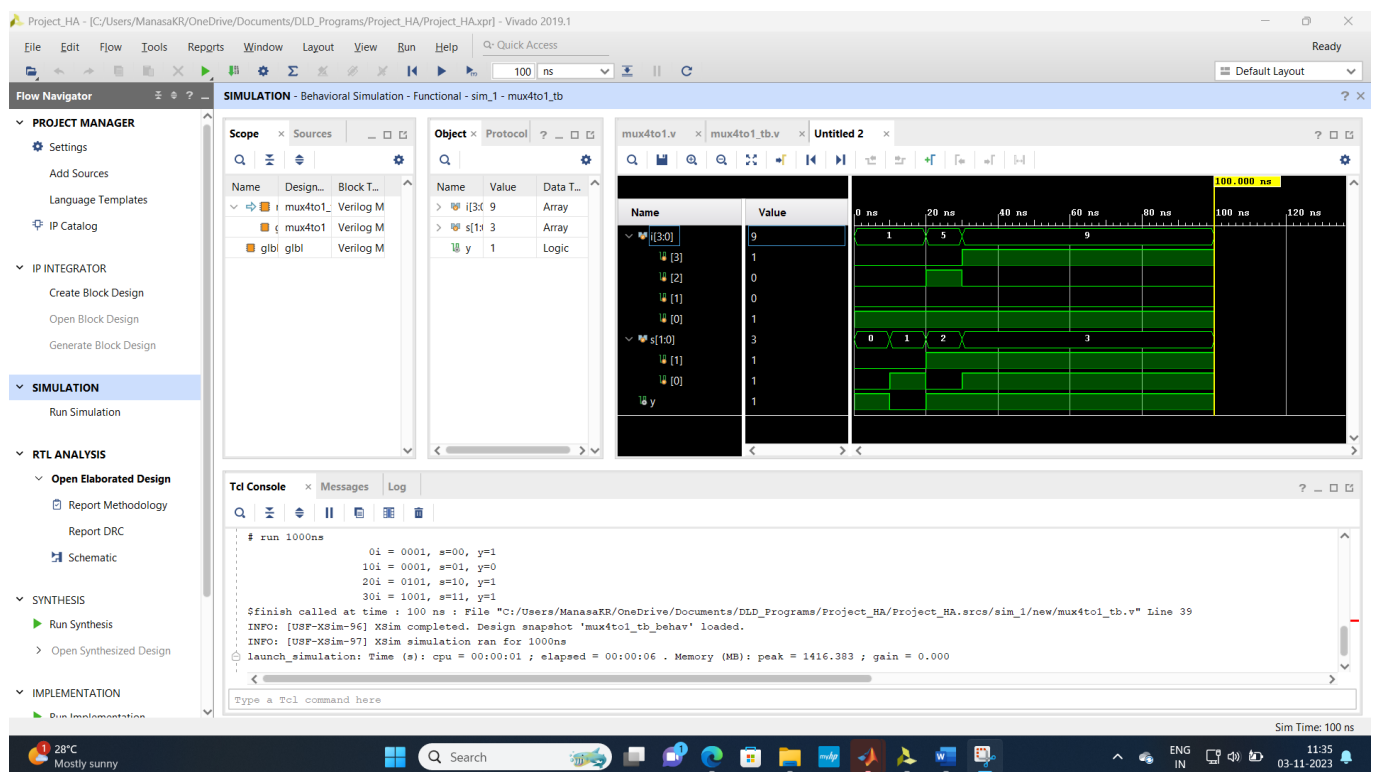
VERILOG CODE USING IF ELSE STATEMENT

```
module mux4to1(i, s, y);
input [3:0] i;
input [1:0] s;
output reg y;
always @ (*)
if (s==0)
y=i[0];
else if (s==1)
y=i[1];
else if (s==2)
y=i[2];
else
y=i[3];
endmodule
```

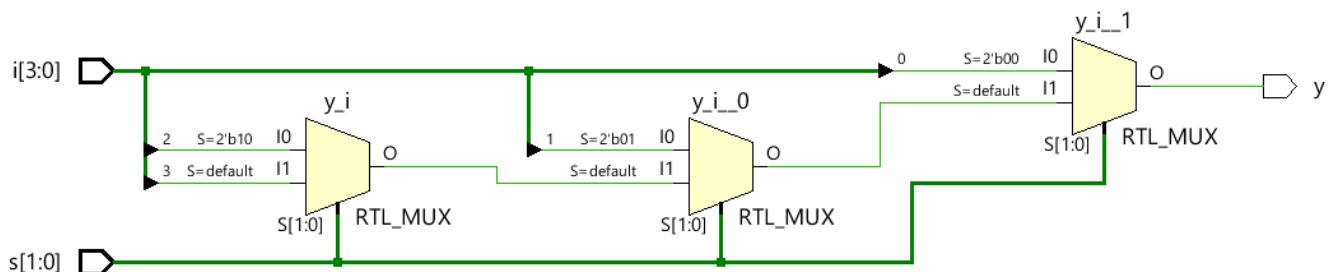
TESTBENCH FOR 4:1 MUX

```
module mux4to1_tb();
reg [3:0] i;
reg [1:0] s;
wire y;
mux4to1 g1(i, s, y);
initial
begin
#0 s=2'b00; i=4'b0001;
#10 s=2'b01; i=4'b0001;
#10 s=2'b10; i=4'b0101;
#10 s=2'b11; i=4'b1001;
end
initial
$monitor ($time, "i = %b, s=%b, y=%b", i, s, y);
initial
#100 $finish; endmodule
```

SIMULATION RESULTS:

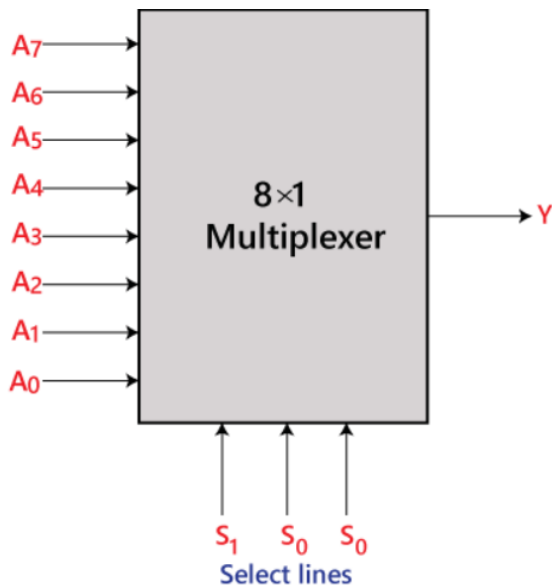


SYNTHESIS RESULTS:



8:1 MUX

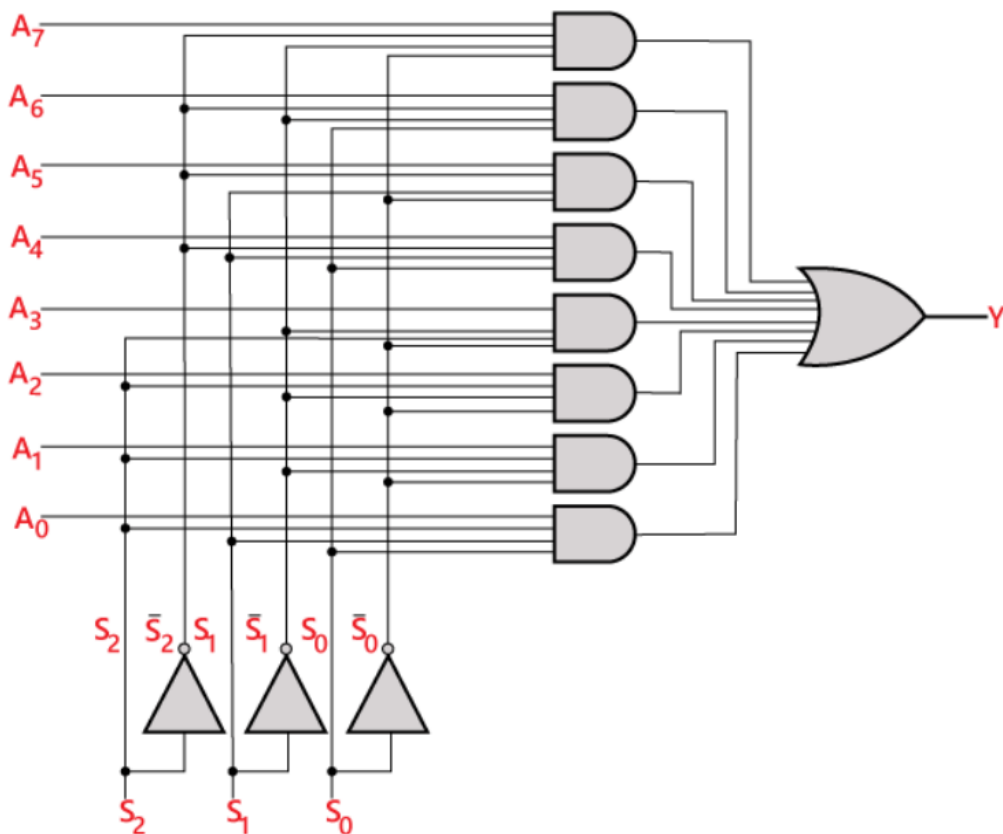
Truth Table:



INPUTS			Output
S_2	S_1	S_0	Y
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7

$$Y = S_0' \cdot S_1' \cdot S_2' \cdot A_0 + S_0' \cdot S_1' \cdot S_2 \cdot A_1 + S_0' \cdot S_1 \cdot S_2' \cdot A_2 + S_0' \cdot S_1 \cdot S_2 \cdot A_3 + S_0 \cdot S_1' \cdot S_2' \cdot A_4 + S_0 \cdot S_1' \cdot S_2 \cdot A_5 + S_0 \cdot S_1 \cdot S_2' \cdot A_6 + S_0 \cdot S_1 \cdot S_2 \cdot A_7$$

Logical circuit of the above expression is given below:



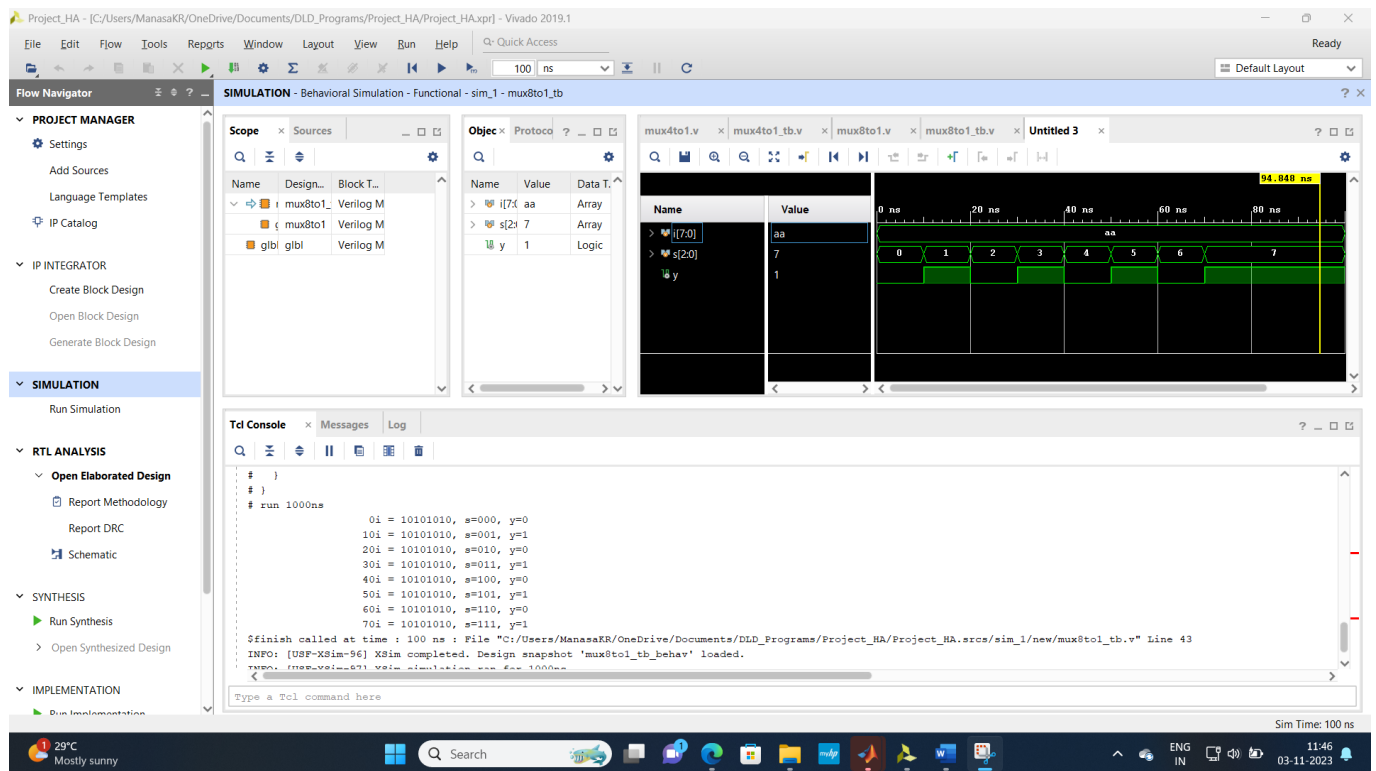
VERILOG CODE USING CASE STATEMENT

```
module mu8to1(i, s, y);
input [7:0] i;
input [2:0] s;
output reg y;
always @ (*)
begin
case (s)
3'b000: y=i[0];
3'b001: y=i[1];
3'b010: y=i[2];
3'b011: y=i[3];
3'b100: y=i[4];
3'b101: y=i[5];
3'b110: y=i[6];
3'b111: y=i[7];
default: y=8'b00000000;
endcase
end
endmodule
```

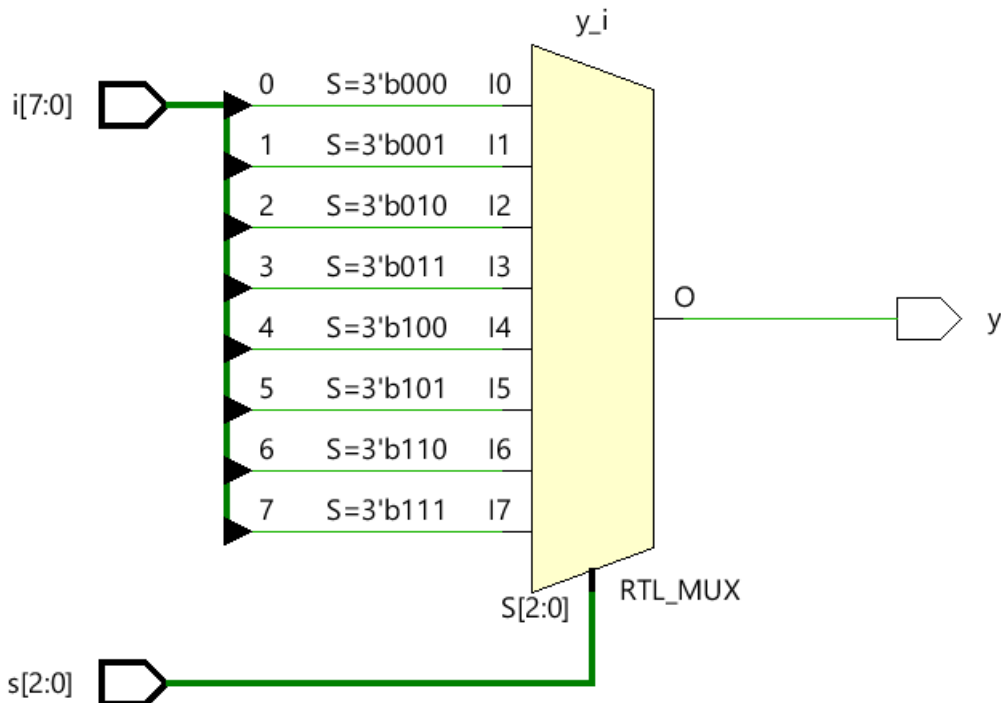
TESTBENCH FOR 8:1 MUX

```
module mux8to1_tb( );
reg [7:0] i;
reg [2:0] s;
wire y;
mux8to1 g1(i, s, y);
initial
begin
#0 s=3'b000; i=8'b10101010;
#10 s=3'b001; i=8'b10101010;
#10 s=3'b010; i=8'b10101010;
#10 s=3'b011; i=8'b10101010;
#10 s=3'b100; i=8'b10101010;
#10 s=3'b101; i=8'b10101010;
#10 s=3'b110; i=8'b10101010;
#10 s=3'b111; i=8'b10101010;
end
initial
$monitor ($time, "i = %b, s=%b, y=%b", i, s, y);
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:

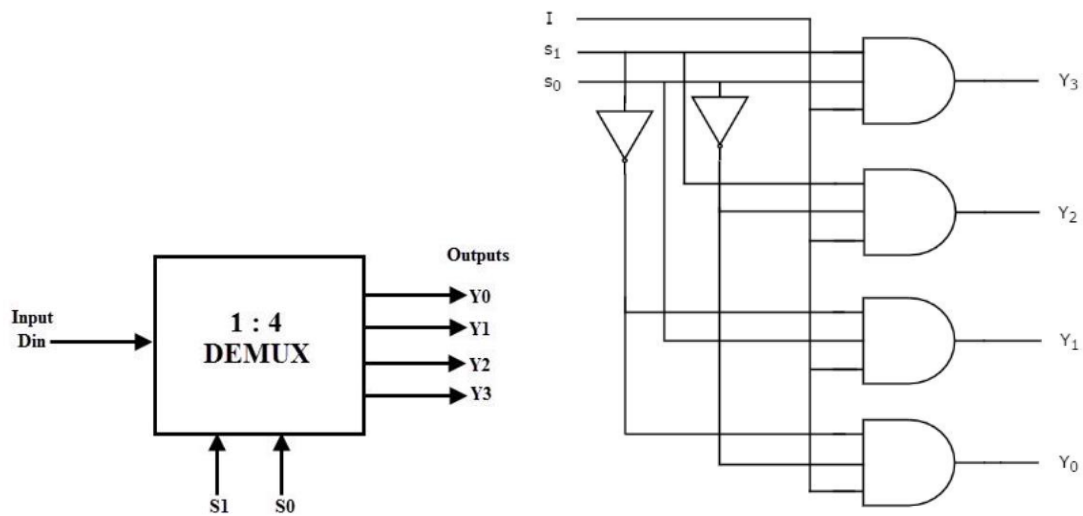


SYNTHESIS RESULTS:



1:4 DEMUX

Circuit Diagram of 1:4 DE MUX



Truth Table for 1:4 Demultiplexers

Data Input	Select Inputs		Outputs				
D	S ₁	S ₀	Y ₃	Y ₂	Y ₁	Y ₀	$Y_0 = \overline{S_1} \overline{S_0} D$
D	0	0	0	0	0	D	$Y_1 = \overline{S_1} S_0 D$
D	0	1	0	0	D	0	$Y_2 = S_1 \overline{S_0} D$
D	1	0	0	D	0	0	$Y_3 = S_1 S_0 D$
D	1	1	D	0	0	0	

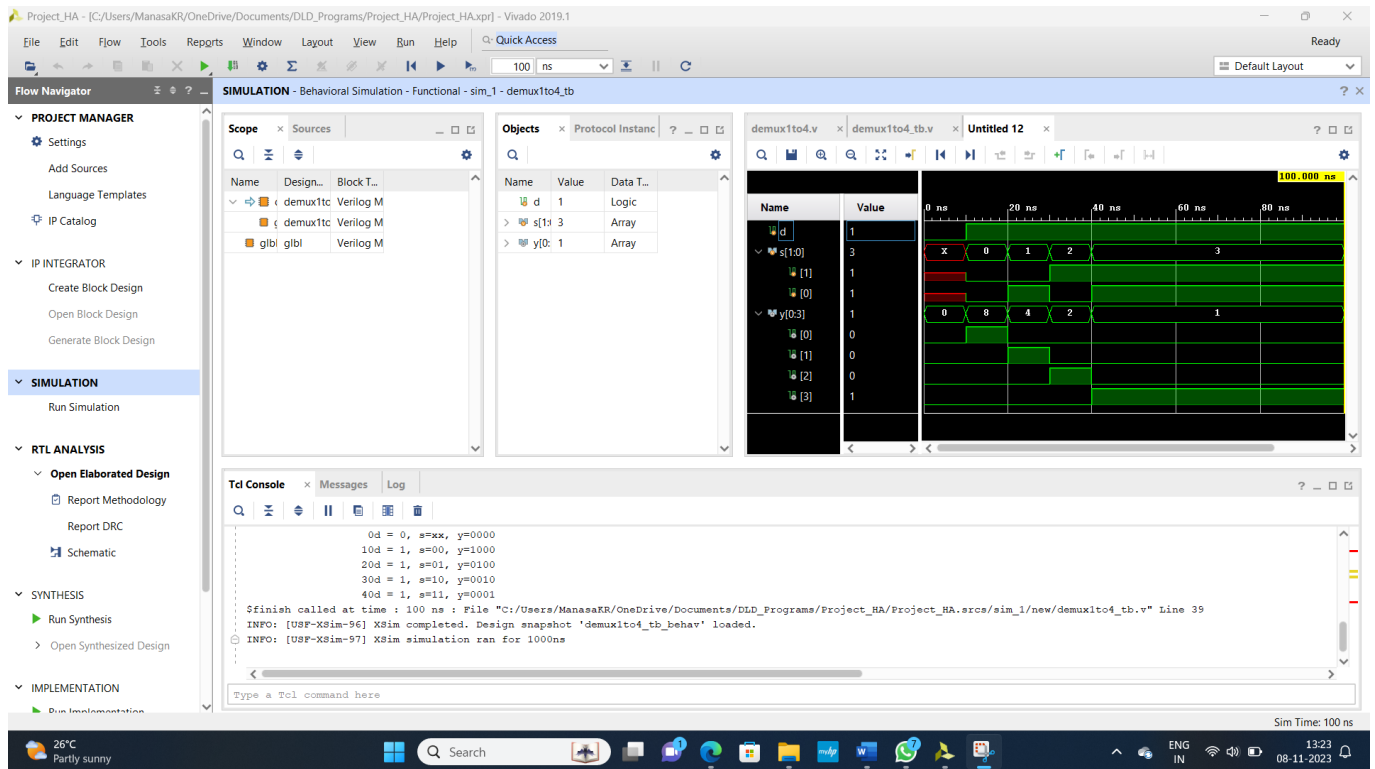
VERILOG CODE 1:4 DEMUX USING CASE STATEMENT

```
module demux1to4 (d, s, y);
input d;
input [1:0] s;
output reg [0:3] y;
always @ (*)
begin
case(s)
0: y={d,3'b000};
1: y={1'b0,d,2'b00};
2: y={2'b00,d,1'b0};
3: y={3'b000,d};
default: y=4'b0000;
endcase
end
endmodule
```

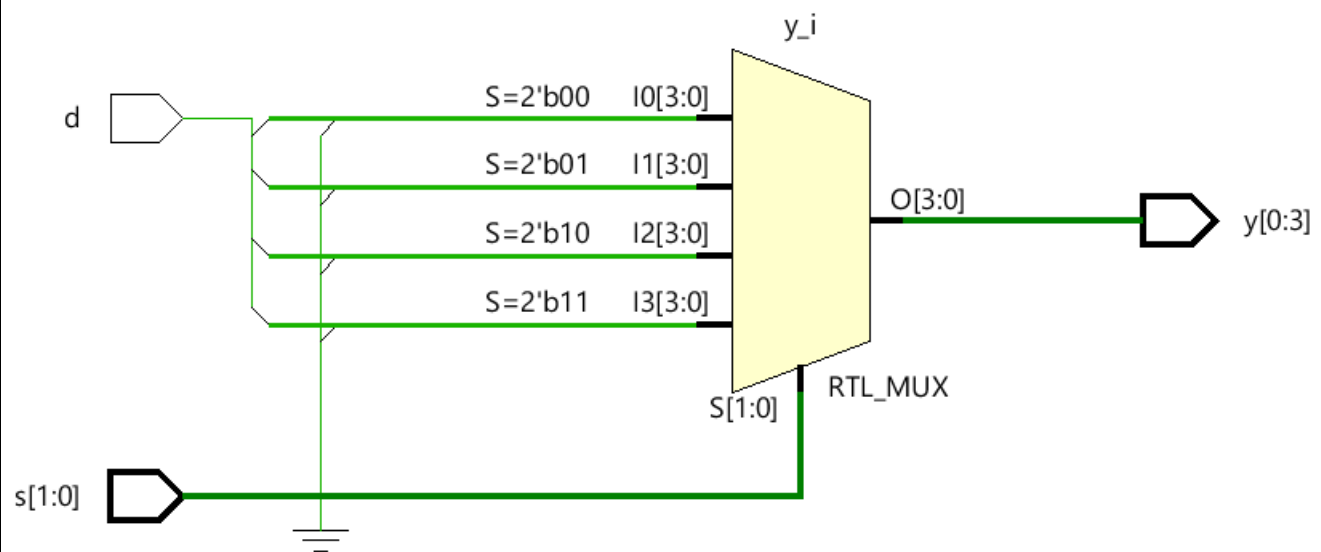

TESTBENCH FOR 1:4 DEMUX

```
module demux1to4_tb( );
reg d;
reg [1:0] s;
wire [0:3] y;
demux1to4 g1(d,s,y);
initial
begin
#0 d=1'b0;
#10 d=1'b1; s=2'b00;
#10 s=2'b01;
#10 s=2'b10;
#10 s=2'b11;
End
initial
$monitor ($time, "d = %b, s=%b, y=%b", d, s, y);
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:



SYNTHESIS RESULTS:



1:8 DEMUX

A	S2	S1	S0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

Boolean Expression for 1:8 Demultiplexer

$$Y0 = A (S2)'(S1)'(S0)'$$

$$Y1 = A (S2)'(S1)'(S0)$$

$$Y2 = A (S2)'(S1)(S0)'$$

$$Y3 = A (S2)'(S1)(S0)$$

$$Y4 = A (S2)(S1)'(S0)'$$

$$Y5 = A (S2)(S1)'(S0)$$

$$Y6 = A (S2)(S1)(S0)'$$

$$Y7 = A (S2)(S1)(S0)$$

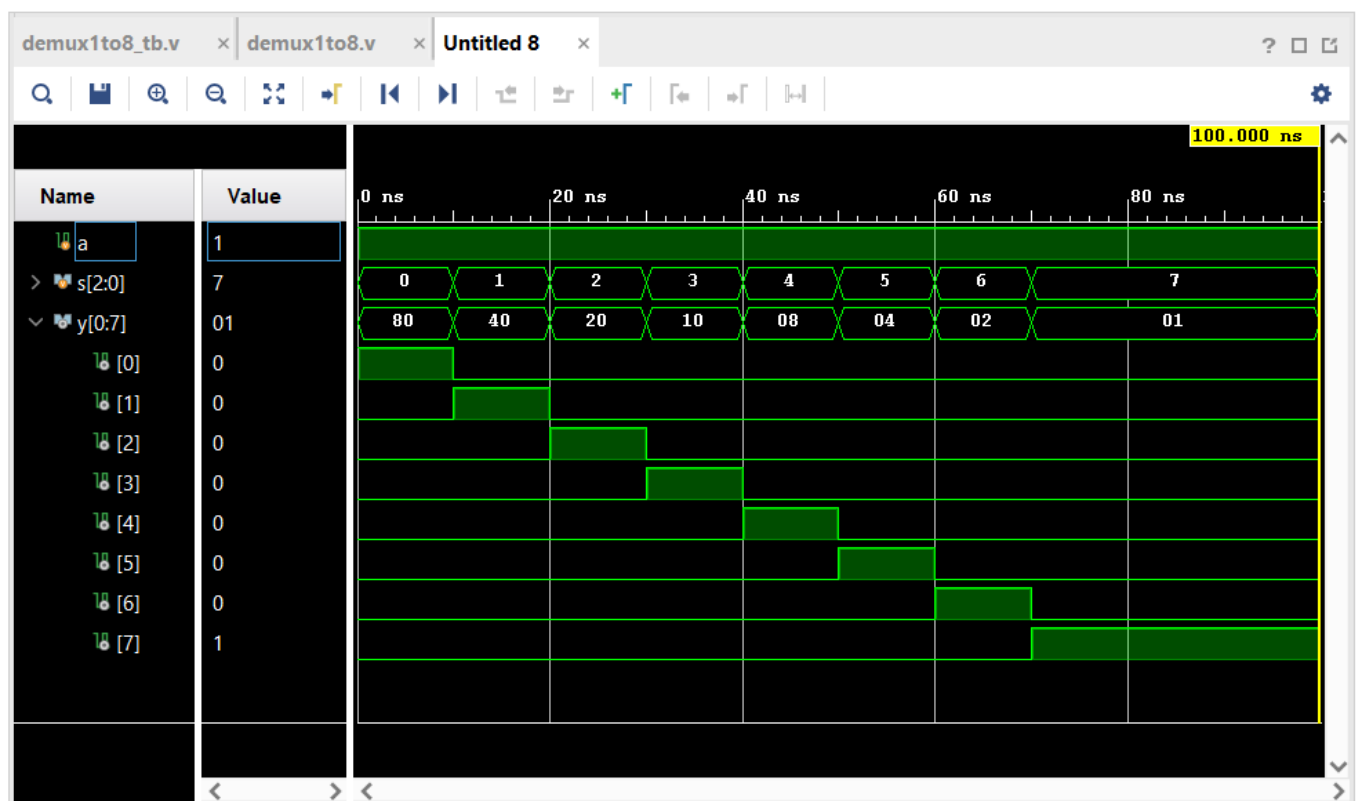
VERILOG CODE FOR 1:8 DEMUX USING CASE STATEMENT

```
module demux1to8(y,s,a);
output reg [0:7]y;
input [2:0]s;
input a;
always @(*)
begin
case(s)
0: y={a,7'b00000000};
1: y={1'b0,a,6'b0000000};
2: y={2'b00,a,5'b000000};
3: y={3'b000,a,4'b00000};
4: y={4'b0000,a,3'b000};
5: y={5'b00000,a,2'b00};
6: y={6'b000000,a,1'b0};
7: y={7'b0000000,a};
default: y=8'b00000000;
endcase
end
endmodule
```

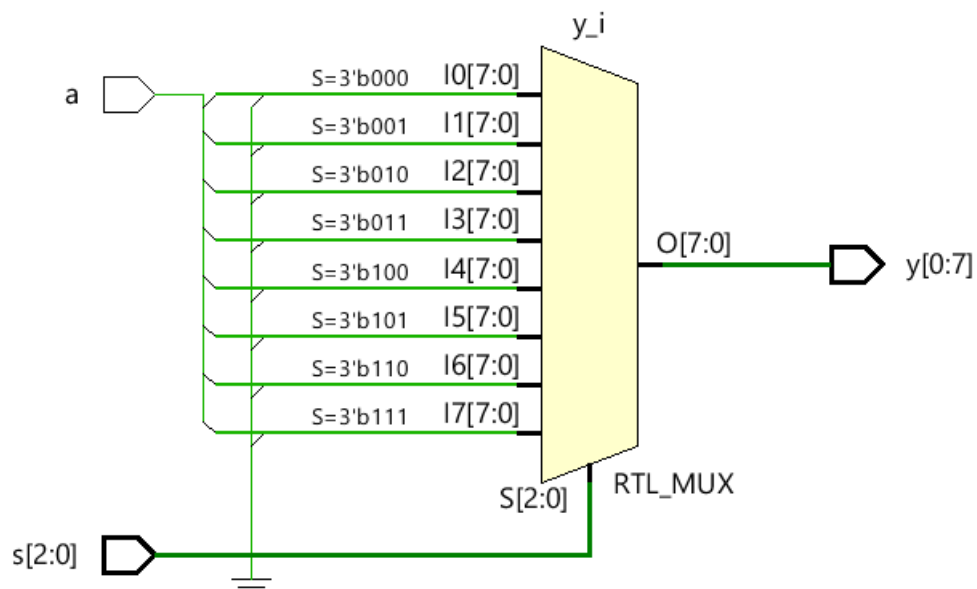
TESTBENCH FOR 1:8 DEMUX

```
module demux1to8_tb( );
reg a;
reg [2:0] s;
wire [0:7] y;
demux1to8 g1(y,s,a);
initial
begin
#0 a=1'b1; s=3'd0;
#10 s=3'd1;
#10 s=3'd2;
#10 s=3'd3;
#10 s=3'd4;
#10 s=3'd5;
#10 s=3'd6;
#10 s=3'd7;
End
initial
$monitor ($time, "a = %b, s=%b, y=%b", a, s, y);
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:



SYNTHESIS RESULTS:



Experiment -4

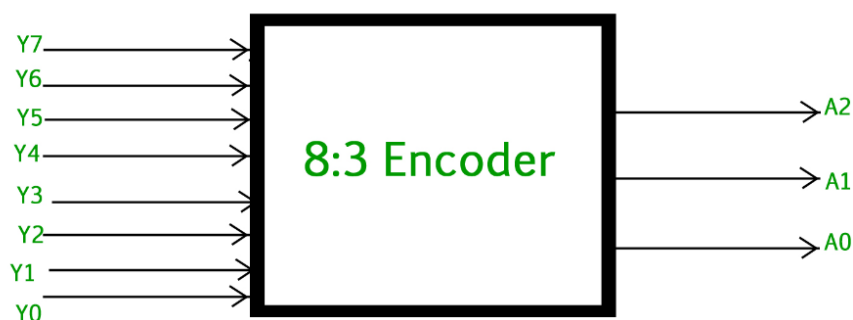
Combinational designs – II (different types of case statements)

c. Encoder with and without Priority: 8:3 and 4:2.

d. Decoder: 3:8 and 2:4.

AIM: To design encoder with and without priority and decoder using Behavioral Modelling style with different case statements and verify the functionalities along with their synthesis and simulation results and also implement of FPGA kits.

ENCODER WITH OUT PRIORITY 8:3

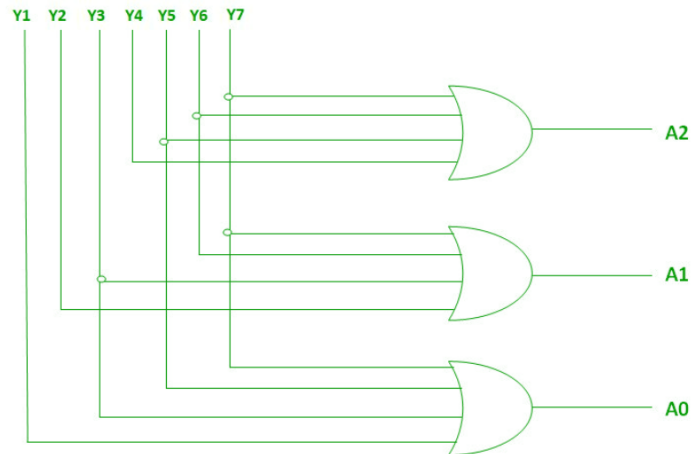


INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A2 = Y7 + Y6 + Y5 + Y4$$

$$A1 = Y7 + Y6 + Y3 + Y2$$

$$A0 = Y7 + Y5 + Y3 + Y1$$



VERILOG CODE FOR 8:3 ENCODER WITHOUT PRIORITY

```

module encwtoutprio(a,en,y);
input [7:0] a;
input en;
output reg [2:0] y;
always@(a or en)
begin
if(!en)
y=3'bzzz;
else
case(a)
8'b00000001:y=3'b000;
8'b00000010:y=3'b001;
8'b00000100:y=3'b010;
8'b00001000:y=3'b011;
8'b00010000:y=3'b100;
8'b00100000:y=3'b101;
8'b01000000:y=3'b110;
8'b10000000:y=3'b111;
endcase
end
endmodule

```

TESTBENCH FOR 8:3 ENCODER WITHOUT PRIORITY

```

module enc_tb( );
reg [7:0] a;
reg en;
wire [2:0] y;

encwtoutprio u1 (a,en,y);
initial
begin
#0
en=1'b0;
a= 8'b00000001;
#30 a=8'b00000010;
#30 a=8'b00000100;

```

```

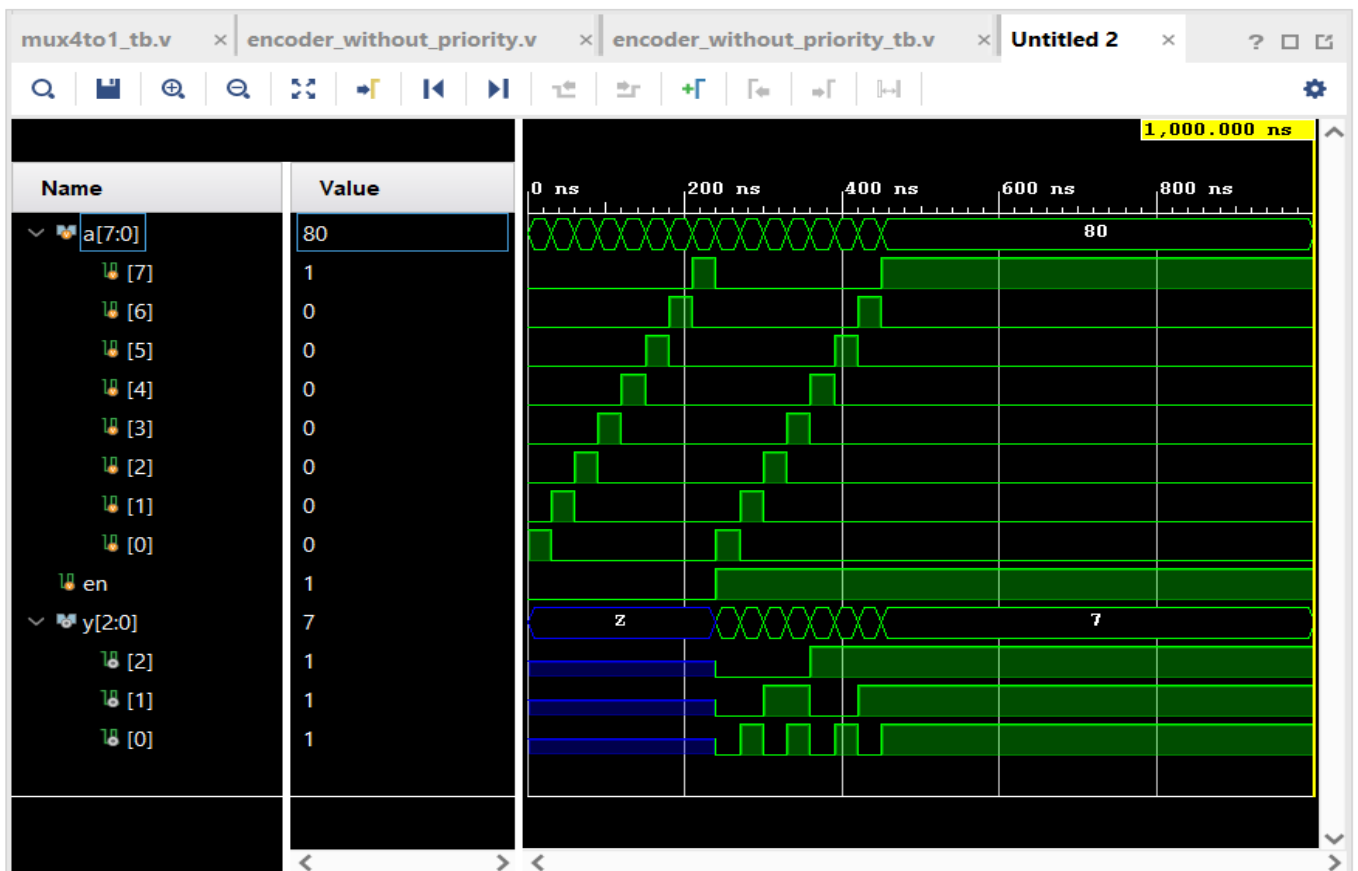
#30 a=8'b00001000;
#30 a=8'b00010000;
#30 a=8'b00100000;
#30 a=8'b01000000;
#30 a=8'b10000000;

#30 en=1'b1;
a= 8'b00000001;
#30 a=8'b00000010;
#30 a=8'b00000100;
#30 a=8'b00001000;
#30 a=8'b00010000;
#30 a=8'b00100000;
#30 a=8'b01000000;
#30 a=8'b10000000;
end
initial
$monitor ( $time, "a= %b, en= %b, y = %b", a, en,y);

initial
#1000 $finish;
endmodule

```

SIMULATION RESULTS:

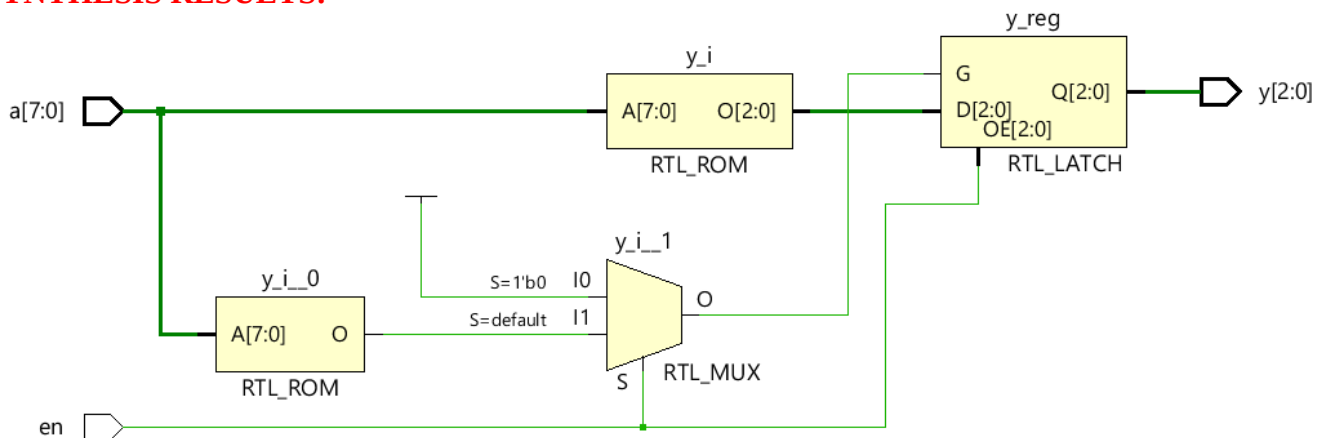


```

Tcl Console  Messages  Log
# send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'
# }
# }
# run 1000ns
0a= 00000001, en= 0, y = zzz
30a= 00000010, en= 0, y = zzz
60a= 00000100, en= 0, y = zzz
90a= 00001000, en= 0, y = zzz
120a= 00010000, en= 0, y = zzz
150a= 00100000, en= 0, y = zzz
180a= 01000000, en= 0, y = zzz
210a= 10000000, en= 0, y = zzz
240a= 00000001, en= 1, y = 000
270a= 00000010, en= 1, y = 001
300a= 00000100, en= 1, y = 010
330a= 00001000, en= 1, y = 011
360a= 00010000, en= 1, y = 100
390a= 00100000, en= 1, y = 101
420a= 01000000, en= 1, y = 110
450a= 10000000, en= 1, y = 111
$finish called at time : 1 us : File "C:/Users/ManasaRR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/encoder_without_priority_tb.v" Line 57
INFO: [USF-XSim-96] XSim completed. Design snapshot 'encoder_without_priority_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:09 . Memory (MB): peak = 894.168 ; gain = 0.000

```

SYNTHESIS RESULTS:



VERILOG CODE FOR 8:3 PRIORITY ENCODER

```

module encwtprio(a,en,y);
input [7:0] a;
input en;
output reg [2:0] y;
always@(a, en)
begin
if(en==1)
y=3'bzzz;
else
casex(a)
8'b00000001:y=3'b000;
8'b0000001x:y=3'b001;
8'b000001xx:y=3'b010;
8'b00001xxx:y=3'b011;
8'b0001xxxx:y=3'b100;
8'b001xxxxx:y=3'b101;
8'b01xxxxxx:y=3'b110;
8'b1xxxxxxx:y=3'b111;
endcase
end
endmodule

```


TESTBENCH FOR 8:3 PRIORITY ENCODER

```
module prienc_tb( );
reg [7:0] a;
reg en;
wire [2:0] y;

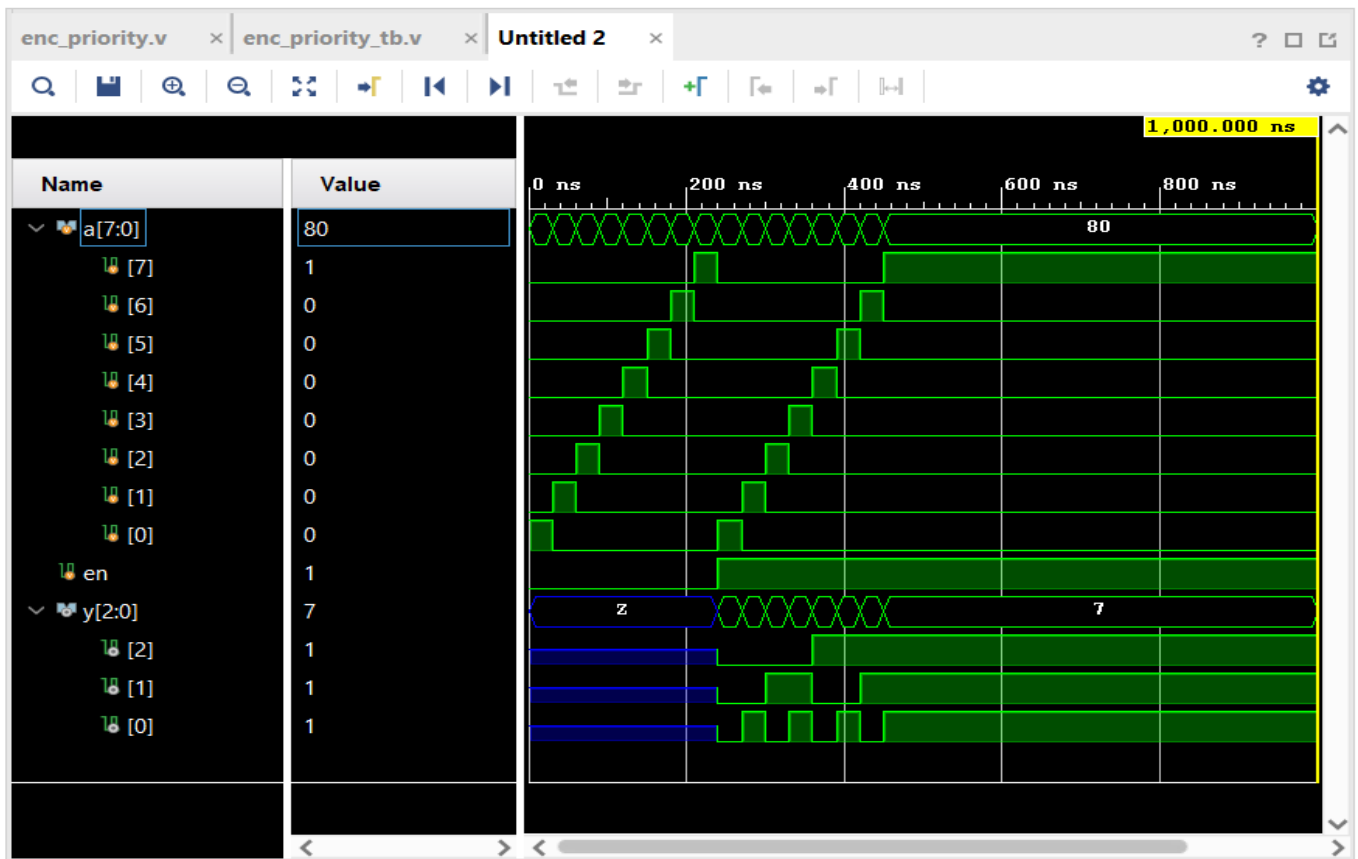
encwtprio u1 (a,en,y);
initial
begin

#0
en=1'b0;
a= 8'b00000001;
#30 a=8'b10000010;
#30 a=8'b00000101;
#30 a=8'b00001011;
#30 a=8'b0001xxxx;
#30 a=8'b00101000;
#30 a=8'b01001000;
#30 a=8'b10001011;

#30 en=1'b1;
a= 8'b00000001;
#30 a=8'b00000010;
#30 a=8'b00000101;
#30 a=8'b00001011;
#30 a=8'b0001xxxx;
#30 a=8'b0010xxxx;
#30 a=8'b01001000;
#30 a=8'b10001011;
end
initial
$monitor ( $time, "a= %b, en= %b, y = %b", a, en, y);

initial
#1000 $finish;
endmodule
```

SIMULATION RESULTS:



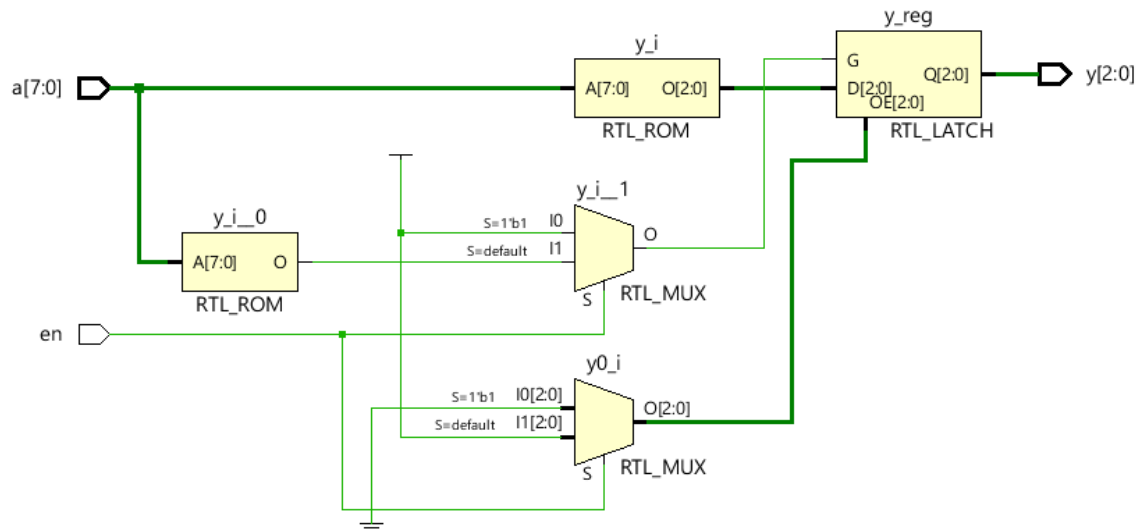
Tcl Console Messages Log

```
# }
# run 1000ns

0a= 00000001, en= 0, y = 000
30a= 10000010, en= 0, y = 111
60a= 00000101, en= 0, y = 010
90a= 00001011, en= 0, y = 011
120a= 0001xxxx, en= 0, y = 100
150a= 00101000, en= 0, y = 101
180a= 01001000, en= 0, y = 110
210a= 10001011, en= 0, y = 111
240a= 00000001, en= 1, y = zzz
270a= 00000010, en= 1, y = zzz
300a= 00000101, en= 1, y = zzz
330a= 00001011, en= 1, y = zzz
360a= 00010000, en= 1, y = zzz
390a= 00101000, en= 1, y = zzz
420a= 01001000, en= 1, y = zzz
450a= 10001011, en= 1, y = zzz

$finish called at time : 1 us : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/enc_priority_tb.v" Line 57
INFO: [USF-XSim-96] XSim completed. Design snapshot 'enc_priority_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:07 . Memory (MB): peak = 1490.555 ; gain = 0.000
```

SYNTHESIS RESULTS:



VERILOG CODE FOR ENCODER 4:2

```
module enc4_2(din, dout );
input [3:0] din;
output [1:0] dout;
reg [1:0] dout;
always @ (din)
```

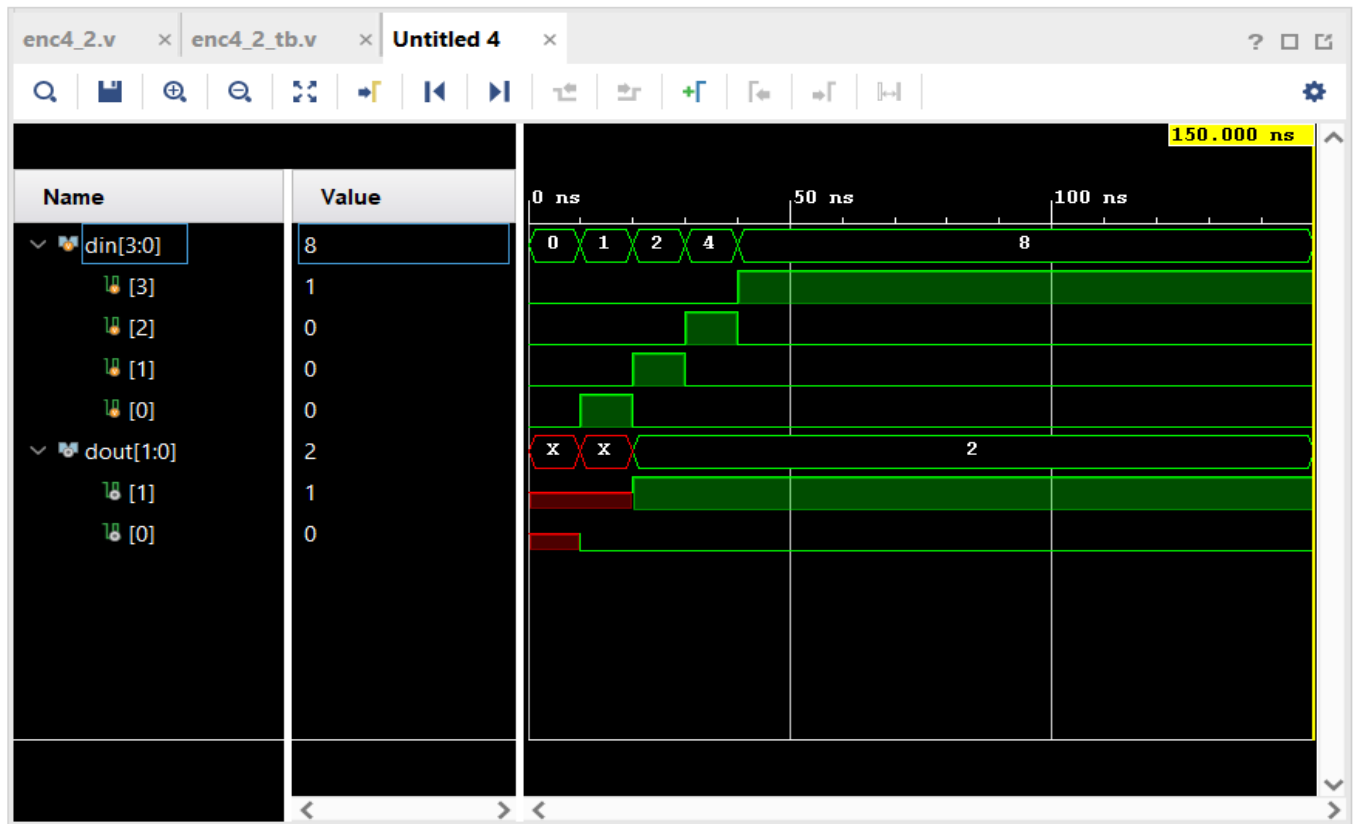
```
case (din)
  1 : dout[0] = 0;
  2 : dout[1] = 1;
  4 : dout[2] = 2;
  8 : dout[3] = 3;
  default : dout = 2'bxx;
endcase
endmodule
```

TEST BENCH FOR ENCODER 4:2

```
module enc4_2_tb( );
reg [3:0] din;
wire [1:0] dout;
enc4_2 g1 (din, dout);
initial
begin
  din = 0;
  #10; din=1;
  #10; din=2;
  #10; din=4;
  #10; din=8;
end
initial
begin
  $monitor($time, "din=%b, dout=%b", din, dout);
end
initial
#150 $finish;
```

endmodule

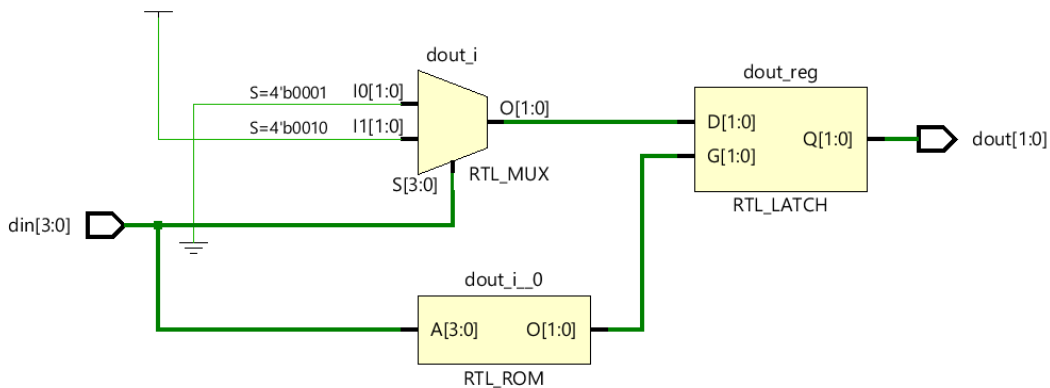
SIMULATION RESULTS:



Tcl Console

```
# send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'
# }
# }
# run 1000ns
    0din=0000, dout=xx
    10din=0001, dout=x0
    20din=0010, dout=10
    30din=0100, dout=10
    40din=1000, dout=10
$finish called at time : 150 ns : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/enc4_2_tb.v" Line 40
INFO: [USF-XSim-96] XSim completed. Design snapshot 'enc4_2_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 1526.812 ; gain = 0.000
```

SYNTHESIS RESULTS:



VERILOG CODE FOR 4:2 PRIORITY ENCODER

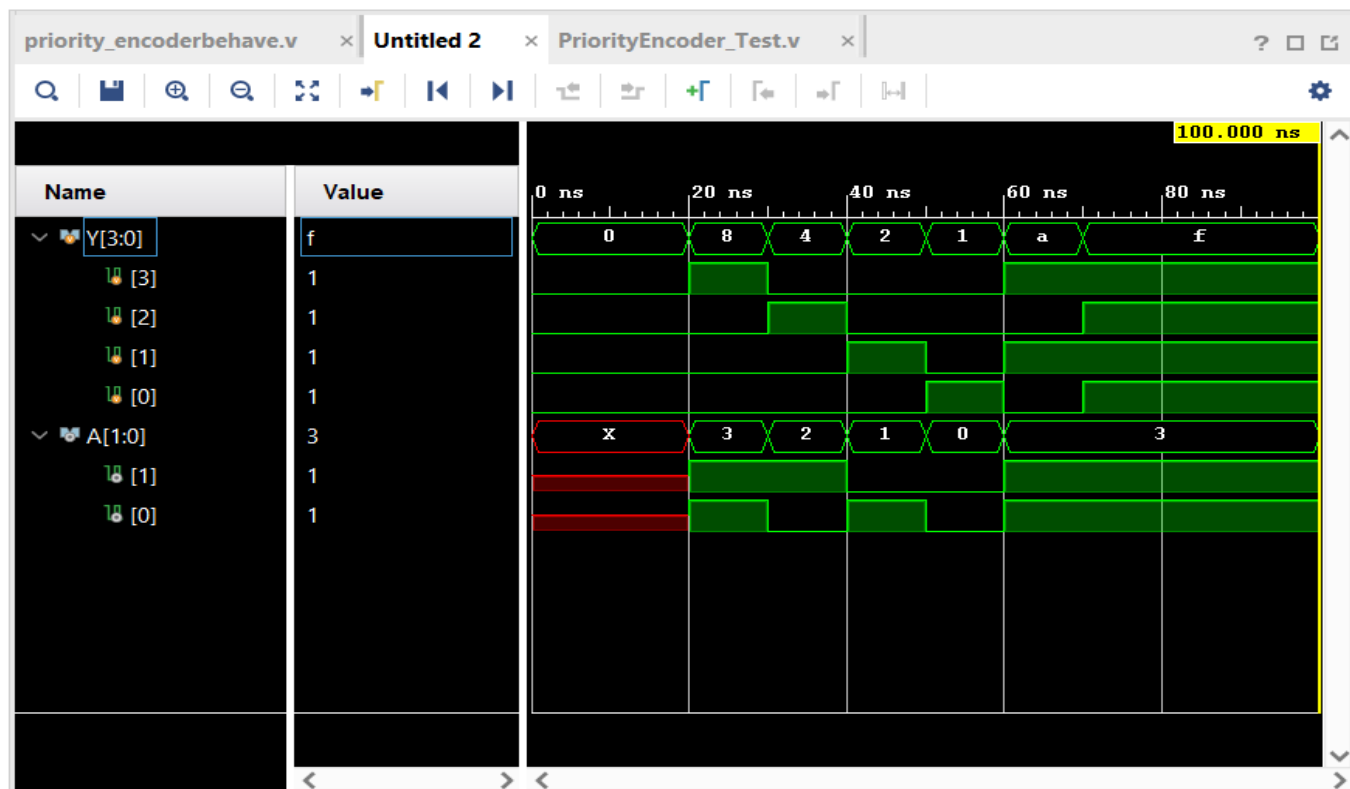
```
module priority_encoder behave(A, Y);
input [3:0]Y;
output reg [1:0]A;
always@(Y)
begin
casex(Y)
4'b0001:A = 2'b00;
4'b001x:A = 2'b01;
4'b01xx:A = 2'b10;
4'b1xxx:A = 2'b11;
default:$display("Error!");
endcase
end
endmodule
```

TESTBENCH FOR 4:2 PRIORITY ENCODER

```
module PriorityEncoder_Test ( );
reg [3:0] Y;
wire [1:0] A;
priority_encoder behave g1 (.Y(Y), .A(A));
initial
begin

#0 Y = 0;
#10 Y = 4'b0000;
#10 Y = 4'b1000;
#10 Y = 4'b0100;
#10 Y = 4'b0010;
#10 Y = 4'b0001;
#10 Y = 4'b1010;
#10 Y = 4'b1111;
end
initial
begin
$monitor("time=", $time, "A=%b : Y=%b", A, Y);
end
initial
#100 $finish;
endmodule
```

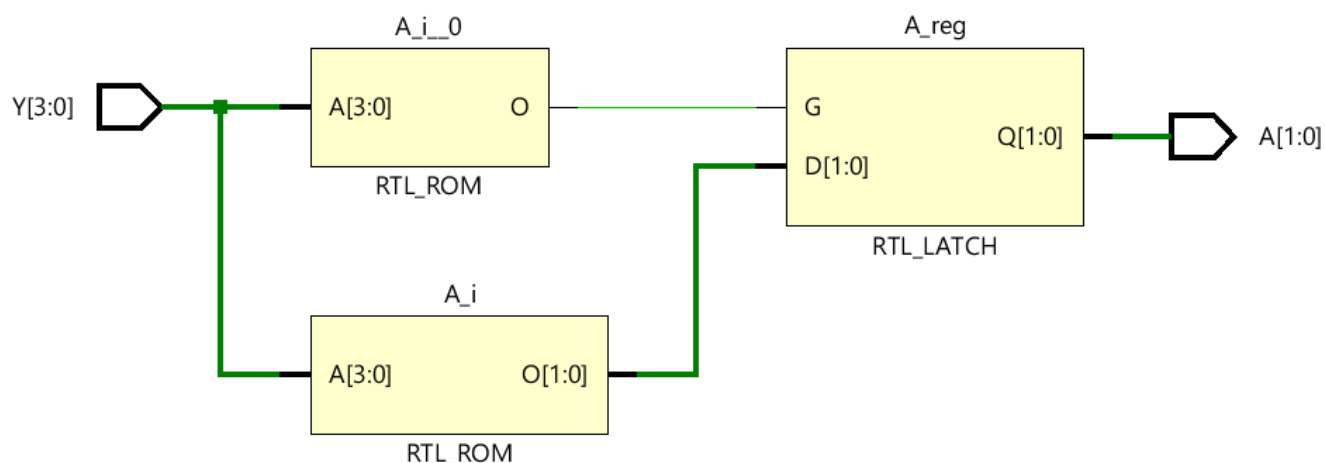
SIMULATION RESULTS:



Tcl Console x Messages Log

```
# } else {
#   send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
# }
# }
# run 1000ns
Error!
time=      0A=xx : Y=0000
time=     20A=11 : Y=1000
time=     30A=10 : Y=0100
time=     40A=01 : Y=0010
time=     50A=00 : Y=0001
time=     60A=11 : Y=1010
time=     70A=11 : Y=1111
$finish called at time : 100 ns : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/PriorityEncoder_Test.v" Line 44
INFO: [USF-XSim-96] XSim completed. Design snapshot 'PriorityEncoder_Test_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

SYNTHESIS RESULTS:



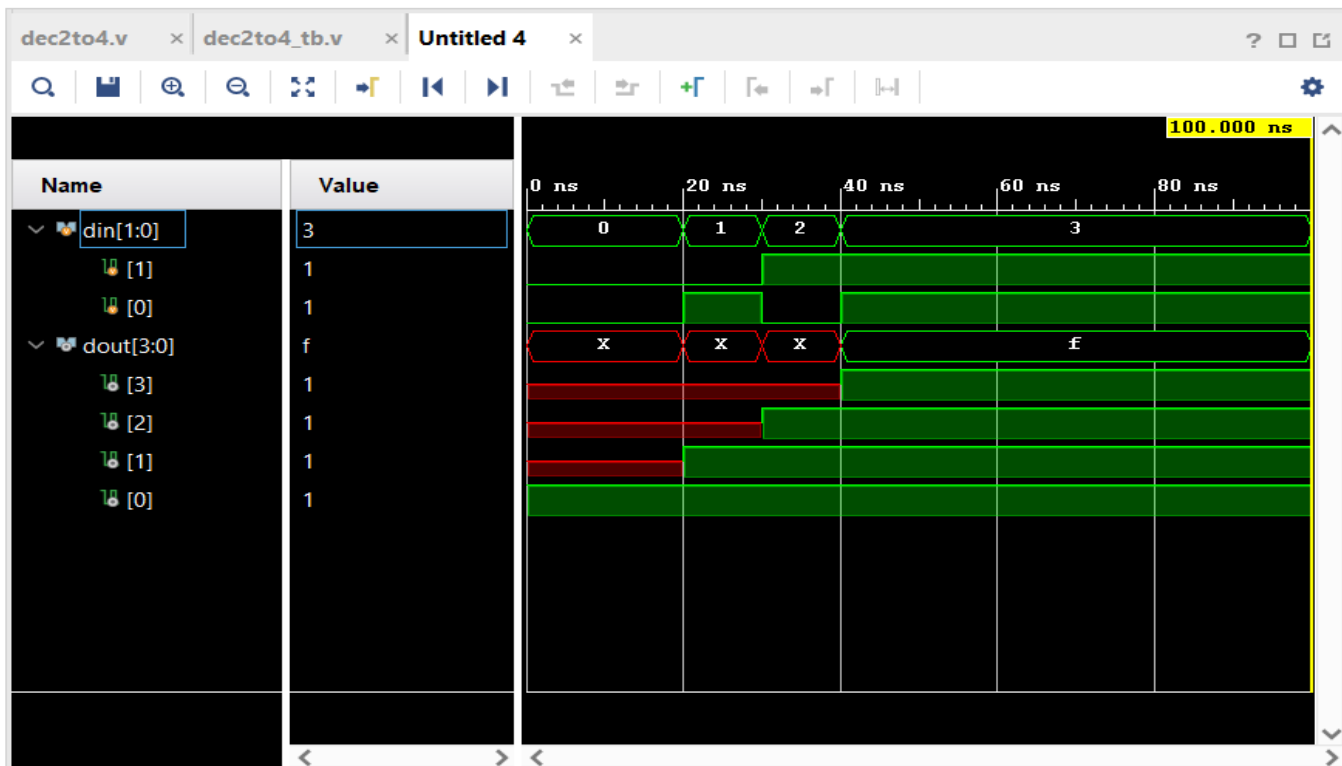
VERILOG CODE FOR DECODER 2:4

```
module dec2to4(din, dout);
input [1:0] din;
output reg [3:0] dout;
always @ (din)
case (din)
    0 : dout[0] = 1;
    1 : dout[1] = 1;
    2 : dout[2] = 1;
    3 : dout[3] = 1;
    default : dout = 4'bxxxx;
endcase
endmodule
```

TEST BENCH FOR DECODER 2:4

```
module dec2to4_tb( );
reg [1:0] din;
wire [3:0] dout;
dec2to4 g1 (din, dout);
initial
begin
#0 din = 0;
#10; din=0;
#10; din=1;
#10; din=2;
#10; din=3;
end
initial
begin
$monitor($time, "din=%b, dout=%b", din, dout);
end
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:

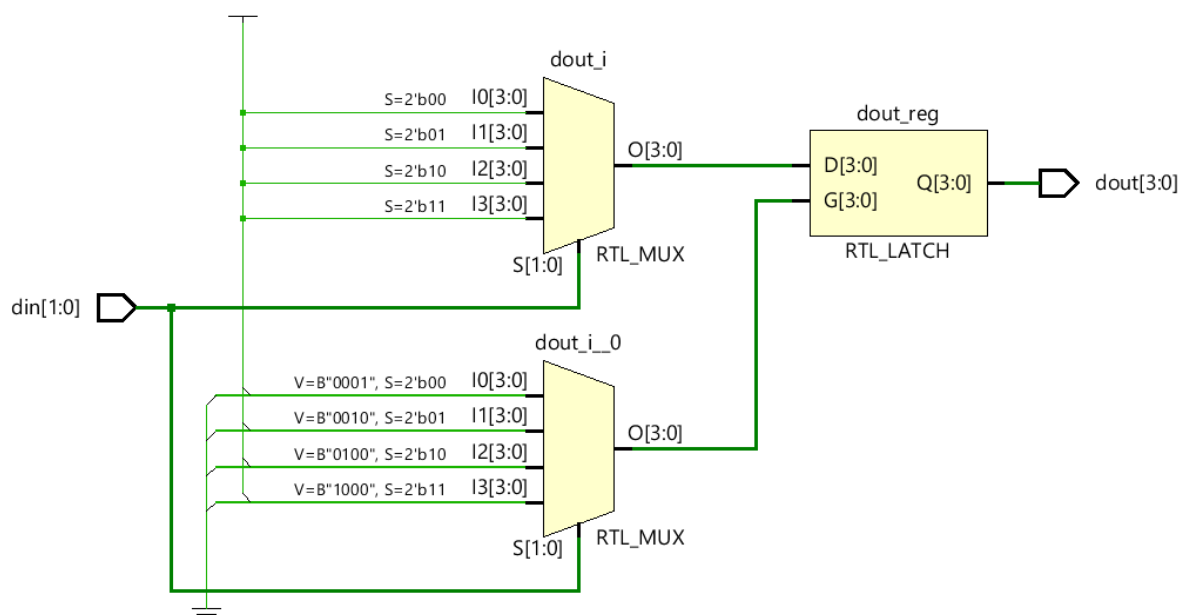


Tcl Console Messages Log

```
# } else {
#   send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'
# }
# }
# run 1000ns
    0din=00, dout=xxx1
    20din=01, dout=xx11
    30din=10, dout=x111
    40din=11, dout=1111

$finish called at time : 100 ns : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srcs/sim_1/new/dec2to4_tb.v" Line 40
INFO: [USF-XSim-96] XSim completed. Design snapshot 'dec2to4_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:08 . Memory (MB): peak = 1443.082 ; gain = 0.000
```

SYNTHESIS RESULTS:



VERILOG CODE FOR DECODER 3: 8

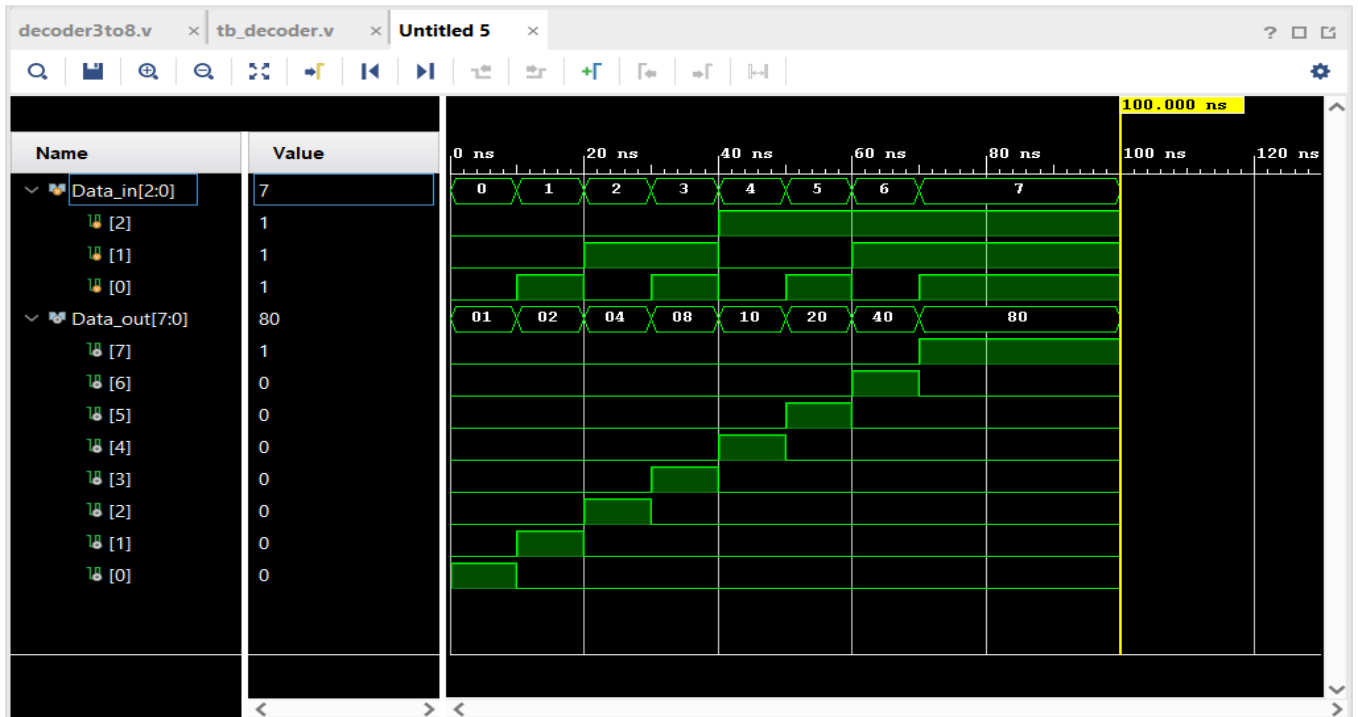
```
module decoder3to8(Data_in, Data_out );
input [2:0] Data_in;
output [7:0] Data_out;
reg [7:0] Data_out;

always @(Data_in)
    case (Data_in)
        3'b000 : Data_out = 8'b00000001;
        3'b001 : Data_out = 8'b00000010;
        3'b010 : Data_out = 8'b00000100;
        3'b011 : Data_out = 8'b00001000;
        3'b100 : Data_out = 8'b00010000;
        3'b101 : Data_out = 8'b00100000;
        3'b110 : Data_out = 8'b01000000;
        3'b111 : Data_out = 8'b10000000;
        default : Data_out = 8'b00000000;
    endcase
endmodule
```

TEST BENCH FOR DECODER 3:8

```
module tb_decoder ( );
reg [2:0] Data_in;
wire [7:0] Data_out;
decoder3to8 g1 (Data_in, Data_out );
initial
begin
#0   Data_in = 3'b000;
#10  Data_in = 3'b001;
#10  Data_in = 3'b010;
#10  Data_in = 3'b011;
#10  Data_in = 3'b100;
#10  Data_in = 3'b101;
#10  Data_in = 3'b110;
#10  Data_in = 3'b111;
end
initial
begin
$monitor($time, "Data_in=%b, Data_out=%b", Data_in, Data_out);
end
initial
#100 $finish;
endmodule
```

SIMULATION RESULTS:

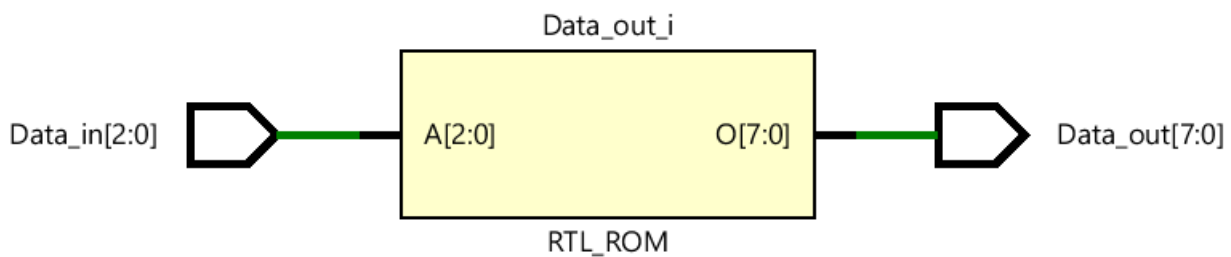


Tcl Console Messages Log

```
# }
# run 1000ns
0Data_in=000, Data_out=000000001
10Data_in=001, Data_out=000000010
20Data_in=010, Data_out=000000100
30Data_in=011, Data_out=000001000
40Data_in=100, Data_out=000100000
50Data_in=101, Data_out=001000000
60Data_in=110, Data_out=010000000
70Data_in=111, Data_out=100000000

$finish called at time : 100 ns : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/tb_decoder.v" Line 43
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_decoder_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 1449.684 ; gain = 0.000
```

SYNTHESIS RESULTS:



Experiment-5

Design of 4 bit ALU

Aim: To design the 4 bit ALU for the following functions and verify its functionality with test bench and implement on FPGA kit.

Table 4.1			The functionality of the 74381 ALU.
Operation	Inputs $s_2 s_1 s_0$	Outputs F	
Clear	0 0 0	0 0 0 0	
B-A	0 0 1	$B - A$	
A-B	0 1 0	$A - B$	
ADD	0 1 1	$A + B$	
XOR	1 0 0	$A \text{ XOR } B$	
OR	1 0 1	$A \text{ OR } B$	
AND	1 1 0	$A \text{ AND } B$	
Preset	1 1 1	1 1 1 1	

VERILOG CODE FOR 4 BIT ALU

```
module alu (S, A, B, F);
input [2:0] S;
input [3:0] A, B;
output reg [3:0] F;
always @ (S, A, B)
case (S)
0: F = 4'b0000;
1: F = B - A;
2: F = A - B;
3: F = A + B;
4: F = A ^ B;
5: F = A | B;
6: F = A & B;
7: F = 4'b1111;
endcase
endmodule
```

TEST BENCH FOR ALU

```
module alu_tb ( );
reg [2:0] S;
reg [3:0] A,B;
wire [3:0] F;
alu g1 (S,A,B, F);
initial
begin
```

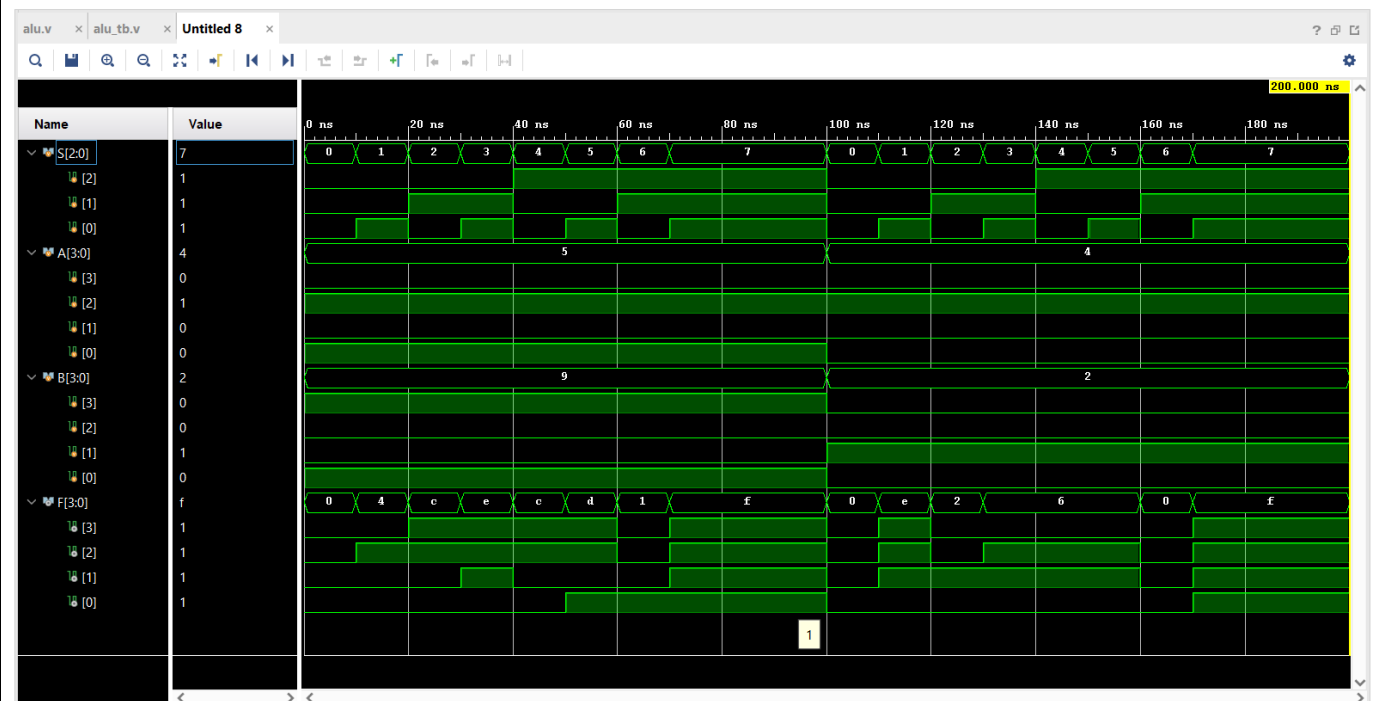
```

#0 A=4'b0101; B=4'b1001; S=3'b000;
#10 S=3'b001;
#10 S=3'b010;
#10 S=3'b011;
#10 S=3'b100;
#10 S=3'b101;
#10 S=3'b110;
#10 S=3'b111;

#30 A=4'b0100; B=4'b0010; S=3'b000;
#10 S=3'b001;
#10 S=3'b010;
#10 S=3'b011;
#10 S=3'b100;
#10 S=3'b101;
#10 S=3'b110;
#10 S=3'b111;
end
initial
begin
$monitor ($time, "S=%b, A=%b, B=%b, F=%b", S,A,B,F);
end
initial
#220 $finish;
Endmodule

```

SIMULATION RESULTS:



Tcl Console Messages Log

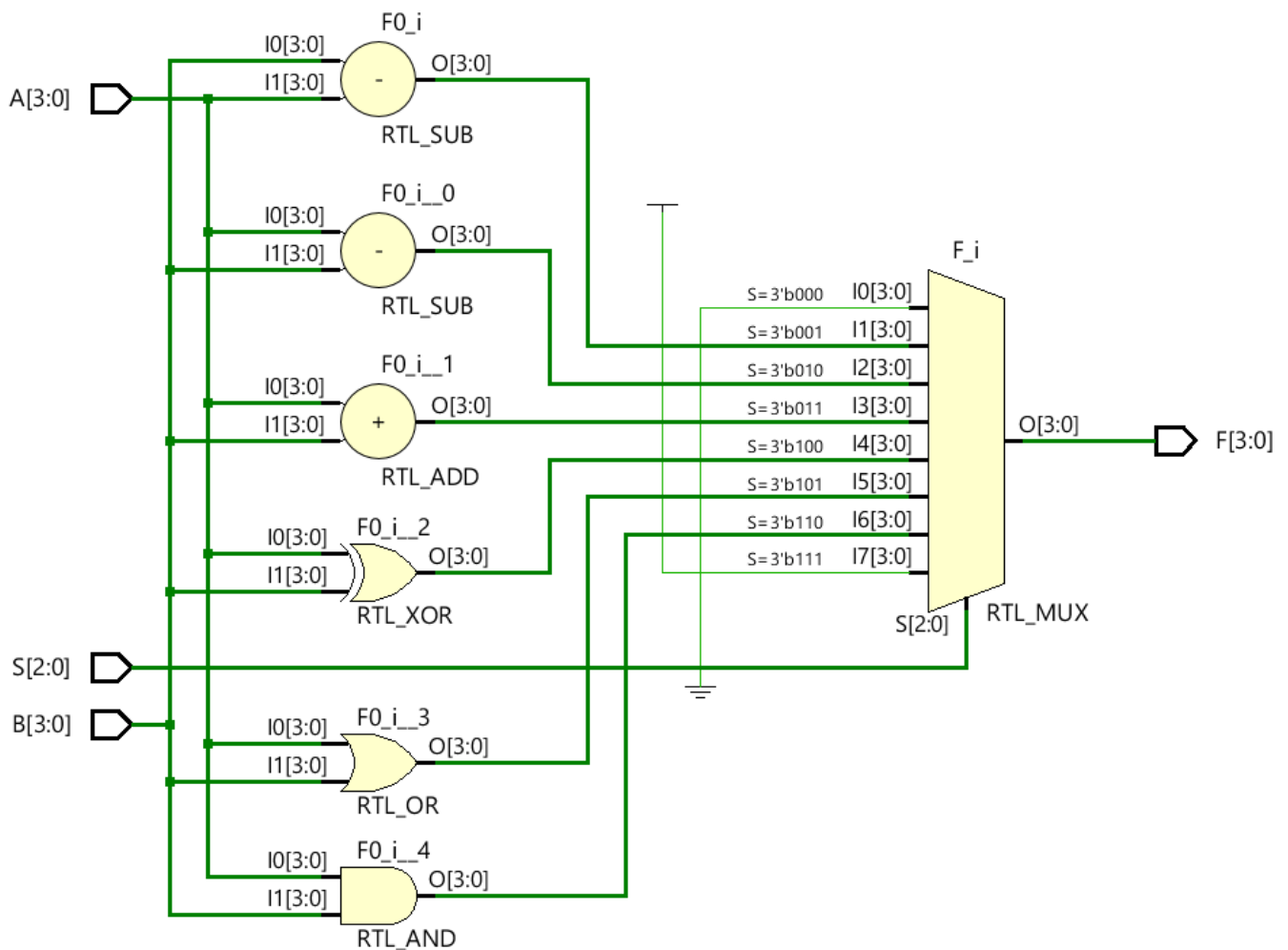
```

# } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'
# }
# }
# run 1000ns

08=000, A=0101, B=1001, F=0000
108=001, A=0101, B=1001, F=0100
208=010, A=0101, B=1001, F=1100
308=011, A=0101, B=1001, F=1110
408=100, A=0101, B=1001, F=1100
508=101, A=0101, B=1001, F=1101
608=110, A=0101, B=1001, F=0001
708=111, A=0101, B=1001, F=1111
1008=000, A=0100, B=0010, F=0000
1108=001, A=0100, B=0010, F=1110
1208=010, A=0100, B=0010, F=0010
1308=011, A=0100, B=0010, F=0110
1408=100, A=0100, B=0010, F=0110
1508=101, A=0100, B=0010, F=0110
1608=110, A=0100, B=0010, F=0000
1708=111, A=0100, B=0010, F=1111
$finish called at time : 200 ns : File "C:/Users/ManasaKR/OneDrive/Documents/DLD_Programs/Project_HA/Project_HA.srscs/sim_1/new/alu_tb.v" Line 53

```

SYNTHESIS RESULTS:

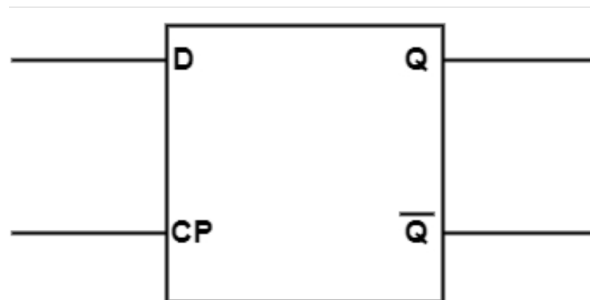


Experiment: 6

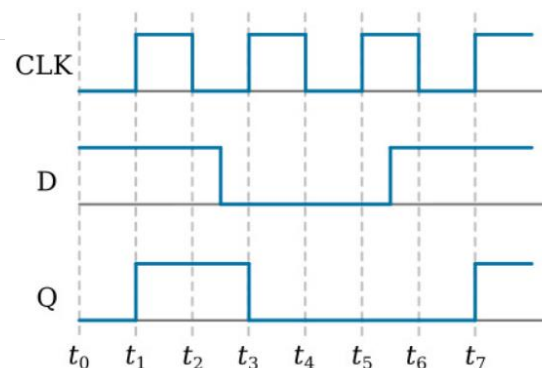
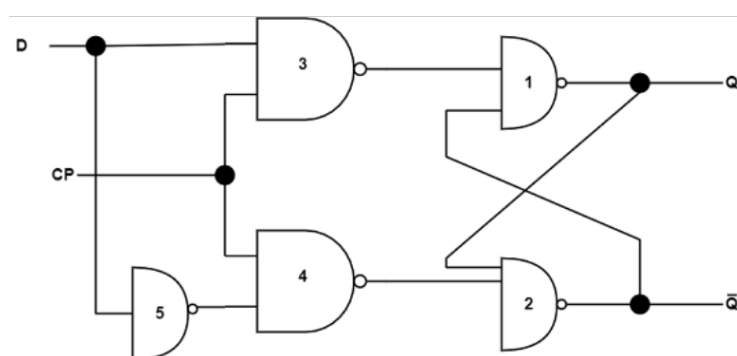
Flip Flop: D FF, T FF, JK FF

Aim: To design different flipflop such as DFF, TFF and JK FF with various combinations of Reset and verify its functionality with testbench and implement on FPGA kit.

D Filp flop



S	D	Q _{N+1}
0	0	0
0	1	1
1	0	0
1	1	1



VERILOG CODE FOR D FILP FLOP WITH OUT RESET

```
module dff (D, Clock, Q);
input D, Clock;
output reg Q;
always @(posedge Clock)
Q = D;
endmodule
```

TESTBENCH FOR D FLIP FLOP

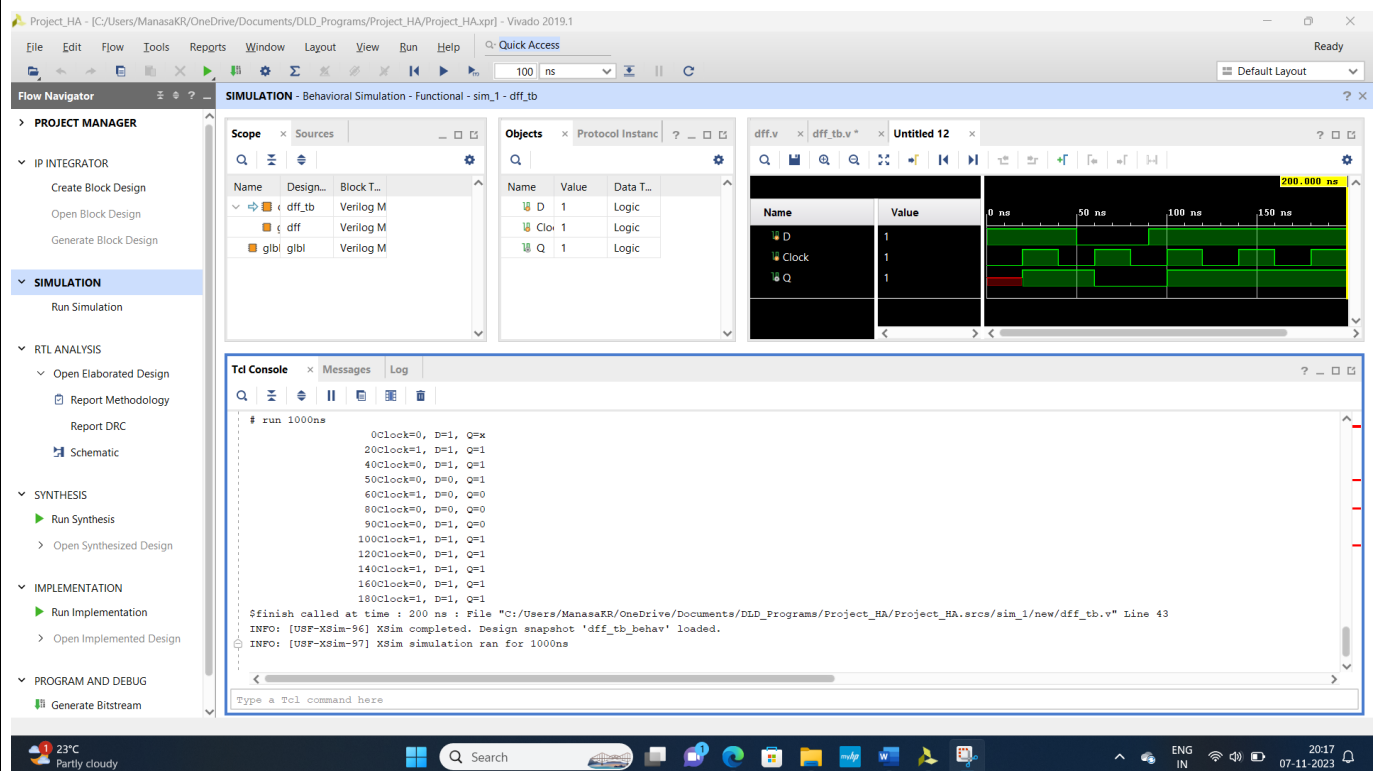
```
module dff_tb ();
reg D,Clock;
wire Q;
dff g1 (D,Clock, Q);
initial
begin
Clock=1'b0;
forever #20 Clock=~Clock;
end
```

```

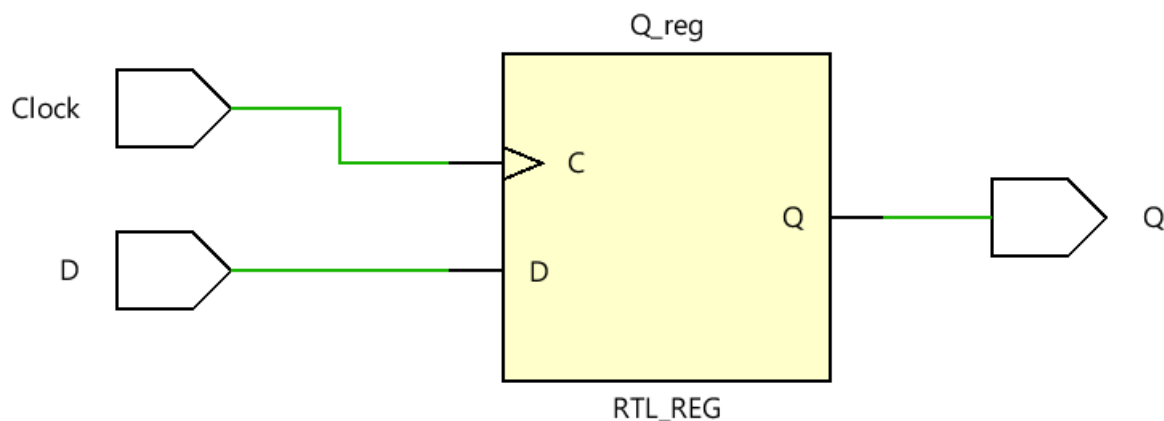
initial
begin
#0 D=1'b1;
#50 D=1'b0;
#40 D=1'b1;
end
initial
begin
$monitor($time, "Clock=%b, D=%b, Q=%b", Clock, D, Q);
end
initial
#200 $finish;
endmodule

```

SIMULATION RESULTS:

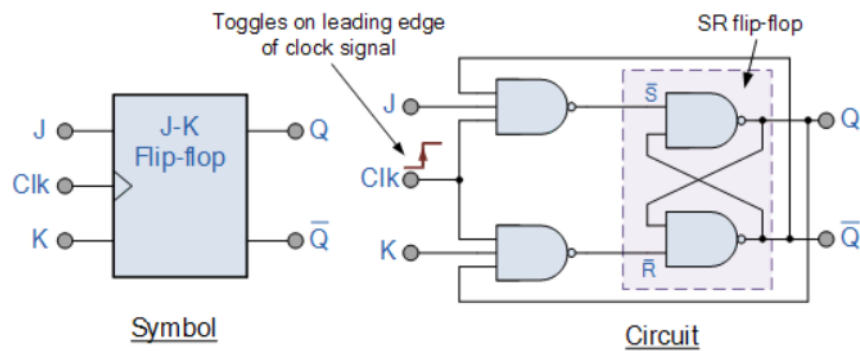


SYNTHESIS RESULTS:



J K Flip Flop

The Basic JK Flip-flop



J	K	Q _n	Q(n+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

J \ KQ _n				
	00	01	11	10
J' 0		1		
J 1	1	1		1

$$Q(n+1) = JQ_n' + K'Q_n$$

Synchronous resets are triggered on active edge of a clock

VERILOG CODE FOR J K FLIP FLOP WITH SYNCHRONOUS RESET

```
module jk_flipflop (clk, rst_n, j,k, q, q_bar );
input clk, rst_n, j,k;
output reg q, q_bar;

always@(posedge clk)
begin // for synchronous reset
    if(!rst_n)
    {q,q_bar} <= {1'b0,1'b1};
    else
    begin
        case({j,k})
            2'b00: {q,q_bar} <= {q,q_bar}; // No change
            2'b01: {q,q_bar} <= {1'b0,1'b1}; // reset
            2'b10: {q,q_bar} <= {1'b1,1'b0}; // set
            2'b11: {q,q_bar} <= {q_bar,q}; // Toggle
        endcase
    end
end
endmodule
```

TESTBENCH FOR J K FLIP FLOP

```
module jkff_tb ();
reg clk, rst_n, j,k;
wire q,q_bar;
jk_flipflop g1 (clk, rst_n, j,k, q, q_bar );

initial
begin
    clk=1'b0;
    forever #5 clk=~clk;
end

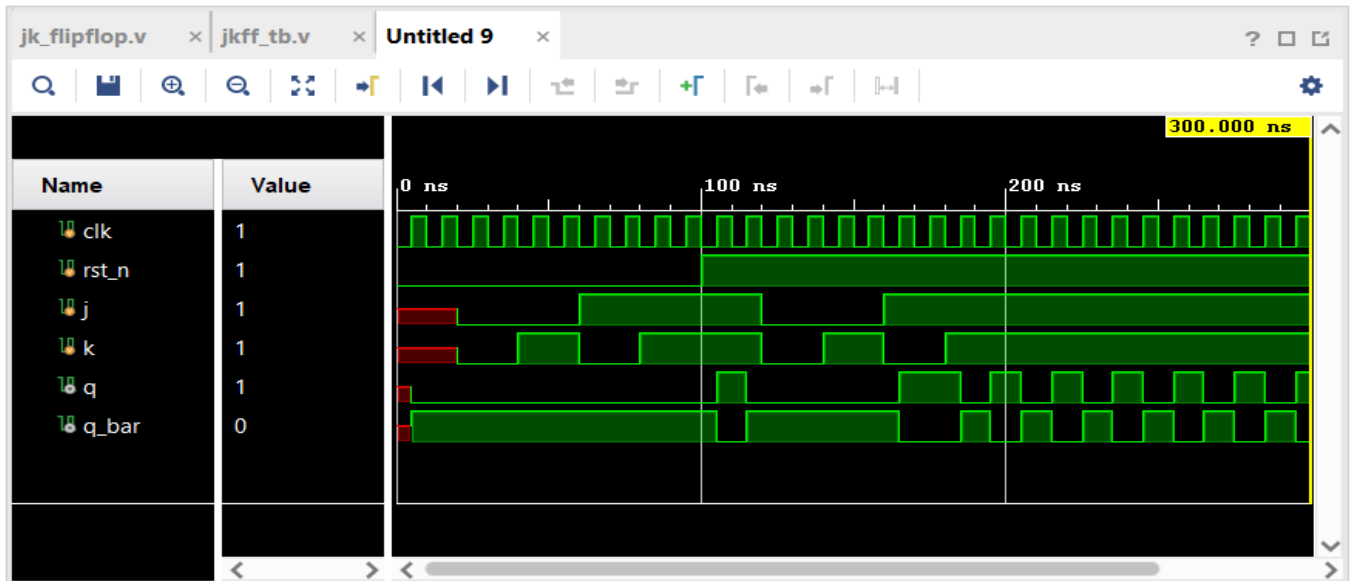
initial
begin
    #0 rst_n =0;
    #20 j=0; k=0;
    #20 j=0; k=1;
    #20 j=1;k=0;
    #20 j=1;k=1;
    #20 rst_n =1;
    #20 j=0; k=0;
    #20 j=0; k=1;
    #20 j=1;k=0;
    #20 j=1;k=1;
end

initial
begin
    $monitor($time, "clk=%b, rst_n=%b, j=%b, k=%b, q=%b, q_bar=%b", clk, rst_n,j,k,q,q_bar);
end

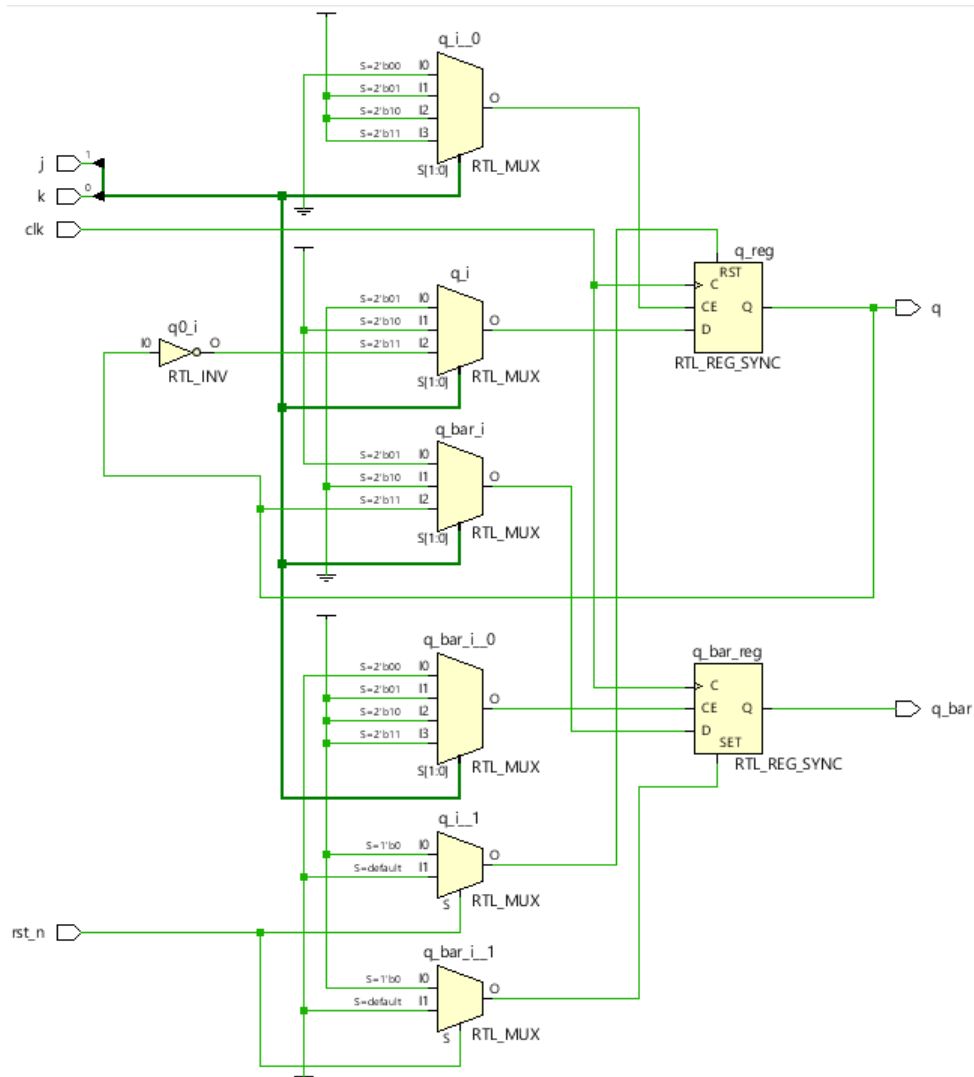
initial
```

```
#200 $finish;
endmodule
```

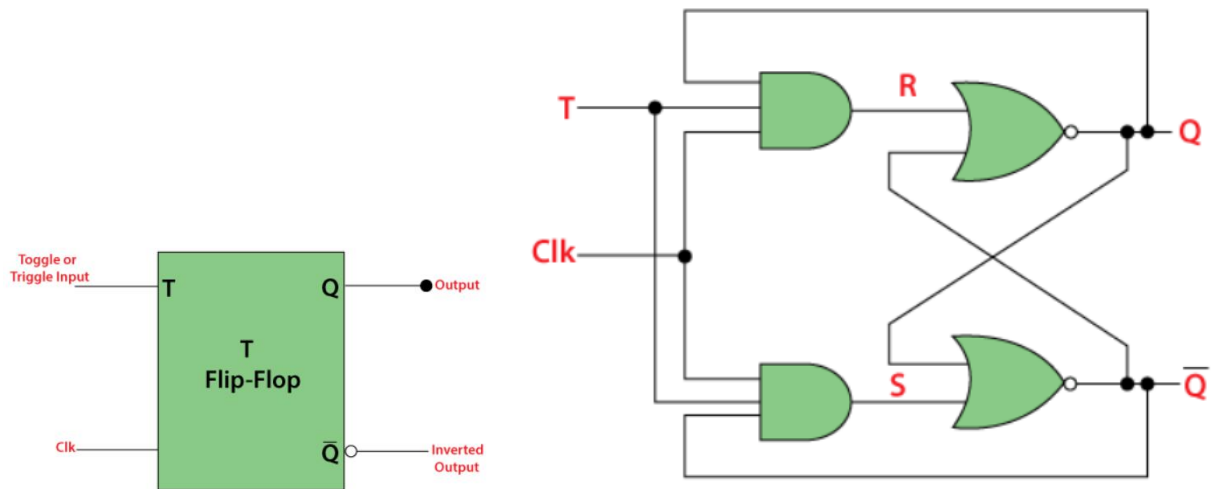
SIMULATION RESULTS:



SYNTHESIS RESULTS:



T Flip Flop



T	Previous		Next	
	Q	Q'	Q	Q'
0	0	1	0	1
0	1	0	1	0
1	0	1	1	0
1	1	0	0	1

$$Q+1 = T \cdot \bar{Q} + \bar{T} \cdot Q$$

VERILOG CODE FOR T FLIP FLOP WITH ASYNCHRONOUS RESET

```

module tff (clk, rstn, t, q);
input clk,rstn,t;
output reg q;
    always @ (posedge clk or negedge rstn)
begin
    if (!rstn)
        q <= 0;
    else
        if (t)
            q <= ~q;
        else
            q <= q;
    end
endmodule
    
```

TEST BENCH FOR T FLIP FLOP

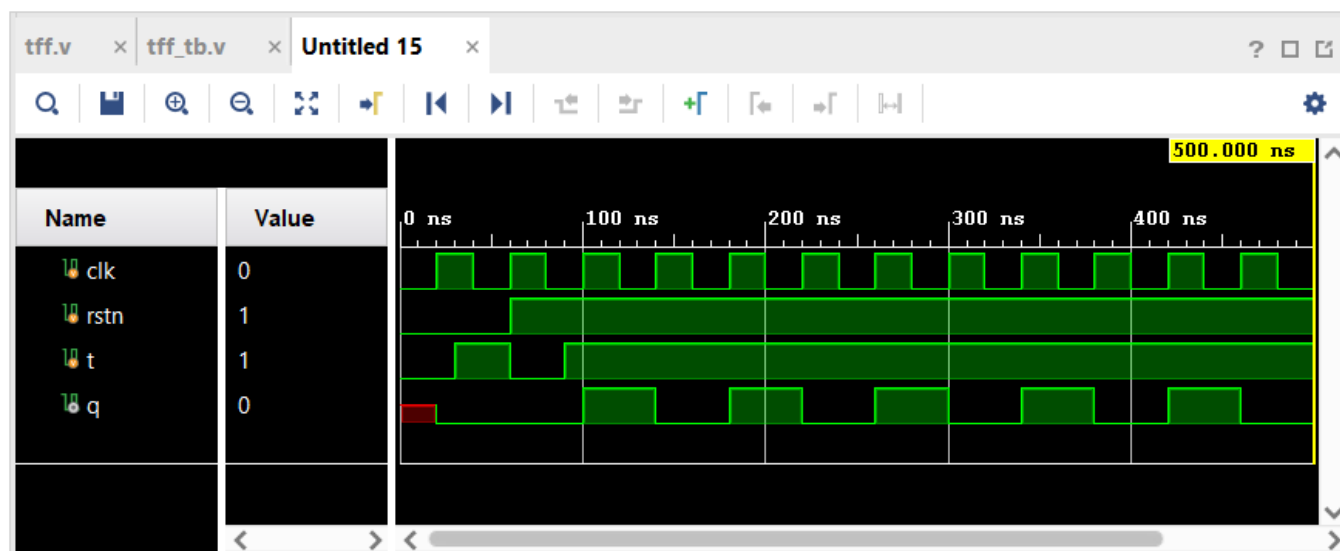
```
module tff_tb ( );
  reg clk;
  reg rstn;
  reg t;
  wire q;

  tff g1 (clk, rstn, t,q);

  initial
  begin
    clk=1'b0;
    forever #20 clk=~clk;
  end

  initial
  begin
    #0 rstn =0; t=0;
    #30 t=1;
    #30 rstn=1; t=0;
    #30 t=1;
  end
  initial
  begin
    $monitor($time "clk=%b, rstnt=%b, t=%b, q=%b", clk, rstn,t,q);
  end
  initial
  #500 $finish;
endmodule
```

SIMULATION RESULTS:



Tcl Console

Messages

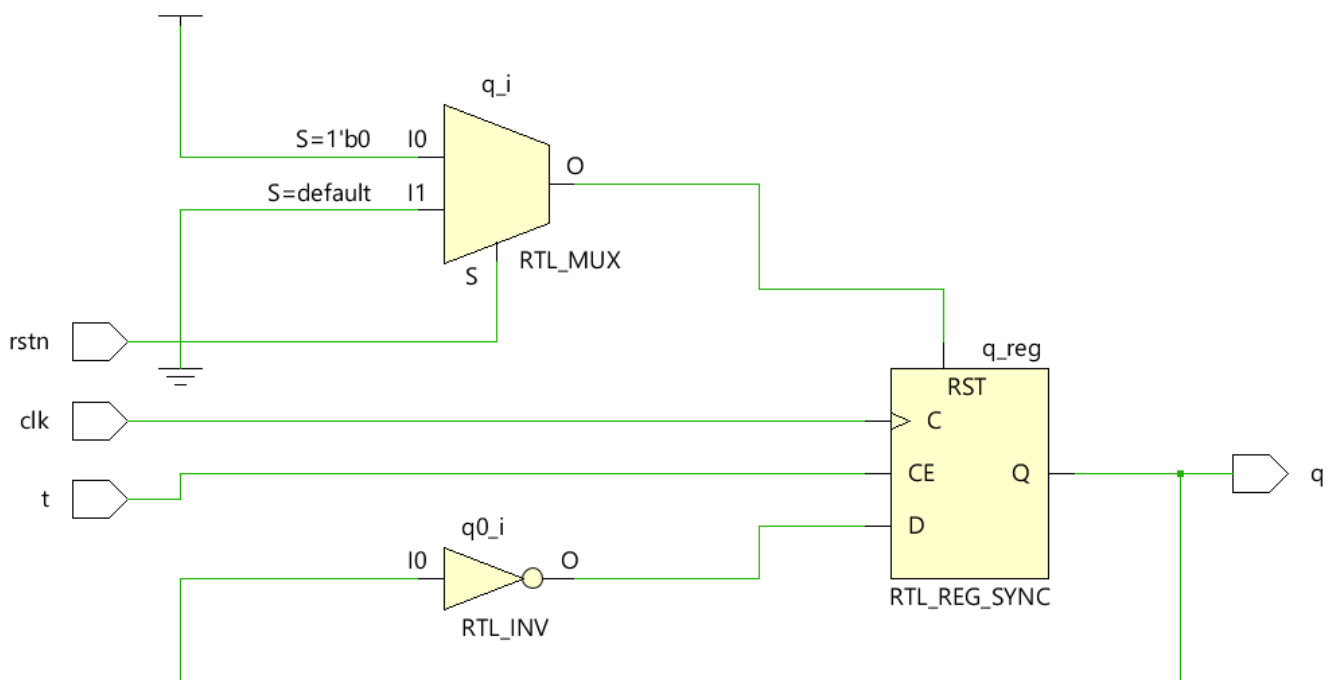
Log



run 1000ns

```
0clk=0, rstnt=0, t=0, q=x
20clk=1, rstnt=0, t=0, q=0
30clk=1, rstnt=0, t=1, q=0
40clk=0, rstnt=0, t=1, q=0
60clk=1, rstnt=1, t=0, q=0
80clk=0, rstnt=1, t=0, q=0
90clk=0, rstnt=1, t=1, q=0
100clk=1, rstnt=1, t=1, q=1
120clk=0, rstnt=1, t=1, q=1
140clk=1, rstnt=1, t=1, q=0
160clk=0, rstnt=1, t=1, q=0
180clk=1, rstnt=1, t=1, q=1
200clk=0, rstnt=1, t=1, q=1
220clk=1, rstnt=1, t=1, q=0
240clk=0, rstnt=1, t=1, q=0
260clk=1, rstnt=1, t=1, q=1
280clk=0, rstnt=1, t=1, q=1
300clk=1, rstnt=1, t=1, q=0
320clk=0, rstnt=1, t=1, q=0
340clk=1, rstnt=1, t=1, q=1
360clk=0, rstnt=1, t=1, q=1
380clk=1, rstnt=1, t=1, q=0
400clk=0, rstnt=1, t=1, q=0
420clk=1, rstnt=1, t=1, q=1
```

SYNTHESIS RESULTS:



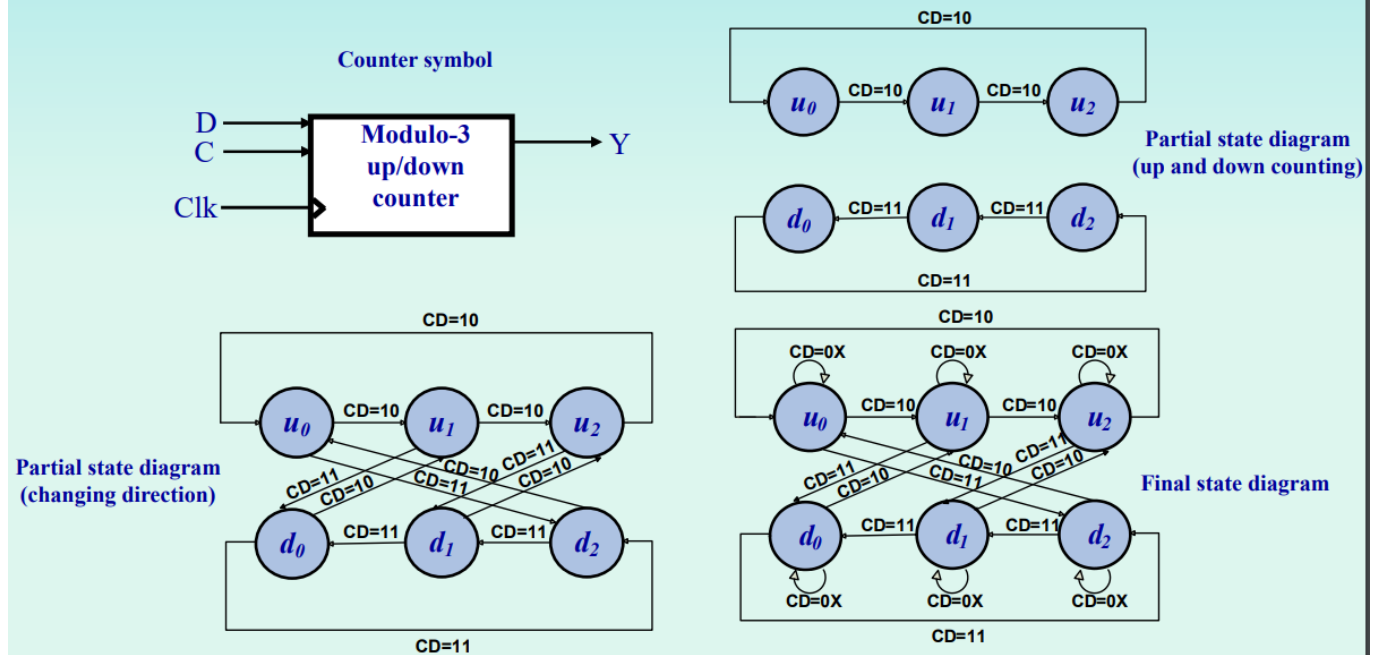
Experiment 8

Design of Mod – n Up/Down Counter with Synchronous reset

Aim : Design of mod n counter using FSM with synchronous reset and implementing on FPGA kit

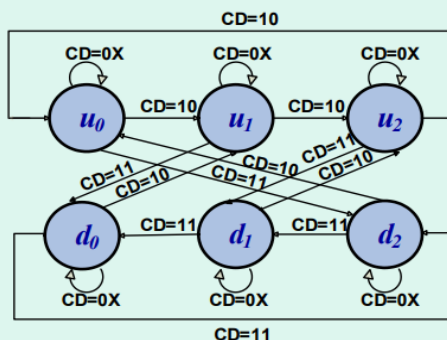
Example: Modulo-3 up-down counter

Problem: Derive the FSM diagram for an up-down, modulo-3 counter. The counter has two inputs: count enable (C) and count direction (D). When C=1, the counter will count in the direction specified by D, and it will stop counting when C=0. the counter will count up when D=0 and down when D=1. The counter has one output Y which will be asserted when the counter reaches 2 while counting up, or when it reaches 0 while counting down.



PRESENT STATE	NEXT STATE / OUTPUT		
	CD=0X	CD=10	CD=11
u_0	$u_0 / 0$	$u_1 / 0$	$d_2 / 1$
u_1	$u_1 / 0$	$u_2 / 0$	$d_0 / 0$
u_2	$u_2 / 0$	$u_0 / 1$	$d_1 / 0$
d_0	$d_0 / 0$	$u_1 / 0$	$d_2 / 1$
d_1	$d_1 / 0$	$u_2 / 0$	$d_0 / 0$
d_2	$d_2 / 0$	$u_0 / 1$	$d_1 / 0$

Initial next-state/output table



Partition into arrays with the same output

$(u_0, u_1, u_2, d_0, d_1, d_2)$						
CD = 0X	0	0	0	0	0	0
10	0	0	1	0	0	1
11	1	0	0	1	0	0

Output values

Partition into arrays with the same next state

CD = 0X	$G_0 G_0$	$G_1 G_1$	$G_2 G_2$
10	$G_1 G_1$	$G_2 G_2$	$G_0 G_0$
11	$G_2 G_2$	$G_0 G_0$	$G_1 G_1$

Next states

Partitioning into equivalence classes

PRESENT STATE	NEXT STATE / OUTPUT		
	CD=0X	CD=10	CD=11
G_0	$G_0 / 0$	$G_1 / 0$	$G_2 / 1$
G_1	$G_1 / 0$	$G_2 / 0$	$G_0 / 0$
G_2	$G_2 / 0$	$G_0 / 1$	$G_1 / 0$

Final next-state/output table

The "parameter N=5 and width =3" can be changed for different value of modulus

VERILOG CODE MOD 5 UPDOWN COUNTER SYNCHRONOUS RESET

```
module counter_syn(clk,reset,up_down,count);
parameter N=5;
parameter width=3;
input clk,reset,up_down;
output reg [width-1:0]count;

always @(posedge clk)
begin
if (reset)
count<=0;
else if (up_down)
begin
if(count==(N-1))
count<=0;
else
count<=count+1;
end
else
begin
count<=count-1;
if(count==0)
count<=(N-1);
end
end

endmodule
```

TESTBENCH FOR MOD 5 UPDOWN COUNTER SYNCHRONOUS RESET

```
module counter_tb();
parameter N=5,width=3;
reg clk,reset,up_down;
wire [width-1:0]count;

counter_syn G1(clk,reset,up_down,count);

initial
begin
clk=1'b0;
forever #10 clk=~clk;
end

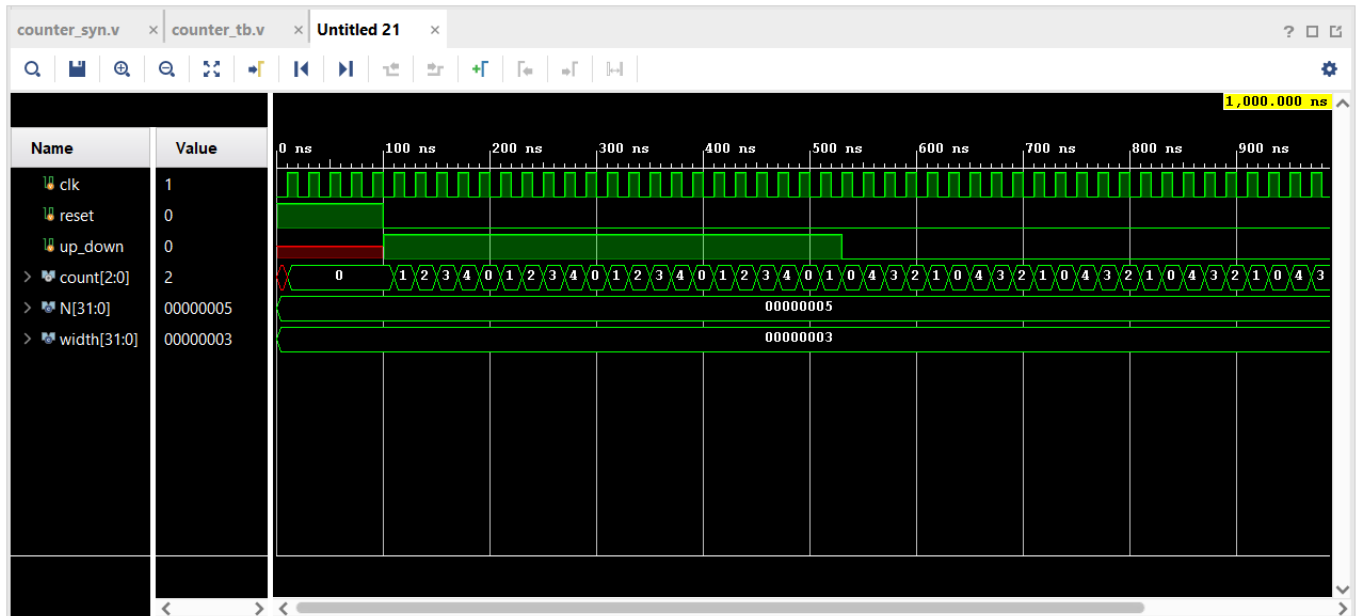
initial
begin
#0 clk=0;reset=1;
#100 reset=0;up_down=1;
#430 up_down=0;
end

initial
```

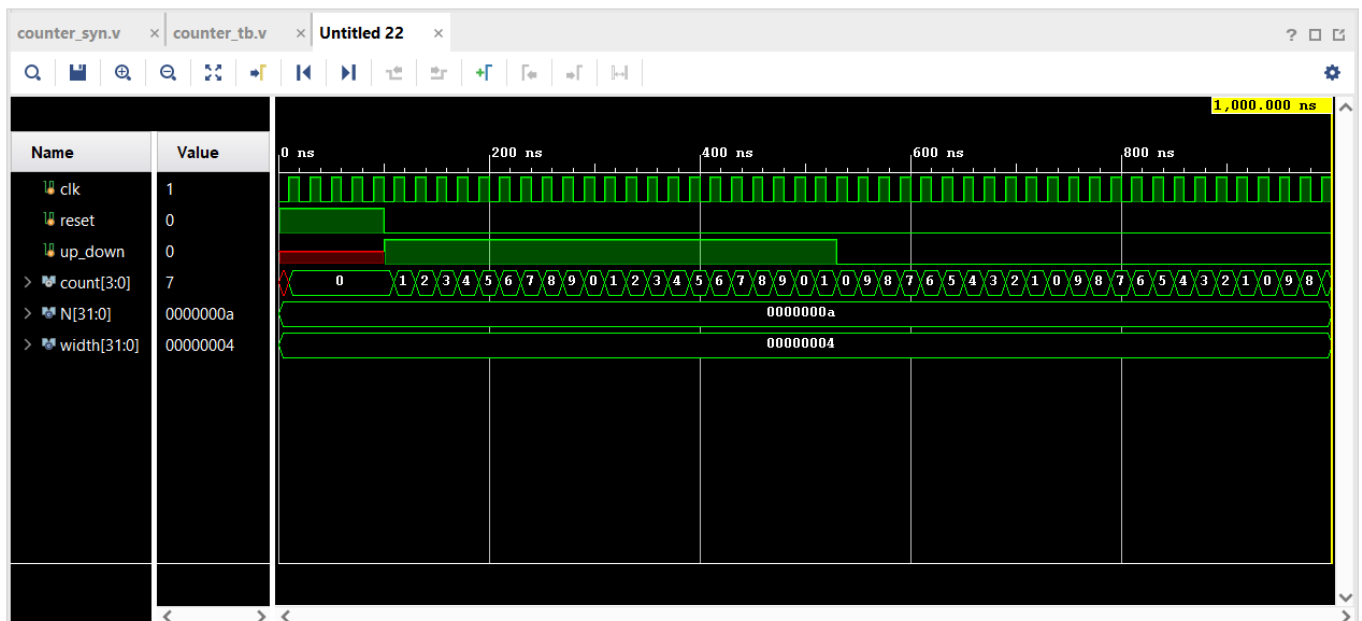
```
#1000 $finish;
endmodule
```

SIMULATION RESULTS:

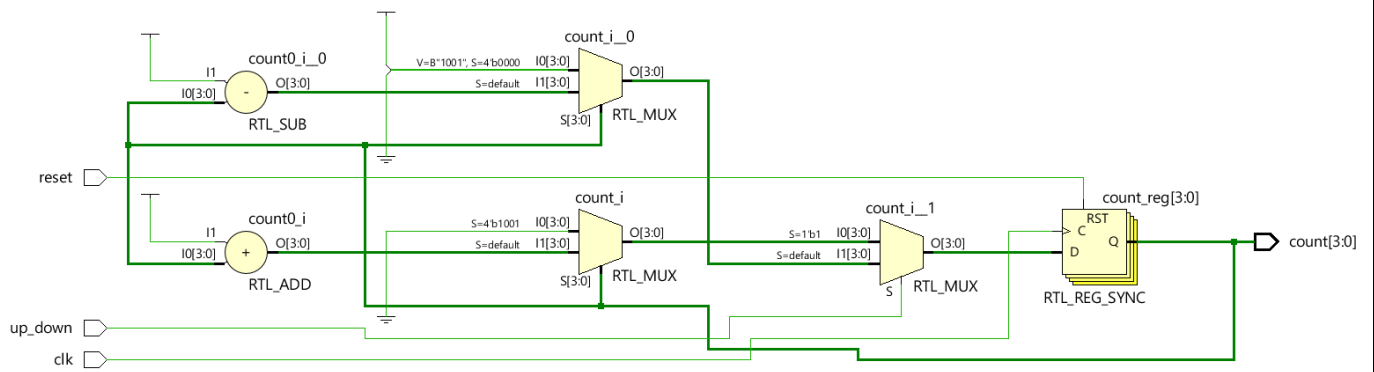
Simulation result for mod 5 counter



Simulation result for mod 10 counter



SYNTHESIS RESULTS:



Experiment 9

Design of Mod – n Up/Down Counter with asynchronous reset

Aim : Design of mod n counter using FSM with asynchronous reset and implementing on FPGA kit

VERILOG CODE MOD 5 UPDOWN COUNTER AYNCHRONOUS RESET

```
module counter_asyn(clk,reset,up_down,count);
parameter N=5;
parameter width=3;
input clk,reset,up_down;
output reg [width-1:0]count;
```

```
always @(posedge clk or negedge reset)
```

```
begin
```

```
if (reset)
```

```
count<=0;
```

```
else if (up_down)
```

```
begin
```

```
if(count==(N-1))
```

```
count<=0;
```

```
else
```

```
count<=count+1;
```

```
end
```

```
else
```

```
begin
```

```
count<=count-1;
```

```
if(count==0)
```

```
count<=(N-1);
```

```
end
```

```
end
```

```
endmodule
```

TESTBENCH FOR MOD 5 UPDOWN COUNTER ASYNCHRONOUS RESET

```
module counter_tb();
parameter N=5,width=3;
reg clk,reset,up_down;
wire [width-1:0]count;
```

```
counter_asyn G1(clk,reset,up_down,count);
```

```
initial
```

```
begin
```

```
clk=1'b0;
```

```
forever #10 clk=~clk;
```

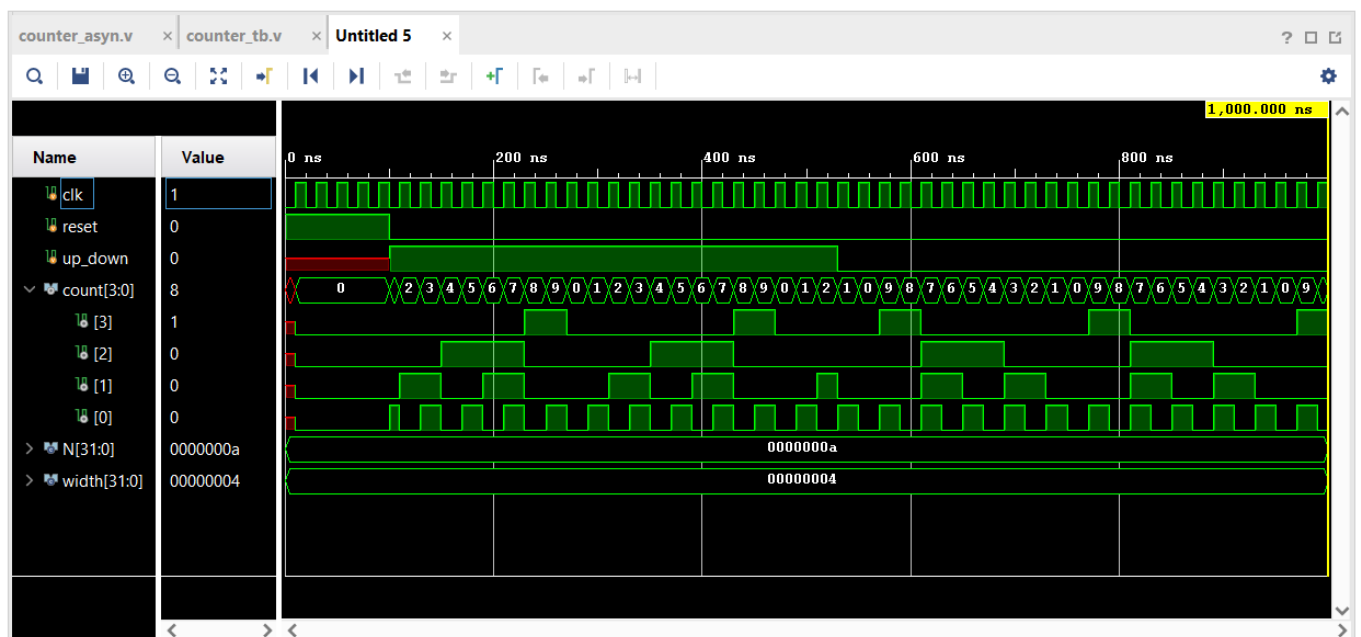
```
end
```

```
initial
```

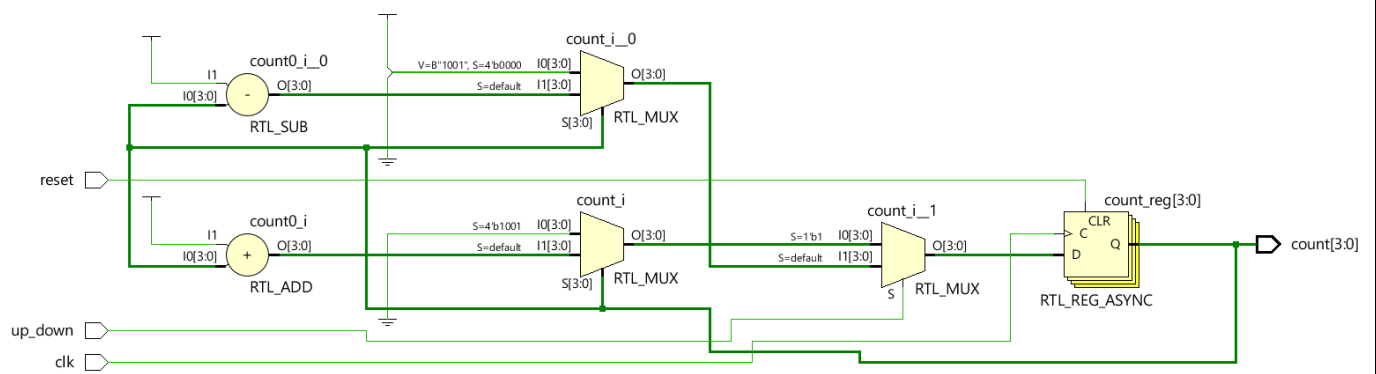
```
begin
```

```
initial
#1000 $finish;
endmodule
```

Simulation result for mod 5 counter



SYNTHESIS RESULTS:



Experiment 9

Design of Universal shift Register

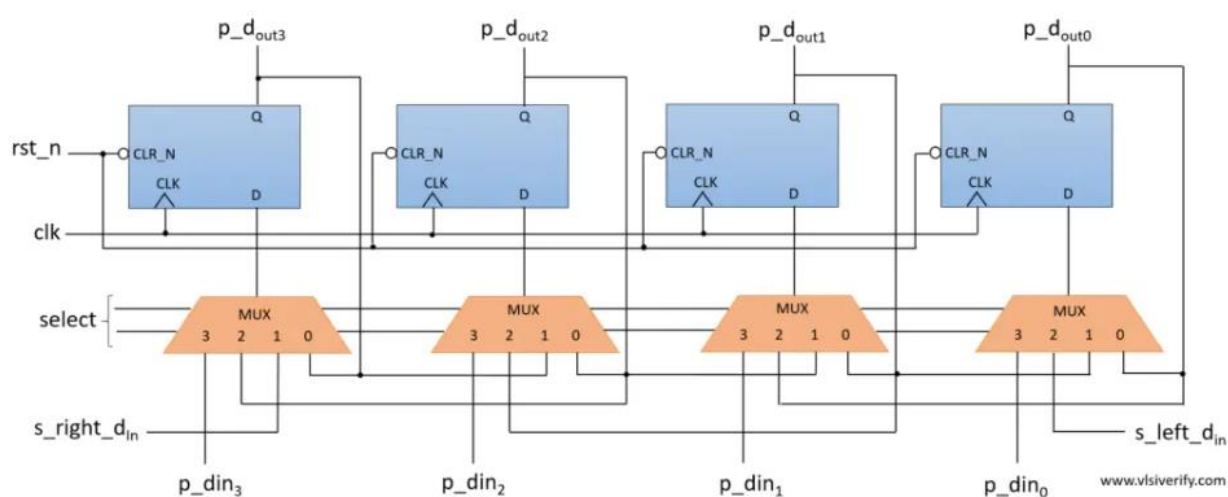
Aim: Design of Universal shift register and implementing of FPGA kit

A universal shift register is a sequential logic that can store data within and on every clock pulse it transfers data to the output port.

The universal shift register can be used as

1. Parallel In Parallel Out shift register
2. Parallel In Serial Out shift register
3. Serial In Parallel Out shift register
4. Serial In Serial Out shift register

Mode control (select)		Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load



VERILOG CODE FOR UNIVERSAL SHIFT REGISTER

```
module universal_shift_reg(
    input clk, rst_n,
    input [1:0] select, // select operation
    input [3:0] p_din, // parallel data in
    input s_left_din, // serial left data in
    input s_right_din, // serial right data in
    output reg [3:0] p_dout, //parallel data out
    output s_left_dout, // serial left data out
    output s_right_dout // serial right data out
);
always@(posedge clk) begin
    if(!rst_n) p_dout <= 0;
    else begin
        case(select)
            2'h1: p_dout <= {s_right_din,p_dout[3:1]}; // Right Shift
            2'h2: p_dout <= {p_dout[2:0],s_left_din}; // Left Shift
            2'h3: p_dout <= p_din; // Parallel in - Parallel out
            default: p_dout <= p_dout; // Do nothing
        endcase
    end
end
assign s_left_dout = p_dout[0];
assign s_right_dout = p_dout[3];
endmodule
```

TESTBENCH FOR UNIVERSAL SHIFT REGISTER

```
module USR_tb();
    reg clk, rst_n;
    reg [1:0] select;
    reg [3:0] p_din;
    reg s_left_din, s_right_din;
    wire [3:0] p_dout; //parallel data out
    wire s_left_dout, s_right_dout;

    universal_shift_reg usr(clk, rst_n, select, p_din, s_left_din, s_right_din, p_dout, s_left_dout,
s_right_dout);

    always #2 clk = ~clk;
    initial begin
        $monitor("select=%b, p_din=%b, s_left_din=%b, s_right_din=%b --> p_dout = %b, s_left_dout = %b,
s_right_dout = %b",select, p_din, s_left_din, s_right_din, p_dout, s_left_dout, s_right_dout);
        clk = 0; rst_n = 0;
        #3 rst_n = 1;

        p_din = 4'b1101;
        s_left_din = 1'b1;
        s_right_din = 1'b0;
    end
endmodule
```

```

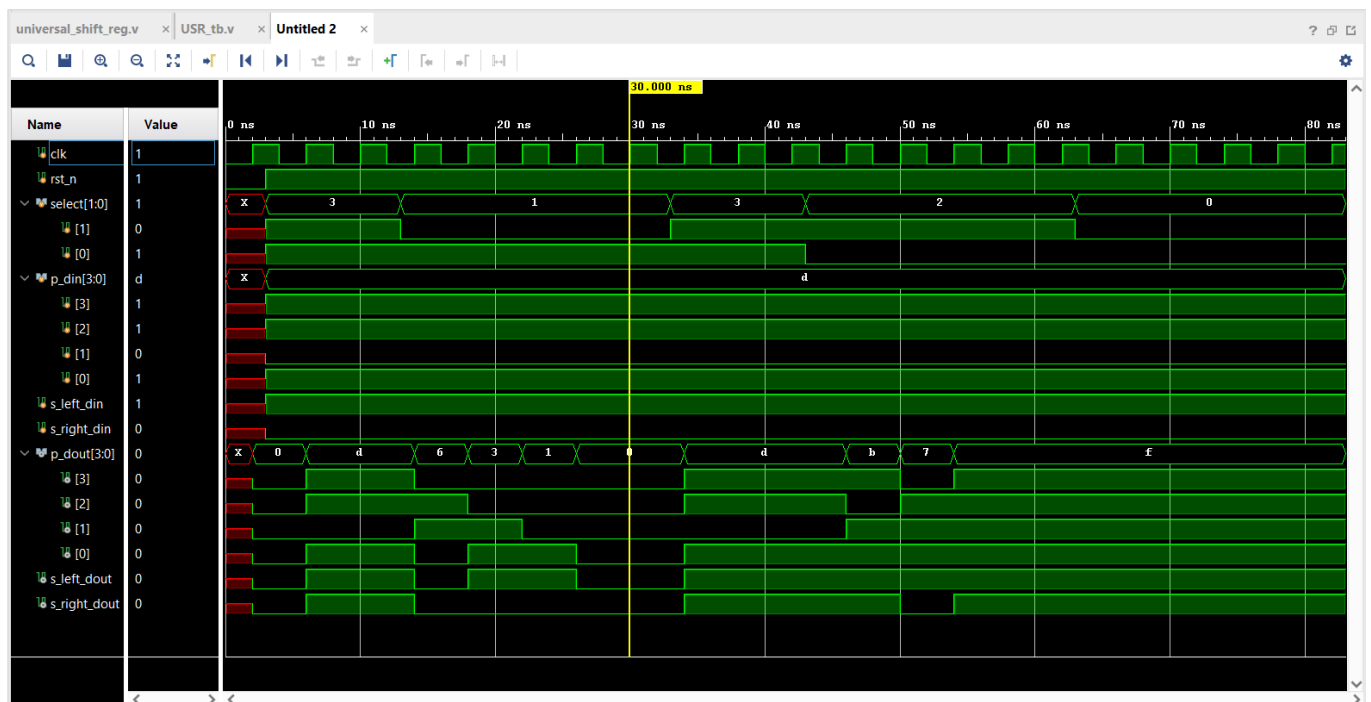
select = 2'h3; #10;
select = 2'h1; #20;
p_din = 4'b1101;
select = 2'h3; #10;
select = 2'h2; #20;
select = 2'h0; #20;

$finish;
end
// To enable waveform
initial begin
    $dumpfile("dump.vcd"); $dumpvars;
end

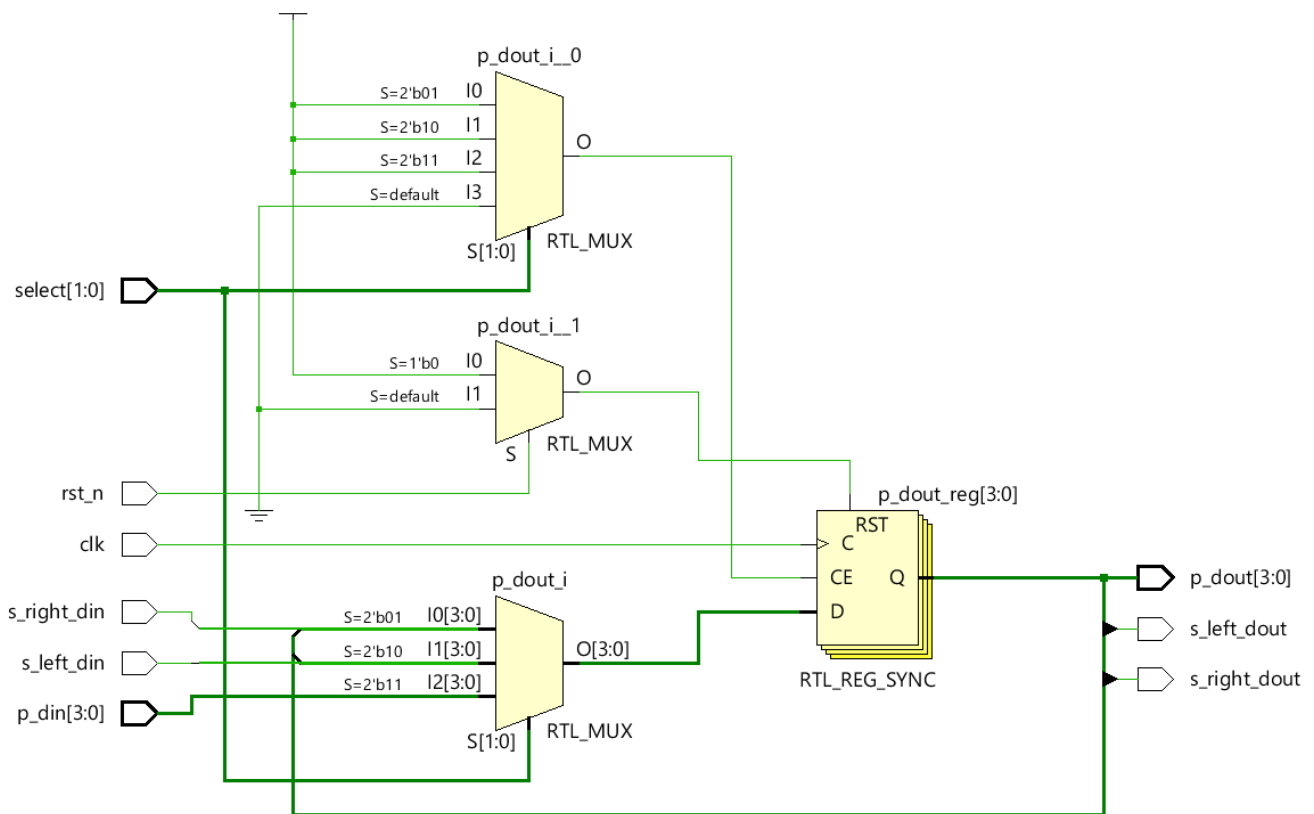
endmodule

```

SIMULATION RESULTS:



SYNTHESIS RESULTS:



Open Ended Experiment List (Any 2 can selected out of 5)

1. Design and implement with Verilog code for an unsigned adder circuit to add any two 8-digit BCD numbers. Justify the design with suitable examples.
2. Design and implement a digital clock to display HH:MM:SS using Verilog
3. Design and implement a 128 bits array multiplier using Verilog coding. Justify the design with suitable examples.
4. Design and implement sequentially operated traffic light control system with m seconds for pass and 4 Junctions
5. Design an elevator system 4 floors with various selection priority

Sample VIVA Question

1. What does wire refer to?

It is a physical connection between structural elements that enable Verilog to function. A continuous assignment or gate output denotes its value. A wire cannot store value when there is no connection between a and b. The Default value of a wire is Z.

2. What is reg in Verilog?

The reg represents the abstract data storage element. It is also called a register type integer, real and real-time. Its value is assigned within an always or an initial statement. The default value of reg is X.

3. What are Blocking and non-blocking in Verilog?

The blocking assignment completes the entire statement before the control goes to the following statement. It behaves similarly to older programming languages. It is symbolized as =.

A non-blocking assignment evaluates the right-hand side for the current time unit and the left-hand side later at the end of the time unit. It is symbolized as <=.

4. What is Verilog used for?

Uses of Verilog

To model electronic systems

Designing and verifying digital circuits

Verification of analog circuits and mixed-signal circuits

Designing genetic circuits

5. What are the data types in Verilog?

Data types are used to represent the data storage and transmission elements that are found in digital hardware. These are of 2 types NETS and REGISTERS.

6. What is Verilog?

Verilog is a Hardware Description Language (HDL) used for describing a digital system such as a network switch, a microprocessor, a memory, or a flip-flop. Verilog is mainly used to verify analog circuits, mixed-signal circuits, and the design of genetic circuits. It is also used in the design and verification of digital circuits at the register-transfer level of abstraction.

7. Who is the founder of the Verilog programming language?

Verilog was introduced by Prabhu Goel, Phil Moorby, Chi-Lai Huang, and Douglas Warmke between late 1983 and early 1984.

8. What is the difference between == and === in Verilog?

==	===
In Verilog, the main difference between == and === is that the output of == can be either 1, 0 or X.	On the other hand, the output of === can only be 0 or 1.
If you compare two numbers using == and if one or both the figures have one or more bits as X, then the output would be X.	If you are comparing two numbers using ===, the output would be 0 or 1.
== can only be used to compare 1s and 0s since it cannot compare Xs.	=== can also be used to compare Xs.

9. What is the difference between \$monitor and \$display?

\$monitor	\$display
The monitor is used to make changes in the value of the signal. A signal can be variable, string, or expression.	It is used to display the value of a signal.
The monitor command can be invoked only once.	The display command can be invoked more than once.

10. What are the main differences between Wire and Reg?

Wire	Reg
The wire is used to assume value.	Reg is used to hold value.
Wire requires drivers to get output values.	Reg does not need a driver to get output.
The wire elements can only be used to model combinational logic.	The reg elements can also be used for combinational as well as sequential logic.
We can use wire at the left-hand side of an assigned statement.	We cannot use reg on the left-hand side of an assigned statement.

11. What is sensitivity list?

The sensitivity list indicates that when a change occurs to any one of elements in the list change, begin...end statement inside that always block will get executed.

12. What are casex and casez statements used for in Verilog?

Casez allows matching one case item to multiple case expressions by not taking into consideration the Z bit. It is written as Z or ?.

For a case expression 2'b0, 2'b1, 2'bX, or 2'bZ will be matched to an item 2'bZ

Whenever comparing the case item and case expression, the don't care bit (represented by Z or ?) will be ignored completely irrespective of its value.

Casex is absolutely similar to casez with the addition of 'X' to the don't care bit. So here in casex 'Z', '?' and 'X' bits are ignored in case item and/or case expression while comparing.

13. What is the difference between Latch And Flip-flop?

The difference between latches and Flip-flop is that the latches are level triggered and flip-flops are edge triggered. In latches level triggered means that the output of the latches changes as we change the input and edge triggered means that control signal only changes its state when goes from low to high or high to low.

14. Explain the difference between Sequential and Combinational circuits.

Combinational Circuit	Sequential Circuit
The output depends on the present input only and there is no need for feedback for input and output, so a memory element is not required.	The output depends upon both the present input and the present state (previous output), so a memory element is required to save the feedback state.
Clock signals are not required and the combinational circuit is not dependent on time.	Clock signals are required and sequential circuits are dependent on time and clock so it needs triggering.
It is easier to use, design, and handle.	It is more complex to use, design, and handle.
Faster logic circuits.	Slower than combinational circuits.
Elementary building blocks are logic gates only.	Elementary building blocks are Flip-Flops.

15. What is the difference between Synchronous and Asynchronous Counters?

Synchronous Counter	Asynchronous Counter
All flip-flops are triggered with the same clock simultaneously.	Different flip-flops are triggered with different clocks, not simultaneously.
A synchronous Counter is faster than an asynchronous counter in operation.	The asynchronous Counter is slower than the synchronous counter in operation.
Also known as Parallel Counter.	Also known as Serial Counter.
Examples: Ring counter , Johnson counter .	Examples: Ripple UP counter and the Ripple DOWN counter.
In synchronous counter, propagation delay is less.	In the asynchronous counter, there is a high propagation delay.